



Universidad Internacional de la Rioja (UNIR)

Escuela Superior de Ingeniería y Tecnología

Máster Universitario en Análisis y Visualización de Datos Masivos

ProcureWatch Analytics:
Plataforma Multiagente para la Detección de Patrones de Riesgo en Contratación Pública mediante Open Data, Análisis de Redes, NLP y Modelos Locales

Trabajo Fin de Estudios

presentado por: Sebastián Alfonso Correa y Saturía María Hurtado Álvarez

Dirigido por: Enrique de Miguel Ambite

Fecha: 20 de mayo de 2026

Índice de Contenidos

Resumen	IX
Abstract	X
1. Introducción	1
1.1. Motivación y justificación del tema	1
1.2. Planteamiento del trabajo	1
1.3. Estructura del trabajo	2
2. Análisis del Contexto	3
2.1. Dimensión económica de la contratación pública en España	3
2.2. Pregunta 1: Volumen de contratos en PLACE y organismos participantes .	4
2.3. Pregunta 2: Tipos de irregularidades documentadas en España (2014–2024)	4
2.4. Pregunta 3: Limitaciones de los sistemas actuales de auditoría y control .	6
2.5. Pregunta 4: Evidencia de eficacia del análisis de datos para detectar riesgo .	7
2.6. Dominio de análisis seleccionado: CPV 71 — Servicios de arquitectura, in- geniería e inspección	8
2.7. Brecha identificada: posicionamiento del proyecto	9
3. Contexto y Estado del Arte	10
3.1. Introducción: criterios de inclusión y organización	10
3.2. Bloque A: Open Data de contratación pública y estándar OCDS	10
3.2.1. El estándar OCDS como eje de integración	10
3.2.2. Fuentes de datos europeas y plataformas de referencia	11
3.3. Bloque B: Red flags, indicadores de riesgo y machine learning	11
3.3.1. Taxonomía de red flags en contratación pública	11
3.3.2. Machine learning aplicado a la detección de fraude	12
3.3.3. Positive-Unlabeled Learning	12
3.3.4. Detección de anomalías no supervisada	12
3.4. Bloque C: Análisis de redes en contratación pública	13
3.4.1. Grafos bipartitos empresa-organismo	13
3.4.2. Detección de comunidades: algoritmos Louvain y Leiden	13

3.5.	Bloque D: NLP y procesamiento de documentos de contratación	14
3.5.1.	Named Entity Recognition en documentos legales	14
3.5.2.	Herramientas NLP para español: spaCy y transformers	14
3.5.3.	Web scraping: extracción estructurada de documentos administrativos	15
3.5.4.	LLMs locales para análisis jurídico-administrativo	15
3.6.	Bloque E: Visualización analítica y explicabilidad	15
3.6.1.	Explicabilidad en modelos de scoring de riesgo	15
3.6.2.	Visualización de grafos interactivos	16
3.6.3.	Dashboards de riesgo	16
3.7.	Conclusiones del estado del arte y justificación del proyecto	16
4.	Objetivos	18
4.1.	Objetivo General	18
4.2.	Objetivos Específicos	18
5.	Marco Normativo	21
5.1.	Ley 9/2017, de 8 de noviembre, de Contratos del Sector Público (LCSP) . .	21
5.2.	Directiva 2014/24/UE del Parlamento Europeo y del Consejo, de 26 de febrero de 2014	22
5.3.	Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno	22
5.4.	RGPD (Reglamento (UE) 2016/679) y LOPDGDD (Ley Orgánica 3/2018) .	23
5.5.	Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024 (AI Act)	24
5.5.1.	Clasificación del sistema como de alto riesgo	24
5.5.2.	Implicaciones específicas para el motor de <i>red flags</i> y <i>scoring</i>	25
5.5.3.	Consecuencias del soporte opcional de un LLM local	25
5.6.	Otras normativas de referencia	26
5.7.	Síntesis de implicaciones para el diseño de ProcureWatch Analytics	26
5.8.	Conclusión del marco normativo	26
6.	Metodología y diseño del sistema	28
6.1.	Visión general	28
6.2.	Criterios metodológicos del prototipo funcional	29

6.3.	Alcance evaluable y alcance extensible	30
6.4.	Stack tecnológico de la versión evaluada	31
6.5.	Justificación del uso de IA y LLM	34
6.6.	Pipeline de datos	34
6.6.1.	Fases del pipeline	35
6.6.2.	Modelo de datos analítico	35
6.7.	Motor de red flags y scoring	36
6.8.	Grafo de contratación	37
6.9.	Módulo documental, embeddings y RAG	38
7.	Implementación por agentes y componentes	39
7.1.	Implementación multiagente	39
7.2.	Interfaz analítica	41
7.3.	Protocolo de revisión humana asistida	41
7.4.	Diseño de visualización y explicabilidad	42
7.5.	Trazabilidad, calidad y reproducibilidad	43
7.6.	Orquestación batch, planificación reproducible y evidencia operativa	44
7.7.	Cierre integrado de Agent3 y Agent4	44
7.8.	Implementación final del prototipo	45
7.9.	Flujo extremo a extremo del prototipo	47
7.10.	Contratos de datos entre agentes	48
7.11.	Implementación detallada de los agentes	50
7.11.1.	Agent1: ingesta, normalización y canónico	50
7.11.2.	Agent2: red flags, score y priorización	51
7.11.3.	Agent3: grafo derivado y variables relacionales	51
7.11.4.	Agent4: evidencia documental y ficha de caso	52
7.11.5.	Separación entre capa analítica y capa de demostración	53
7.12.	Política de clave canónica y trazabilidad del cruce	53
7.12.1.	Trazabilidad cuando no hay intersecciones	54
7.13.	Gobernanza de evidencias y revisión humana	55
7.14.	Evaluación por objetivos específicos	55
7.15.	Decisiones de diseño por agente	57
7.15.1.	Agent1: priorizar trazabilidad frente a completitud aparente	57

7.15.2. Agent2: scoring explicable antes que modelo opaco	57
7.15.3. Agent3: grafo derivado y no fuente primaria	58
7.15.4. Agent4: evidencia citada y control de alucinaciones	58
7.16. Controles de calidad y riesgos técnicos	59
7.17. Lectura operativa de los resultados	60
8. Orquestación, reproducibilidad y artefactos	61
8.1. Reproducibilidad y gestión de artefactos	61
8.2. Estrategia batch de la versión de cierre	61
8.3. Manifiestos, hashes y limpieza de temporales	62
8.4. Comandos de ejecución y reproducción	63
8.5. Artefactos generados por el sistema	65
8.6. Fragmentos de implementación relevantes	67
8.6.1. Esquema analítico ejecutable	67
8.6.2. Construcción conservadora de <code>contract_key_canon</code>	68
8.6.3. Red flags y score de Agent2	69
8.6.4. Batch incremental y estado <code>skipped</code>	71
8.6.5. Consulta de vecindario en Neo4j	71
9. Resultados y evaluación	73
9.1. Resultados de ingesta y normalización	74
9.2. Calidad del canónico y cobertura entre fuentes	75
9.3. Resultados de red flags y scoring	77
9.4. Evaluación cualitativa de diez fichas	80
9.5. Resultados de grafo, RAG y dashboard	82
9.6. Evaluación técnica y tests	84
9.7. Matriz de evidencias de evaluación	85
9.8. Lectura de evaluación por agente	87
9.9. Interpretación de la evaluación proxy	88
10. Limitaciones, Adaptación del Alcance y Trabajo Futuro	90
10.1. Adaptación del alcance y grado de cumplimiento de los objetivos	90
10.2. Impacto de las limitaciones por componente	92
10.3. Limitaciones de datos	93

10.4. Limitaciones de modelado y evaluación	94
10.5. Limitaciones operativas	95
10.6. Trabajo futuro	95
11. Conclusiones	97
Referencias	99
A. Apéndice: Tipos de contrato incluidos en el análisis	106
B. Apéndice: Estructura técnica del proyecto	107
C. Apéndice: Esquema analítico resumido	108
D. Apéndice: Ejemplo de report y manifest	109
E. Apéndice: Comandos de validación	111

Índice de Ilustraciones

6.1. Arquitectura del prototipo reproducible de ProcureWatch Analytics	29
7.1. Flujo de trazabilidad desde la fuente hasta la ficha explicativa.	40

Índice de Tablas

2.1. Volumen y cobertura de datos disponibles para contratación pública en España y Europa	3
2.2. Gap identificado en la literatura que justifica ProcureWatch Analytics . . .	9
5.1. Requisitos normativos e implementación en ProcureWatch Analytics	27
6.1. Separación entre alcance evaluable y alcance extensible	31
6.2. Stack tecnológico implementado y extensiones opcionales de ProcureWatch Analytics	32
6.3. Modelo de datos analítico propuesto	36
7.1. Agentes previstos en ProcureWatch Analytics	39
7.2. Protocolo de revisión humana asistida	42
7.3. Componentes implementados en ProcureWatch Analytics	45
7.4. Flujo extremo a extremo del prototipo	47
7.5. Contratos de datos entre agentes del prototipo	49
7.6. Política funcional de <code>contract_key_canon</code>	54
7.7. Evaluación de objetivos específicos frente a la implementación	56
7.8. Riesgos técnicos y controles aplicados	59
8.1. Modos de ejecución batch del prototipo	62
8.2. Comandos disponibles en la CLI de ProcureWatch	63
8.3. Artefactos principales producidos por ProcureWatch	65
9.1. Matriz de alcance evaluado en el benchmark final del TFM	73
9.2. Filas observadas en artefactos generados por Agent1	74
9.3. Métricas de calidad de Agent1 en la ejecución disponible	76
9.4. Distribución de riesgo de Agent2 sobre 3.437 contratos	77
9.5. Evaluabilidad por regla de Agent2 en el escenario base	78
9.6. Red flags implementadas en Agent2	78
9.7. Sensibilidad de Agent2 ante variación de umbrales	80
9.8. Selección de diez fichas cualitativas	81
9.9. Importes, procedimiento y contexto relacional de las diez fichas	81

9.10. Evaluación práctica de Agent4 para fichas trazables	83
9.11. Resumen de pruebas automatizadas existentes	85
9.12. Matriz de evidencias utilizadas en la evaluación del prototipo	86
10.1. Correspondencia entre alcance previsto e implementación final	90
10.2. Impacto de las limitaciones sobre los componentes del prototipo	92
A.1. Subtipos CPV 71 relevantes para ProcureWatch Analytics	106
B.1. Estructura funcional del proyecto de implementación	107
C.1. Bloques funcionales del esquema CONTRATO	108

Resumen

ProcureWatch Analytics es una plataforma analítica multiagente orientada a identificar patrones de riesgo en contratación pública española mediante análisis integrado de datos abiertos, un esquema OCDS adaptado, indicadores de red flags, análisis de redes, procesamiento documental y generación de fichas explicativas. El prototipo se aplica al dominio CPV 71 (Servicios de arquitectura, ingeniería e inspección) y combina datos BOE, PLACE y OpenTender cuando las fuentes disponibles lo permiten. La evaluación se basa en métricas de calidad de datos, trazabilidad, consistencia de reglas, pruebas automatizadas y validación exploratoria del flujo multiagente. El sistema no declara fraude ni sustituye la revisión experta: prioriza contratos y adjudicatarios para revisión humana, mostrando las señales activadas, las métricas de red y las evidencias documentales disponibles.

Palabras clave: contratación pública, red flags, scoring explicable, análisis de redes, NLP, datos abiertos, CPV 71, OCDS, priorización de riesgo, plataforma multiagente.

Abstract

ProcureWatch Analytics is a multi-agent analytical platform designed to identify risk patterns in Spanish public procurement through open data integration, an adapted OCDS analytical schema, red flag indicators, network analysis, document processing and explainable case reports. The prototype focuses on CPV division 71 (Architecture, Engineering and Inspection Services) and combines BOE, PLACE and OpenTender data whenever the available sources allow it. The evaluation relies on data-quality metrics, traceability, rule consistency, automated tests and exploratory validation of the integrated multi-agent workflow. The system does not declare fraud or replace expert assessment: it prioritises contracts and suppliers for human review, exposing the activated signals, network metrics and available documentary evidence.

Keywords: public procurement, red flags, explainable scoring, network analysis, NLP, open data, CPV 71, OCDS, risk prioritisation, multi-agent platform.

1. Introducción

1.1. Motivación y justificación del tema

La contratación pública es uno de los mecanismos de gasto público de mayor volumen económico y mayor exposición al riesgo de irregularidad. En España representa entre el 12 y el 15 % del PIB nacional (OECD, 2016), y desde 2007 la legislación obliga a publicar contratos en formatos abiertos y reutilizables (Jefatura del Estado, 2017). Pese a esta disponibilidad, los análisis sistemáticos, automatizados y reproducibles sobre datos de contratación pública en España siguen siendo escasos.

El reciente *dataset* longitudinal del BOE 2014–2024 (Muñoz Plà, 2026), publicado con aproximadamente 97.000 contratos estructurados según el estándar OCDS, abre una oportunidad concreta para construir análisis de calidad sobre este dominio. La literatura internacional ha demostrado de forma consistente que los métodos de machine learning y análisis de redes mejoran significativamente la detección de irregularidades frente a enfoques puramente basados en reglas explícitas (Wachs, Fazekas, y Kertész, 2025; Fazekas y Cingolani, 2020; Coviello y Mariniello, 2023).

1.2. Planteamiento del trabajo

Este TFM propone diseñar, desarrollar y evaluar **ProcureWatch Analytics**, una plataforma multiagente que integra: (a) un *pipeline* de adquisición y normalización de datos abiertos en un esquema OCDS adaptado; (b) un motor de priorización basado en reglas deterministas y explicables RF-01 a RF-06; (c) un grafo derivado de relaciones entre compradores, contratos, adjudicatarios, CPV y fuentes; (d) un componente documental con fragmentación, recuperación, citas y fallback determinista; y (e) un dashboard Streamlit con ranking, detalle de caso, relaciones, evidencias y trazabilidad. El enriquecimiento societario, los modelos de anomalías y el OCR masivo se mantienen como extensiones y no como resultados cerrados.

El prototipo se valida sobre el dominio CPV 71 (Servicios de arquitectura, ingeniería e inspección), seleccionado por razones metodológicas detalladas en la Sección 2.6.

1.3. Estructura del trabajo

- **Capítulo 2** — Análisis del contexto: descripción del problema, datos justificativos, preguntas de investigación y dominio de análisis.
- **Capítulo 3** — Contexto y estado del arte: revisión de los cinco bloques temáticos con referencias académicas y conclusiones sobre la brecha que justifica el proyecto.
- **Capítulo 4** — Objetivos: objetivo general SMART y ocho objetivos específicos medibles y operativos.
- **Capítulo 5** — Marco normativo aplicable a datos abiertos, contratación pública, protección de datos y sistemas de IA.
- **Capítulo 6** — Metodología y diseño del sistema, incluyendo arquitectura, modelo de datos, agentes y criterios de evaluación.
- **Capítulo 7** — Implementación del prototipo multiagente y descripción técnica de sus componentes.
- **Capítulo 8** — Reproducibilidad, artefactos, comandos de ejecución, manifests, hashes y trazabilidad.
- **Capítulo 9** — Resultados y evaluación técnica del prototipo.
- **Capítulo 10** — Limitaciones, adaptación del alcance y trabajo futuro.
- **Capítulo 11** — Conclusiones del TFM.

Nota: el planteamiento metodológico detallado (fases, cronograma, entregables) se presenta en el documento independiente `metodologia_tfm.pdf`.

2. Análisis del Contexto

2.1. Dimensión económica de la contratación pública en España

La contratación pública en España constituye uno de los mecanismos de gasto público de mayor impacto económico y social. Representa entre el **12 y el 15 % del PIB nacional** (OECD, 2016), movilizando anualmente decenas de miles de millones de euros en obras, servicios y suministros adjudicados por miles de organismos públicos.

Esta magnitud, sumada a la complejidad de los procedimientos contractuales y a la asimetría de información entre licitadores y organismos, convierte la contratación pública en un ámbito especialmente expuesto al riesgo de irregularidad. Al mismo tiempo, la obligación legal de publicar los contratos en formatos abiertos desde 2007 (Jefatura del Estado, 2017, 2013) genera una base de datos extraordinariamente rica para el análisis de patrones mediante técnicas de datos masivos.

Tabla 2.1: Volumen y cobertura de datos disponibles para contratación pública en España y Europa

Fuente	Volumen	Referencia
Licitaciones anuales PLACE (2024, sin menores)	206.242	(Oficina Independiente de Regulación y Supervisión de la Contratación, 2025)
Contratos BOE 2014–2024 (formato OCDS)	~97.000	(Muñoz Plà, 2026)
Contratos CPV 71 en dataset BOE	3.443	(Muñoz Plà, 2026)
Órganos de contratación activos en PLACE	>10.000	(Ministerio de Hacienda y Función Pública, 2024)
Contratos europeos en OpenTender	>19 millones	(Government Transparency Institute, 2024)
Red flags documentados (OCP + DIGIWHIST)	>150 indicadores	(Open Contracting Partnership, 2024b)

2.2. Pregunta 1: Volumen de contratos en PLACE y organismos participantes

La Plataforma de Contratación del Sector Público (PLACSP) es el principal sistema de publicación de la contratación pública en España, integrando datos de la Administración General del Estado, comunidades autónomas, entidades locales y el sector público institucional (Ministerio de Hacienda y Función Pública, 2024).

Según los informes de la Oficina Independiente de Regulación y Supervisión de la Contratación (OIReScon), en el ejercicio 2024 se registraron **206.242 licitaciones** en la plataforma, sin incluir contratos menores, lo que representa una parte sustancial del total de la contratación pública gestionada en España (Oficina Independiente de Regulación y Supervisión de la Contratación, 2025). Si se consideran contratos menores, adjudicaciones y desgloses por lotes, el volumen total de registros anuales asciende a varios millones, aunque esta cifra varía según el criterio de contabilización (contrato, expediente o lote).

En cuanto a los organismos participantes, la plataforma da servicio a **más de 10.000 órganos de contratación activos** pertenecientes a todos los niveles administrativos: Administración General del Estado, comunidades autónomas, diputaciones, ayuntamientos, universidades y entidades del sector público institucional (Ministerio de Hacienda y Función Pública, 2024). Esta fragmentación administrativa supone uno de los principales retos para el análisis sistemático, ya que los datos están distribuidos en múltiples fuentes con formatos y calidades heterogéneas (Open Contracting Partnership, 2023).

2.3. Pregunta 2: Tipos de irregularidades documentadas en España (2014–2024)

La literatura y los informes de supervisión han identificado múltiples tipos de irregularidades en la contratación pública española durante la última década. Las más frecuentes son:

Fraccionamiento indebido de contratos. División artificial de contratos para eludir los umbrales de publicidad y concurrencia. Es uno de los indicadores con mayor soporte empírico en la literatura (Open Contracting Partnership, 2024b, 2016).

Uso inadecuado del contrato menor. Recurso sistemático al procedimiento negocia-

do sin publicidad —que no exige concurrencia— con el mismo proveedor o en importes próximos al umbral legal (Oficina Independiente de Regulación y Supervisión de la Contratación, 2025). Los informes OIREscon han documentado un incremento de este tipo de irregularidades.

Falta de transparencia y publicidad. Procedimientos negociados sin publicidad en contratos que superan umbrales razonables, contratos publicados fuera del horario administrativo habitual o en periodos de menor visibilidad (festivos, cierres de ejercicio).

Conflictos de interés y adjudicaciones dirigidas. Caso emblemático: la operación de la Agencia Tributaria en Cantabria (2023), donde se detectaron adjudicaciones sistemáticamente dirigidas a empresas vinculadas a un funcionario responsable de los informes técnicos de valoración (Agencia Tributaria de España, 2023). La AEAT ha señalado explícitamente el cruce entre datos tributarios y contractuales como línea prioritaria de detección.

Modificaciones contractuales injustificadas. Alteraciones del objeto, plazo o importe de contratos ya adjudicados, sin fundamento legal suficiente.

Prácticas colusorias. Empresas adjudicatarias que comparten administradores, domicilios sociales o estructura accionarial, lo que puede indicar coordinación en las ofertas (colusión) o la existencia de grupos empresariales que se presentan formalmente como competidores (Open Contracting Partnership, 2024b).

Sobrecoste y desviaciones de importe. Diferencias sistemáticamente elevadas entre el importe estimado de licitación y el importe real adjudicado, lo que puede indicar estimaciones deliberadamente infladas o reducidas para condicionar la concurrencia (Coviello y Mariniello, 2023).

Los informes recientes de OIREscon (Oficina Independiente de Regulación y Supervisión de la Contratación, 2025) señalan además un aumento significativo de denuncias relacionadas con irregularidades en la adjudicación. En el ámbito académico, Falcón-Cortés, Aldana, y Larralde (2021) demuestran que la concentración de adjudicaciones y la baja concurrencia son los indicadores más predictivos de comportamiento irregular.

2.4. Pregunta 3: Limitaciones de los sistemas actuales de auditoría y control

Los sistemas de control de la contratación pública presentan diversas limitaciones estructurales que justifican el desarrollo de herramientas analíticas avanzadas:

1. **Fragmentación de información.** Los datos están distribuidos entre múltiples plataformas y niveles administrativos, dificultando un análisis global y consistente (Oficina Independiente de Regulación y Supervisión de la Contratación, 2025).
2. **Insuficiencia de recursos de supervisión.** El volumen de contratos gestionados anualmente (>200.000 licitaciones) excede ampliamente la capacidad de revisión exhaustiva con los medios actuales (Oficina Independiente de Regulación y Supervisión de la Contratación, 2025).
3. **Falta de estandarización y calidad homogénea.** La ausencia de un estándar universal en los formatos de publicación introduce heterogeneidad que dificulta el análisis automatizado (Open Contracting Partnership, 2023).
4. **Carácter reactivo.** Los sistemas de control actúan principalmente una vez detectada la irregularidad, en lugar de prevenirla mediante análisis predictivo (OECD, 2016). Este enfoque reactivo limita la eficacia del sistema.
5. **Ausencia de perspectiva relacional.** Los controles tradicionales no detectan comunidades de proveedores con comportamientos coordinados, ni patrones de concentración atípica que sólo son visibles en el grafo de relaciones empresa-organismo (Wachs y cols., 2025; Martins y cols., 2022).
6. **Invisibilidad del contexto documental.** Los pliegos de prescripciones técnicas, memorias justificativas y resoluciones contienen señales textuales —direccionamientos técnicos, criterios de adjudicación subjetivos— que permanecen invisibles en análisis puramente estructurados (Kalamkar y cols., 2024).

2.5. Pregunta 4: Evidencia de eficacia del análisis de datos para detectar riesgo

La literatura científica reciente ofrece evidencias sólidas sobre la eficacia de los métodos de análisis de datos para la detección de patrones de riesgo en contratación pública:

- Coviello y Mariniello (2023) demuestran, en un estudio empírico con datos de licitaciones italianas, que los indicadores de red flags alcanzan precisiones superiores al 70 % en detección de contratos con corrupción confirmada, especialmente cuando se combinan indicadores de concurrencia, importe y procedimiento.
- Wachs y cols. (2025) aplican un enfoque combinado de machine learning y análisis de redes a datos de contratación en México, demostrando que las métricas de red (centralidad, comunidades) añaden poder predictivo significativo respecto a los modelos tabulares puros.
- El proyecto europeo DIGIWHIST (Fazekas y Cingolani, 2020; Government Transparency Institute, 2021) desarrolló el *Corruption Risk Index* (CRI) combinando cuatro indicadores observables —riesgo de licitación, conexiones políticas, riesgo del proveedor y riesgo del órgano contratante— con validación sobre datos de 35 jurisdicciones europeas.
- Liu y cols. (2023) proponen el framework PANG (*Pattern-based Anomaly detection in Graphs*) aplicado específicamente a contratación pública, obteniendo mejoras significativas sobre métodos basados en reglas.
- Muñoz Plà (2026) señalan que el *dataset* longitudinal del BOE 2014–2024 permite análisis de patrones estructurales y temporales del riesgo que refuerzan la utilidad del análisis de datos como herramienta de supervisión preventiva.

En conjunto, la evidencia sugiere que la combinación de indicadores de red flags, análisis de grafos y NLP mejora sustancialmente la capacidad de detección temprana respecto a los enfoques de auditoría tradicional.

2.6. Dominio de análisis seleccionado: CPV 71 — Servicios de arquitectura, ingeniería e inspección

Tras el análisis exploratorio del *dataset* BOE 2014–2024, se ha seleccionado la **división CPV 71** (Servicios de arquitectura, construcción, ingeniería e inspección) como dominio de validación del prototipo ProcureWatch Analytics (Portal de Contratación Pública, 2026). Esta división comprende **3.443 contratos adjudicados** con un importe total adjudicado superior a **2.179 millones de euros**, distribuidos entre 394 organismos contratantes y 2.031 empresas adjudicatarias distintas.

La elección se fundamenta en tres criterios metodológicos:

Validez de indicadores de riesgo tabulares. A diferencia del CPV 45 (Obras, $n = 11.808$), los servicios de ingeniería presentan menor incidencia de modificaciones contractuales legalmente justificadas. En obras, las desviaciones de importe son frecuentes y a menudo legítimas, lo que introduce ruido en los modelos de detección. Los servicios de ingeniería —mayoritariamente estructurados en tarifas horarias y entregables definidos— ofrecen una base más estable para indicadores como la desviación de importe (RF-05) o los patrones temporales anómalos cuando existen fechas suficientes (RF-06) (Open Contracting Partnership, 2024b).

Riqueza para el análisis de redes. El mercado de la ingeniería pública presenta una estructura fragmentada pero interconectada: con 2.031 empresas para 3.443 contratos (ratio 1,7), la concentración del Top 1 es inferior al 1 %, lo que sugiere la existencia de nichos de concentración por organismo o subcategoría CPV sólo detectables mediante análisis de grafos. Además, el sector presenta alta propensión a la formación de Uniones Temporales de Empresas (UTEs), que enriquecen el grafo bipartito con relaciones de colaboración recurrente entre competidores (Wachs y cols., 2025; Traag, Waltman, y van Eck, 2019).

Adecuación al módulo NLP. Los pliegos de prescripciones técnicas en el ámbito de la ingeniería contienen con frecuencia criterios de adjudicación sujetos a juicio de valor —experiencia en proyectos similares, metodología propuesta, cualificación del equipo— que pueden representar hasta el 49 % de la puntuación total. En la versión evaluada, el módulo documental trabaja con anuncios, evidencias disponibles y fichas explicativas; el procesamiento masivo de PDFs y OCR queda como ampliación

natural del corpus. La especificidad del vocabulario (códigos CPV de 8 dígitos como 71311000 –Consultoría en ingeniería civil– o 71356000 –Servicios técnicos–) permite segmentación precisa que reduce la varianza espúria en la comparación de importes (Portal de Contratación Pública, 2026).

Esta aproximación sectorial está alineada con la metodología del proyecto europeo DIGIWHIST (Government Transparency Institute, 2021), que segmenta el análisis por mercados específicos para garantizar la comparabilidad de los indicadores de riesgo.

2.7. Brecha identificada: posicionamiento del proyecto

Tabla 2.2: Gap identificado en la literatura que justifica ProcureWatch Analytics

Dimensión	Estado actual	ProcureWatch
Red flags en literatura	Bien documentados (Open Contracting Partnership, 2024b)	Prototipo con reglas trazables y evaluación proxy
ML para detección fraude	Demostrado en Italia, México, Kenya	Comparativa exploratoria adaptada a datos españoles
Análisis de redes	Probado en Portugal, Europa	Aplicado al dominio CPV 71 en la versión evaluada
NLP en contratación	Survey reciente, sin prototipo	Módulo documental con evidencias y fichas explicativas
Plataforma integrada	No existe en datos españoles	Prototipo integrado de ProcureWatch Analytics
Reproducibilidad	Baja en estudios existentes	Código versionado, CLI, Docker y artefactos trazables

3. Contexto y Estado del Arte

3.1. Introducción: criterios de inclusión y organización

La revisión del estado del arte cubre cinco bloques temáticos alineados con los componentes técnicos de ProcureWatch Analytics: (A) datos abiertos y estándar OCDS; (B) red flags, indicadores de riesgo y machine learning; (C) análisis de redes en contratación pública; (D) NLP y procesamiento de documentos contractuales; y (E) visualización analítica y explicabilidad. En cada bloque se identifican las referencias más relevantes de revistas indexadas (Springer Nature, Nature, IEEE, ACM) con especial atención al periodo 2019–2026.

3.2. Bloque A: Open Data de contratación pública y estándar OCDS

3.2.1. El estándar OCDS como eje de integración

El Open Contracting Data Standard (OCDS) (Open Contracting Partnership, 2024a) es el marco de referencia internacional para la estructuración de datos de contratación pública. Define más de 40 campos por contrato —desde el anuncio de licitación hasta el pago— y ha sido adoptado por más de 60 países. El *World Bank* (World Bank, 2021) ha publicado guías de implementación que detallan cómo armonizar datos nacionales con el estándar. En España, el *dataset* BOE 2014–2024 (Muñoz Plà, 2026) constituye la primera base de datos longitudinal adaptada a OCDS, con 97.000 contratos estructurados que cubren diez años de adjudicaciones.

El informe de la Open Contracting Partnership (OCP) sobre la apertura de datos en la UE (Open Contracting Partnership, 2023) señala que, si bien España ha avanzado en la publicación de datos, subsisten problemas de completitud en campos críticos (NIF del adjudicatario, importe adjudicado, fecha de adjudicación) y de inconsistencias entre lotes y expedientes. Estos hallazgos definen el punto de partida del Agente 1 (pipeline de datos) de ProcureWatch.

3.2.2. Fuentes de datos europeas y plataformas de referencia

A nivel europeo, la plataforma OpenTender.eu (Government Transparency Institute, 2024; Fazekas y Cingolani, 2025) centraliza datos de 35 jurisdicciones, con más de 19 millones de contratos e indicadores de riesgo precalculados. Para España incluye datos desde 2007 en formato JSON con campos OCDS. El portal TED (*Tenders Electronic Daily*) recoge contratos públicos de la UE por encima de umbrales comunitarios, con cobertura desde 1994.

Estas fuentes complementan el *dataset* BOE y los datos de PLACE, permitiendo comparativas de riesgo en contexto europeo alineado con la metodología DIGIWHIST (Fazekas y Cingolani, 2020).

3.3. Bloque B: Red flags, indicadores de riesgo y machine learning

3.3.1. Taxonomía de red flags en contratación pública

La Open Contracting Partnership (Open Contracting Partnership, 2024b) ha publicado la guía de referencia más actualizada sobre red flags, definiendo más de 150 indicadores organizados por fase del ciclo contractual (planificación, licitación, adjudicación, ejecución). Los principales tipos de red flags implementables sobre datos OCDS son:

- **Licitación única o restringida:** contrato con una sola oferta en procedimiento abierto, o uso sistemático del procedimiento negociado sin publicidad por encima de umbrales razonables (Open Contracting Partnership, 2016).
- **Concentración de adjudicatarios:** un proveedor acumula de forma recurrente un porcentaje elevado de contratos de un organismo (Fazekas y Cingolani, 2020).
- **Desviación de importe:** diferencia sistemáticamente alta entre importe estimado y adjudicado (Coviello y Mariniello, 2023).
- **Red flags temporales:** publicación y adjudicación en plazos muy cortos, o en periodos de menor visibilidad (Open Contracting Partnership, 2024b).
- **Procedimiento menor recurrente:** uso del contrato menor con el mismo proveedor o en importes próximos al umbral (Oficina Independiente de Regulación y

Supervisión de la Contratación, 2025).

- **Red flags combinados:** los modelos que combinan múltiples indicadores mejoran significativamente respecto al indicador único (Wachs y cols., 2025).

3.3.2. Machine learning aplicado a la detección de fraude

Coviello y Mariniello (2023) presentan el estudio empírico más sólido en contexto europeo: utilizando datos de licitaciones italianas y modelos de *random forest* con indicadores de red flags como *features*, obtienen precisiones del 70–75 % en la clasificación de contratos corruptos. Su análisis identifica la licitación única y la concentración como los indicadores con mayor poder predictivo.

Ng'ang'a (2025) proponen un framework híbrido (*Logistic Regression + Random Forest + clustering*) para datos de contratación en Kenia, demostrando que la combinación de modelos supera a los enfoques individuales. Almeida y cols. (2025) combinan machine learning y análisis de redes sociales, obteniendo mejoras adicionales cuando se incorporan métricas de grafo como *features*.

El framework PANG de Liu y cols. (2023) es específicamente relevante: aplica *pattern mining* sobre grafos de contratación para detectar subgrafos anómalos, logrando detectar tramas de adjudicaciones coordinadas no detectables con indicadores tabulares.

3.3.3. Positive-Unlabeled Learning

Un desafío metodológico central en detección de fraude contractual es la ausencia de etiquetas negativas claras: disponemos de algunos casos confirmados de irregularidad (etiquetas positivas), pero la mayoría de contratos no tienen etiqueta. El enfoque *Positive-Unlabeled Learning* (PUL) (Su y cols., 2021) aborda este problema entrenando clasificadores que aprendan a distinguir positivos de instancias no etiquetadas, sin asumir que todos los no etiquetados son negativos. Zhang y cols. (2023a) demuestran su eficacia con un enfoque contrastivo aplicado a detección de fraude.

3.3.4. Detección de anomalías no supervisada

Para el escenario en que no existen etiquetas positivas conocidas, métodos no supervisados como *Isolation Forest* (Pedregosa y cols., 2011) permiten identificar contratos atípicos basados en la distancia de sus características al resto del conjunto. Ocampo-Gutiérrez y

cols. (2019) aplican este enfoque directamente sobre datos OCDS de contratación paraguaya con resultados prometedores. El informe sistemático de Fazekas y cols. (2025) (EPJ Data Science, 2025) recoge y compara los principales enfoques de detección de fraude en la literatura reciente.

3.4. Bloque C: Análisis de redes en contratación pública

3.4.1. Grafos bipartitos empresa-organismo

El grafo bipartito empresa-organismo-contrato es la estructura natural para representar las relaciones de adjudicación: los nodos son empresas y organismos; las aristas son contratos adjudicados, con atributos de importe, tipo y fecha. Las métricas de centralidad (*degree*, *betweenness*, *eigenvector*) sobre esta estructura permiten identificar adjudicatarios con concentración atípica, organismos con comportamiento inusual, y empresas “puente” con alta intermediación en la red (Wachs y cols., 2025).

Noé y cols. (2020) demuestran en redes B2B que el análisis de flujos de pago en grafos bipartitos permite detectar suplantación de proveedores. Martins y cols. (2022) aplican este enfoque a contratación portuguesa, identificando comunidades de riesgo en el grafo de adjudicaciones. Zhang y cols. (2023b) proponen métodos de detección de fraude colectivo en grafos bipartitos sobre plataformas de comercio electrónico, directamente transferibles al contexto de contratación.

3.4.2. Detección de comunidades: algoritmos Louvain y Leiden

La detección de comunidades en el grafo empresa-organismo permite identificar grupos de empresas que licitan sistemáticamente en los mismos organismos —posible indicador de colusión— o grupos de organismos que adjudican sistemáticamente a las mismas empresas.

Traag y cols. (2019) presentan el algoritmo Leiden como mejora demostrada del Louvain: garantiza comunidades bien conectadas y reduce el tiempo de convergencia. Con una modularidad $Q \geq 0.30$ se considera que la partición de red captura estructura comunitaria real, no artefactual (Traag y cols., 2019). La implementación eficiente en memoria compartida para grafos grandes (Blondel y cols., 2024) la hace viable sobre el *dataset* completo de contratación española.

3.5. Bloque D: NLP y procesamiento de documentos de contratación

3.5.1. Named Entity Recognition en documentos legales

El procesamiento de lenguaje natural sobre documentos de contratación —pliegos de prescripciones técnicas, resoluciones de adjudicación, memorias justificativas— constituye la frontera más activa de la investigación en NLP legal. El survey de Kalamkar y cols. (2024) cubre las principales tareas (NER, clasificación, extracción de relaciones) y datasets disponibles para el dominio legal hasta 2024.

La extracción de entidades nombradas (NER) es la tarea central: identifica y clasifica organismos, empresas, importes, plazos y criterios de adjudicación en texto no estructurado. Leitner y cols. (2020) proponen enfoques de granularidad fina para documentos legales en alemán, con resultados aplicables al contexto español. El reto específico en contratación pública es la *fragmentación de entidades*: un nombre de organismo como “Consejería de Educación de la Junta de Andalucía” debe reconocerse como una sola entidad, no como cuatro entidades separadas (Honnibal, Montani, y Van Landeghem, 2020).

3.5.2. Herramientas NLP para español: spaCy y transformers

spaCy (Honnibal y cols., 2020) es el framework NLP de referencia en producción, con soporte nativo para español a través del modelo `es_core_news_lg`. Sus capacidades incluyen tokenización, POS tagging, análisis de dependencias y NER. Para documentos de contratación en español, la combinación de spaCy con *fine-tuning* sobre corpus específico del dominio permite alcanzar precisiones superiores al 80 % en identificación de entidades relevantes.

Los modelos de transformers (BERT, Legal-BETO) aportan comprensión contextual que mejora la NER en casos ambiguos. Para documentos largos (pliegos de 10–50 páginas), se aplican estrategias jerárquicas: fragmentar el documento, clasificar cada fragmento y combinar los resultados (Kalamkar y cols., 2024).

3.5.3. Web scraping: extracción estructurada de documentos administrativos

Una parte sustancial de la información relevante para la detección de riesgos en contratación pública española se encuentra disponible en los anuncios de licitación publicados en el Boletín Oficial del Estado (BOE), accesibles mediante el campo `.Enlace HTML` del dataset longitudinal 2014-2024. A diferencia de los pliegos alojados en PLACE —frecuentemente disponibles como PDF escaneado que requiere OCR—, estos anuncios se publican en formato HTML estructurado, lo que permite una extracción directa y eficiente del texto mediante técnicas de web scraping. BeautifulSoup (Richardson, 2024) es la librería open-source de referencia para el parseo y extracción de información de páginas HTML y XML en Python. Proporciona una API intuitiva para navegar por el árbol DOM, localizar elementos mediante selectores CSS y extraer contenido textual de forma selectiva. Su integración con pipelines de NLP es directa, permitiendo la limpieza y normalización del texto extraído para su posterior procesamiento con herramientas como spaCy o su indexación en bases de datos documentales.

3.5.4. LLMs locales para análisis jurídico-administrativo

Los modelos de lenguaje locales (*Large Language Models* en local) permiten analizar documentos sin dependencia de APIs externas, preservando la confidencialidad de los datos públicos y el cumplimiento con el RGPD. Qwen3-7B (Alibaba Cloud, 2024) ofrece excelente soporte para español con 64 GB de RAM, mientras que Mistral Small 3 (Mistral AI, 2025) es la referencia comparativa para tareas de síntesis y clasificación de documentos.

En ProcureWatch, el LLM local tiene un rol estrictamente auxiliar: estructurar documentos, extraer entidades y generar explicaciones de casos. La decisión sobre si un contrato es sospechoso es siempre humana (European Parliament and Council, 2024).

3.6. Bloque E: Visualización analítica y explicabilidad

3.6.1. Explicabilidad en modelos de scoring de riesgo

El *AI Act* europeo (European Parliament and Council, 2024) clasifica como sistemas de **alto riesgo** aquellos que generan puntuaciones de riesgo que puedan influir en decisiones sobre empresas o contratos. ProcureWatch, al generar scores de riesgo contractual, debe

diseñarse con trazabilidad completa y supervisión humana obligatoria. Los métodos SHAP y LIME (Nallakaruppan y cols., 2025) proporcionan la base técnica para la explicabilidad del scoring: SHAP calcula contribuciones marginales exactas de cada *feature* al score final, mientras que LIME ofrece aproximaciones locales interpretables.

3.6.2. Visualización de grafos interactivos

Para la visualización del grafo empresa-organismo en front-end web, el benchmarking de 2026 (PkgPulse, 2026) posiciona Sigma.js como la mejor opción para grafos grandes (>10.000 nodos) gracias a su motor WebGL, y Cytoscape.js para grafos médios con necesidades de análisis interactivo. PyVis (*vis-network* en Python) se emplea para prototipado rápido integrado en Streamlit.

3.6.3. Dashboards de riesgo

El diseño del panel de riesgo de ProcureWatch sigue los principios de jerarquía de información y flujo de revisión descritos por Ivalua (2024): priorizar el ranking de casos por nivel de riesgo, facilitar la navegación desde el caso general al detalle del contrato, e incorporar la explicación de los indicadores activados. El objetivo de usabilidad SUS>68 corresponde al nivel “aceptable” de la escala de Brooke (1996).

3.7. Conclusiones del estado del arte y justificación del proyecto

El análisis de la literatura permite extraer cuatro conclusiones que justifican directamente la propuesta:

1. **Datos de alta calidad disponibles, sin explotación sistemática.** Existen datos abiertos de alta calidad sobre contratación pública española (BOE, PLACE), pero no hay herramientas open-source que los integren y analicen de forma sistemática con un enfoque de red flags y análisis de redes (Muñoz Plà, 2026; Open Contracting Partnership, 2023).
2. **Eficacia demostrada de métodos combinados.** La literatura demuestra consistentemente que los modelos que combinan indicadores de red flags con análisis de redes mejoran la detección respecto a indicadores aislados, pero esos modelos rara

vez se implementan sobre datos españoles con herramientas reproducibles (Wachs y cols., 2025; Coviello y Mariniello, 2023; Fazekas y Cingolani, 2020).

3. **NLP como dimensión infraexplotada.** El procesamiento de documentos contractuales (pliegos, resoluciones) mediante NLP es una dimensión apenas explorada en la literatura sobre detección de fraude, a pesar de que contiene información cualitativa clave (Kalamkar y cols., 2024).
4. **Ausencia de prototipo integrado local.** No existe un prototipo open-source, local y evaluable que combine pipeline OCDS, análisis de redes, NLP documental y visualización explicable sobre contratación pública española. ProcureWatch Analytics propone cerrar exactamente esta brecha.

4. Objetivos

Los objetivos específicos recogen el alcance inicial del TFM y sirven como marco de trabajo para el diseño del prototipo. Su grado de cumplimiento en la versión de cierre se evalúa posteriormente en las Secciones 7.14 y 10.1, distinguiendo entre funcionalidades implementadas, soporte opcional, validación disponible y líneas de trabajo futuro.

4.1. Objetivo General

Diseñar, desarrollar y evaluar **ProcureWatch Analytics**, una plataforma analítica multiagente que normalice datos abiertos de contratación pública de BOE 2014–2024, PLACE y OpenTender para el dominio CPV 71; calcule indicadores de red flags mediante reglas deterministas RF-01 a RF-06; construya relaciones entre compradores, contratos, adjudicatarios, CPV y fuentes; recupere evidencia documental disponible; y genere fichas explicables y un dashboard que apoyen la priorización de casos para revisión humana, sin declarar fraude de forma automática.

4.2. Objetivos Específicos

OE1 — Análisis del dominio y las fuentes de datos

Estudiar exhaustivamente el ciclo de contratación pública española y la estructura y calidad de los datos disponibles en PLACE y el *dataset* BOE 2014–2024 según estándar OCDS, enfocando el análisis en contratos de servicios y en el dominio CPV 71, e identificando las variables más relevantes para la detección de red flags.

Métrica: Documento de análisis exploratorio con completitud, coherencia temporal y distribución de variables; matriz de relevancia: campos OCDS → indicadores de red flags; mínimo 3 fuentes complementarias identificadas.

OE2 — Diseño del modelo de datos analítico

Definir e implementar un esquema de datos integrado y armonizado con OCDS para contratos, licitaciones, adjudicatarios, órganos contratantes, fuentes y relaciones de contratación del dominio CPV 71.

Métrica: Esquema OCDS adaptado y canónico versionado; tablas analíticas de contratos y adjudicatarios; artefactos reproducibles de nodos y aristas. Parquet es la persistencia principal evaluada; PostgreSQL y Neo4j se soportan como persistencias opcionales.

OE3 — Construcción del *pipeline* de datos

Implementar un *pipeline* reproducible de descarga, limpieza, normalización, enriquecimiento y carga de datos OCDS, con validación automática de calidad y trazabilidad de fuentes.

Métrica: Pipeline ejecutable en Python y pandas, con salidas Parquet, controles de calidad, hashes, cobertura y reportes de ejecución. DuckDB y Polars quedan como opciones de ampliación, no como motores utilizados en la evaluación final.

OE4 — Desarrollo del motor de red flags

Diseñar e implementar un motor explicable de red flags basado en literatura (Open Contracting Partnership, 2024b; Fazekas y Cingolani, 2020), con reglas deterministas para priorizar contratos y variables relacionales que contextualicen la recurrencia y la concentración.

Métrica: Seis indicadores (RF-01 a RF-06) implementados con lógica trazable, flags y scores por contrato y reporte de ejecución; RF-07 a RF-15, Isolation Forest y Positive-Unlabeled Learning quedan como extensiones.

OE5 — Módulo de análisis de redes

Construir y analizar el grafo empresa-organismo-contrato para calcular comunidades mediante Louvain, variables de centralidad y patrones de concentración relevantes en el dominio CPV 71. Leiden queda como ampliación futura.

Métrica: Artefactos reproducibles de nodos, aristas, comunidades Louvain, métricas y features relacionales; modularidad $Q > 0,30$. La muestra evaluada obtiene $Q = 0,6051129926$ y la demo integrada, con 11 nodos, 13 aristas y 2 comunidades, obtiene $Q = 0,3165680473$.

OE6 — Módulo NLP y enriquecimiento Documental

Implementar un componente documental trazable para cargar contenido HTML o textual, fragmentarlo, recuperar evidencias y generar fichas de caso con citas. La versión

evaluada utiliza una ruta local con fallback determinista; Qdrant y Ollama son servicios opcionales, BGE-M3 es una configuración opcional de embeddings y spaCy queda como marco previsto para una ampliación de extracción lingüística.

Métrica: Flujo demostrable de corpus, chunks, retrieval, contexto y citas; la demo integrada recupera 2 evidencias y genera 2 citas sin requerir servicios externos.

OE7 — Desarrollo del *front-end* analítico

Construir una interfaz web local basada en Streamlit con resumen, ranking de contratos priorizados, detalle de caso, grafo de relaciones, evidencias y trazabilidad. Sigma.js queda como opción de extensión para grafos web de mayor escala.

Métrica: Validación headless en estado **ready**, 10 comprobaciones superadas y 0 excepciones. SUS, tiempo por tarea y claridad Likert quedan pendientes de un estudio formal con usuarios.

OE8 — Evaluación integral del sistema

Validar el prototipo mediante tests automatizados, benchmark reproducible, calidad de datos, consistencia y sensibilidad de red flags, modularidad Louvain, trazabilidad documental y diez fichas cualitativas del dominio CPV 71: cinco casos de score máximo, tres de riesgo medio y dos controles.

Métrica: Suite mínima y completa documentadas; benchmark con y sin validación del dashboard; diez fichas trazables; informe de limitaciones y *roadmap*. La precisión frente a fraude, la revisión experta, RAGAS sobre corpus amplio y la usabilidad formal quedan fuera de la validación cerrada.

5. Marco Normativo

El diseño, desarrollo y eventual despliegue de ProcureWatch Analytics deben realizarse en el marco de la normativa española y europea aplicable a la contratación pública, la protección de datos, la transparencia y, de forma especialmente relevante, la inteligencia artificial. A continuación se analizan las principales disposiciones normativas y sus implicaciones concretas para el sistema.

5.1. Ley 9/2017, de 8 de noviembre, de Contratos del Sector Público (LCSP)

La LCSP (Jefatura del Estado, 2017) constituye el eje central de la contratación pública en España. Define los tipos de contrato (obras, servicios, suministros, concesiones), los procedimientos de adjudicación (abierto, restringido, negociado, menor, emergencia), los umbrales de publicidad y las obligaciones de transparencia. Para ProcureWatch Analytics, la LCSP tiene tres implicaciones prácticas:

- **Determina la semántica de los indicadores de riesgo:** Por ejemplo, el uso del contrato menor por encima de los umbrales legales (artículo 118) o la recurrencia a procedimientos negociados sin publicidad (artículo 168) solo son anómalos cuando se exceden ciertos límites. El sistema debe parametrizar sus *red flags* (RF-01 a RF-15) conforme a estos umbrales.
- **Exige publicidad de los contratos** (artículo 63), lo que legitima el uso de datos del BOE 2014-2024 y PLACE como fuentes abiertas. Los anuncios HTML objeto de web scraping se publican en cumplimiento de esta obligación, por lo que su análisis no vulnera derechos de confidencialidad.
- **No otorga potestad sancionadora al sistema:** La LCSP reserva la supervisión y sanción a los órganos de contratación y al Tribunal Administrativo Central de Recursos Contractuales. ProcureWatch Analytics es una herramienta de apoyo a la auditoría, nunca un sustituto de la decisión humana, en consonancia con el principio de supervisión humana obligatoria exigido por el AI Act.

5.2. Directiva 2014/24/UE del Parlamento Europeo y del Consejo, de 26 de febrero de 2014

Esta directiva (European Parliament and Council, 2014) armoniza los procedimientos de contratación en la UE y establece los principios de concurrencia, transparencia, igualdad de trato y no discriminación. Aunque traspuesta a la LCSP, su mención en el marco normativo es relevante porque:

- **Funda la legitimidad del análisis de datos abiertos:** El principio de transparencia activa (artículo 21) impulsa la publicación de datos en formatos reutilizables, base del *pipeline* OCDS de ProcureWatch.
- El proyecto europeo DIGIWHIST (Fazekas y Cingolani, 2020) y la plataforma OpenTender.eu se construyeron sobre datos de 35 jurisdicciones europeas al amparo de esta directiva. El sistema se alinea con esa tradición analítica.

5.3. Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno

Esta ley (Jefatura del Estado, 2013) regula el derecho de acceso a la información pública y la reutilización de datos abiertos. Define qué información es de libre acceso y bajo qué condiciones. Para ProcureWatch Analytics:

- **Los contratos públicos son información pública reutilizable** (artículo 5), sin necesidad de autorización expresa. El web scraping de los anuncios del BOE es conforme con esta ley, siempre que se realice sin sobrecargar los servidores y con fines de análisis no comercial.
- **La reutilización debe respetar la integridad de la información** (artículo 10). El sistema no modifica los datos originales, solo los transforma y analiza. La trazabilidad de fuentes (campo `Enlace HTML` en el dataset longitudinal) garantiza la verificabilidad.

5.4. RGPD (Reglamento (UE) 2016/679) y LOPDGDD (Ley Orgánica 3/2018)

Los datos de contratación incluyen información sobre personas físicas: nombres de administradores, representantes legales o personas autónomas adjudicatarias. Aunque la publicación de estos datos se realiza en cumplimiento de obligaciones de transparencia (base de legitimación del artículo 6.1.e del RGPD: «misión en interés público»), el tratamiento posterior mediante algoritmos de *scoring* de riesgo debe observar:

- **Principio de minimización** (artículo 5.1.c): Solo se tratan los datos estrictamente necesarios para la detección de patrones de riesgo. No se incluyen datos de contacto privados ni información bancaria.
- **Principio de limitación de la finalidad** (artículo 5.1.b): Los *scores* de riesgo solo se utilizan para priorizar casos ante organismos de control, no para decisiones automáticas que afecten a derechos de las empresas. Se prohíbe explícitamente el uso del sistema para crear listas negras o denegar contratación.
- **Derecho de oposición y transparencia** (artículos 21 y 13-15): Las empresas señaladas con alto riesgo deben poder conocer los indicadores que motivaron esa puntuación. Por eso el sistema incluye fichas explicativas generadas con LLM local y desgloses por reglas activadas. SHAP se mantiene como técnica de referencia para extensiones con modelos predictivos, pero no se presenta como componente evaluado de la versión final.

El uso de un **LLM local** en lugar de APIs externas (por ejemplo, OpenAI) elimina el riesgo de transferencia internacional de datos (artículo 44 RGPD) (European Parliament and Council, 2016). Todos los datos permanecen en la infraestructura local del organismo de control, cumpliendo con las garantías de privacidad desde el diseño («*by design and by default*», artículo 25 RGPD). Adicionalmente, la LOPDGDD (Jefatura del Estado, 2018) refuerza estos principios en el ordenamiento español.

5.5. Reglamento (UE) 2024/1689 del Parlamento Europeo y del Consejo, de 13 de junio de 2024 (AI Act)

El AI Act (European Parliament and Council, 2024) es el marco jurídico más relevante para ProcureWatch Analytics, ya que clasifica los sistemas de IA según su nivel de riesgo. Aunque el sistema es un prototipo de investigación, su diseño debe anticipar los requisitos que aplicarían en un posible despliegue real.

5.5.1. Clasificación del sistema como de alto riesgo

El artículo 6 del AI Act define los sistemas de IA de **alto riesgo** como aquellos que pueden afectar negativamente a derechos fundamentales o la seguridad. El anexo III incluye, entre otros, los sistemas utilizados para «evaluar la solvencia o la fiabilidad crediticia de personas físicas» (punto 5b). Aunque no menciona explícitamente la auditoría de contratación pública, la generación de puntuaciones de riesgo sobre empresas que podrían influir en decisiones de organismos de control (investigaciones, inspecciones) ha sido interpretada por la literatura como sistema de alto riesgo por analogía (Wachs y cols., 2025; Nallakuruppan y cols., 2025). Por tanto, ProcureWatch Analytics se diseña bajo los siguientes principios preventivos, exigibles a los sistemas de alto riesgo:

- **Trazabilidad completa** (artículo 13): Cada *score* de riesgo debe poder descomponerse en los indicadores de *red flags* y métricas de red que lo componen. En la versión evaluada esta descomposición se consigue mediante reglas versionadas, evidencias y reportes; SHAP, Positive-Unlabeled Learning e Isolation Forest quedan como técnicas de ampliación o contraste cuando existan artefactos y etiquetas suficientes.
- **Supervisión humana obligatoria** (artículo 14): Ninguna señal de riesgo se considera definitiva sin revisión por parte de un auditor o técnico. La interfaz de Streamlit está diseñada para apoyar la decisión humana, no para reemplazarla. El LLM local solo genera explicaciones, no recomendaciones de acción.
- **No discriminación y equidad** (artículo 10): El sistema debe evaluarse para evitar sesgos por tamaño de empresa, ubicación geográfica o tipo de procedimiento. Se incluirán análisis de equidad en la evaluación final (OE8).
- **Registro y documentación** (artículo 11): Se generará un registro de las versiones

del modelo, los datos de entrenamiento (etiquetas parciales de *ground truth*) y las métricas de validación. Todo el código es *open-source* para permitir auditoría externa.

5.5.2. Implicaciones específicas para el motor de *red flags* y *scoring*

El motor de *red flags* (OE4) se diseña inicialmente con tres posibles enfoques: (i) reglas explícitas basadas en la literatura, (ii) Isolation Forest no supervisado y (iii) Positive-Unlabeled Learning semisupervisado. La versión evaluada prioriza reglas explicables; los enfoques estadísticos se documentan como comparativa o ampliación cuando existan datos suficientes. El AI Act no prohíbe estos métodos, pero exige que:

- El usuario sea informado de que está interactuando con un sistema de IA (artículo 52). La interfaz de ProcureWatch Analytics indicará claramente que el *score* de riesgo es una recomendación estadística, no una prueba de irregularidad.
- Los conjuntos de datos de entrenamiento sean representativos (artículo 10.2). Para una eventual ampliación con PUL, las etiquetas positivas deberían proceder de casos documentados y verificarse su distribución, sin asumir que la ausencia de etiqueta equivale a ausencia de fraude.
- Se realice una evaluación de la conformidad antes del despliegue (artículo 43). En el ámbito del TFM, esta evaluación se aproxima mediante validación técnica, tests automatizados, métricas de calidad de datos, consistencia de reglas, modularidad del grafo, trazabilidad de salidas y análisis de riesgos documentado. Las métricas supervisadas de precisión, la evaluación RAGAS completa y la usabilidad formal quedan reservadas para una validación posterior con datos y usuarios suficientes.

5.5.3. Consecuencias del soporte opcional de un LLM local

El AI Act exime de ciertos requisitos a los sistemas de IA de «uso libre» (artículo 2.6), pero no a los de alto riesgo. El prototipo puede utilizar un **LLM local** con funciones estrictamente auxiliares para estructurar fichas, sin intervenir en el score ni tomar decisiones. Qwen3-8B vía Ollama se contempla como soporte opcional; la demo evaluada emplea el fallback determinista de Agent4 y no depende del modelo. Cuando se active la generación local, se aplican las siguientes buenas prácticas:

- El LLM no se *fine-tune* con datos de contratación sensibles; se usa con *prompting* *few-shot* controlado.
- No se almacenan las interacciones del LLM más allá de la sesión de análisis.
- Se prevé auditar las salidas del LLM mediante métricas de fidelidad, relevancia y cobertura de contexto. RAGAS queda como marco de referencia para una evaluación más amplia, pero no se presenta como resultado masivo validado si no existe artefacto suficiente.

5.6. Otras normativas de referencia

- **Directiva (UE) 2019/1024 sobre datos abiertos y reutilización de la información del sector público** (European Parliament and Council, 2019): Refuerza la obligación de publicar datos públicos en formatos abiertos y reutilizables. Justifica el uso del estándar OCDS en el *pipeline* de datos.
- **Directiva 2022/2557 de Resiliencia de Entidades Críticas (CER)** (European Parliament and Council, 2022): Contextualiza la necesidad de supervisar la contratación en infraestructuras críticas (energía, transporte, sanidad). ProcureWatch Analytics, al centrarse en la división CPV 71 (servicios de arquitectura, ingeniería e inspección), puede incidir en contratos relacionados con dichas infraestructuras.

5.7. Síntesis de implicaciones para el diseño de ProcureWatch Analytics

La Tabla 5.1 resume los principales requisitos normativos y su concreción en la arquitectura del sistema.

5.8. Conclusión del marco normativo

El análisis de las normativas aplicables (LCSP, RGPD, AI Act, etc.) muestra que ProcureWatch Analytics es viable siempre que se respeten los principios de transparencia, explicabilidad, supervisión humana y minimización de datos. El diseño del sistema deberá reflejar estos principios, tal como se desarrollará en los capítulos siguientes.

Tabla 5.1: Requisitos normativos e implementación en ProcureWatch Analytics

Requisito normativo	Implementación en el sistema
Transparencia activa (LCSP, Ley 19/2013)	<i>Pipeline</i> de datos abiertos (BOE 2014-2024, PLACE); web scraping de anuncios HTML públicos.
Minimización y limitación de finalidad (RGPD)	No se tratan datos personales superfluos; <i>scores</i> solo para priorización, no para denegación de contratos.
Supervisión humana obli- gatoria (AI Act)	Interfaz con revisión manual; ninguna acción au- tomática; fichas explicativas trazables con fallback de- terminista y generación local opcional.
Trazabilidad y explicabili- dad (AI Act)	Reglas versionadas, registros de ejecución, trazabili- dad de datos y código versionado para auditoría técni- ca.
Evaluación de riesgos y conformidad (AI Act)	Evaluación técnica del prototipo mediante tests, métricas de calidad de datos, consistencia de reglas, modularidad Louvain, evaluación proxy del scoring y documentación de limitaciones; precisión supervisada, RAGAS y SUS quedan como objetivos de validación futura.
Datos locales y privacidad (RGPD + AI Act)	Ruta local sin API externa; Ollama es opcional y la demo puede ejecutarse mediante fallback determinista.

6. Metodología y diseño del sistema

6.1. Visión general

ProcureWatch Analytics se presenta como un prototipo funcional avanzado que demuestra la viabilidad de integrar múltiples técnicas analíticas —*pipeline* de datos OCDS, indicadores de *red flags*, análisis de redes, procesamiento de lenguaje natural y recuperación aumentada por generación (RAG)— para la detección de patrones de riesgo en contratación pública española. El sistema no emite juicios jurídicos ni declaraciones automáticas de fraude, sino que genera puntuaciones de riesgo trazables, acompañadas de las señales activadas, evidencias documentales y métricas de red, con el fin de priorizar casos para su revisión por parte de organismos de control o auditores.

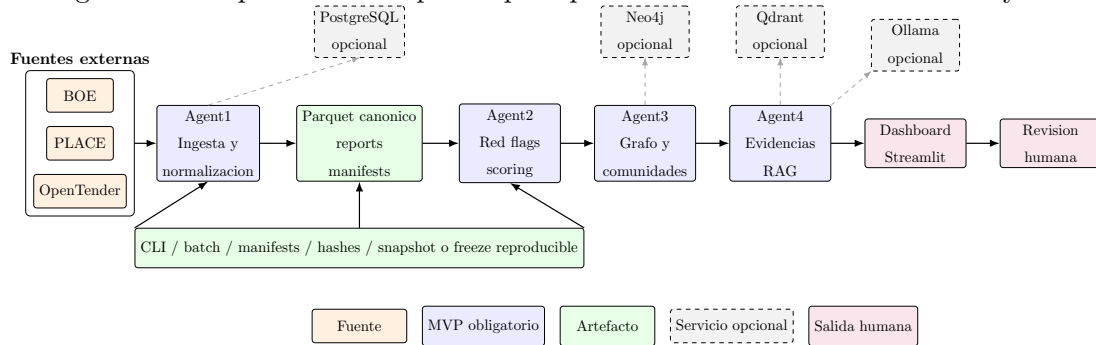
Este capítulo recoge el diseño metodológico del sistema: enfoque multiagente, selección tecnológica, modelo de datos, estrategia de scoring, grafo analítico y componente documental. La implementación concreta de esos elementos, sus salidas y sus mecanismos de reproducción se desarrollan en los capítulos posteriores para separar con claridad el diseño de la ejecución.

La arquitectura separa cuatro responsabilidades analíticas, alineadas con el alcance inicial del TFM:

1. *pipeline* de datos y normalización OCDS;
2. red flags y *scoring*;
3. análisis de redes y comunidades;
4. web scraping, NLP y recuperación documental.

Estas responsabilidades se articulan mediante agentes especializados y componentes de integración que permiten combinar datos tabulares, relaciones de red y evidencias documentales. El resultado principal es un *score* de riesgo trazable, acompañado de *red flags* activadas, evidencias documentales y relaciones de red. La plataforma se concibe, por tanto, como una herramienta de priorización analítica para revisión humana, no como un mecanismo de declaración de fraude.

Figura 6.1: Arquitectura del prototipo reproducible de ProcureWatch Analytics



6.2. Criterios metodológicos del prototipo funcional

El diseño del prototipo funcional no se limita a reducir alcance para facilitar la implementación. Se construye como una versión acotada pero defendible del sistema completo: debe cubrir el flujo de extremo a extremo, generar artefactos verificables y dejar claras sus limitaciones. Esto exige priorizar profundidad operativa sobre amplitud superficial. En otras palabras, resulta preferible implementar menos indicadores con trazabilidad completa que declarar un catálogo amplio de reglas no verificadas.

Los criterios metodológicos aplicados fueron los siguientes:

1. **Ejecución local y reproducible.** El sistema debe poder ejecutarse sin depender de servicios externos no controlados. Los servicios como PostgreSQL, Neo4j, Qdrant u Ollama enriquecen el flujo, pero el prototipo mantiene rutas de ejecución y prueba que no bloquean la validación si no están disponibles.
2. **Fronteras claras entre agentes.** Cada agente produce artefactos que pueden revisarse de forma independiente. Esta frontera evita que un error de visualización o una dependencia de infraestructura oculte problemas de datos, reglas o recuperación documental.
3. **Trazabilidad antes que automatización opaca.** El sistema prioriza reglas, manifests, hashes, reports y evidencias sobre modelos no explicables. La automatización sirve para ordenar revisiones, no para sustituir la decisión experta.
4. **No imputación agresiva.** Cuando faltan datos críticos, el resultado debe registrarse como no evaluable. Esta política reduce falsos positivos derivados de completar información ausente con supuestos no verificables.

5. **Evaluación honesta.** Las métricas disponibles se presentan como calidad de datos, validación técnica o evaluación proxy. No se declara rendimiento supervisado contra fraude si no existen etiquetas confirmadas.

Estos criterios se aplican de manera transversal. En Agent1 justifican una política conservadora de `contract_key_canon`; en Agent2 justifican reglas trazables RF-01 a RF-06; en Agent3 justifican que el grafo sea una proyección derivada del canónico; en Agent4 justifican que el LLM no decida riesgo y que toda ficha de caso deba apoyarse en evidencias recuperadas. La coherencia del prototipo depende precisamente de mantener estas restricciones en todos los componentes.

6.3. Alcance evaluable y alcance extensible

Una distinción clave del trabajo es separar **alcance evaluable** y **alcance extensible**. El alcance evaluable corresponde a lo que se ejecuta, genera salidas y puede defenderse con tests, reports o artefactos. El alcance extensible corresponde a componentes diseñados o compatibles con la arquitectura, pero que no deben presentarse como resultado cuantitativo cerrado si no existe evidencia suficiente.

Esta separación afecta especialmente a tres grupos de elementos. El primero es el catálogo de red flags. El planteamiento inicial contemplaba un conjunto más amplio de indicadores; la versión evaluable del prototipo se concentra en RF-01 a RF-06 porque son las reglas respaldadas por el canónico disponible y por el flujo de scoring documentado. El segundo grupo son las técnicas comparativas de aprendizaje automático, como Isolation Forest y Positive-Unlabeled Learning. Son coherentes con el problema, pero requieren datasets y etiquetas o estrategias de validación que no deben suponerse. El tercer grupo son los servicios de despliegue y visualización avanzada, como API productiva o visualización web de grafos a gran escala, que quedan como extensiones naturales.

La Tabla 6.1 resume esta distinción.

Tabla 6.1: Separación entre alcance evaluable y alcance extensible

Bloque	Alcance evaluable en la versión evaluada	Alcance extensible
Ingesta	BOE, PLACE y OpenTender con canónico, reports y calidad.	Nuevas fuentes, matching probabilístico y validación manual asistida.
Red flags	RF-01 a RF-06 y score 0–100 explicable.	RF-07 a RF-15, fraccionamiento avanzado y reglas documentales.
Modelado estadístico	Evaluación proxy y consistencia de reglas.	Isolation Forest, PU Learning y calibración con etiquetas confirmadas.
Grafo	Nodos, aristas, Louvain, features y Neo4j opcional.	Leiden, escalado completo y visualización web avanzada.
RAG	Corpus, chunks, recuperación, contexto y ficha de caso demostrable.	Evaluación RAGAS masiva, corpus ampliado y control de citas más estricto.
Interfaz	Dashboard Streamlit para revisión humana local.	API productiva, roles, auditoría de usuario y paneles institucionales.

Esta tabla evita dos errores de redacción. El primero sería ocultar trabajo diseñado porque no todo se ejecuta en la misma profundidad; el segundo sería presentar extensiones como si ya fueran resultados cerrados. La memoria adopta una posición intermedia: reconoce la arquitectura ampliable, pero defiende como resultado principal aquello que la versión de cierre permite ejecutar, verificar y explicar.

6.4. Stack tecnológico de la versión evaluada

La selección tecnológica prioriza cuatro criterios: ejecución local, reproducibilidad académica, ecosistema abierto y adecuación al problema de contratación pública. La Tabla 6.2 distingue las tecnologías utilizadas en la versión evaluada de las integraciones opcionales.

Tabla 6.2: Stack tecnológico implementado y extensiones opcionales de ProcureWatch Analytics

Capa	Tecnología elegida	Justificación
Lenguaje principal	Python	Ecosistema maduro para ciencia de datos, NLP, aprendizaje automático, grafos y automatización de pipelines.
Procesamiento tabular	pandas y Parquet	Tecnologías utilizadas por los pipelines y artefactos evaluados. Polars es una dependencia opcional y DuckDB queda como posible extensión para consulta analítica sobre Parquet (DuckDB Foundation, 2025; Polars, 2025).
Persistencia relacional	PostgreSQL opcional (PostgreSQL Global Development Group, 2025)	Agent1 implementa la escritura de tablas analíticas cuando SQLAlchemy y el servicio están disponibles; la ejecución mínima conserva Parquet como salida principal.
Grafo analítico	Neo4j Community y NetworkX (Neo4j, 2025; Hagberg, Schult, y Swart, 2008)	Neo4j facilita consulta y exploración del grafo; NetworkX permite reproducir métricas y experimentos en scripts Python.
Motor de red flags	Python, reglas OCDS y soporte de comparativa con scikit-learn (Pedregosa y cols., 2011; Open Contracting Partnership, 2024b)	La versión evaluada prioriza reglas explícitas y trazables; Isolation Forest se conserva como comparativa o extensión no supervisada.
Web scraping y parsing documental	BeautifulSoup y fallback HTML (Richardson, 2024)	La versión evaluada carga contenido HTML o textual y conserva una ruta determinista; no incluye scraping masivo ni descarga automática de pliegos.
NLP clásico	spaCy como extensión prevista (Honnibal y cols., 2020)	La tokenización, NER y extracción jurídica compleja no forman un pipeline cerrado en el MVP.

Capa	Tecnología elegida	Justificación
Embeddings	BGE-M3 como configuración opcional (BAAI, 2024)	Modelo previsto para recuperación semántica cuando se activa la ruta de embeddings; no es requisito de la demo final.
Vector store	Qdrant opcional (Qdrant, 2025)	Soporte local para indexación vectorial; la recuperación evaluada dispone de fallback sin este servicio.
LLM local	Qwen3-8B vía Ollama, opcional (Alibaba Cloud, 2024; Qwen Team, 2025; Ollama, 2025)	Soporte de generación local para fichas explicativas; el flujo evaluado puede producir una salida determinista y no utiliza el LLM para decidir el score.
Orquestación multi-agente	CLI, batch y diseño compatible con LangGraph/LangChain (LangChain AI, 2024, 2025)	La versión evaluada se reproduce mediante comandos y batch; LangGraph/LangChain se documentan como soporte de orquestación ampliable.
API de servicio	FastAPI como extensión prevista (FastAPI, 2025)	No es requisito de ejecución de la versión evaluada; se reserva para exponer consultas y fichas en una versión de servicio.
Evaluación RAG	RAGAS como marco de referencia (Ragas contributors, 2025)	La evaluación disponible es local/proxy; RAGAS queda para evaluaciones documentales con corpus mayor y artefactos suficientes.
Dashboard	Streamlit, Plotly y soporte de extensiones con Sigma.js/PyVis (Snowflake Inc., 2025; Plotly Technologies Inc., 2025; Sigma.js contributors, 2025; PyVis contributors, 2025)	Streamlit cubre la demo local; Sigma.js/PyVis no son obligatorios para la versión evaluada.
Reproducibilidad	Docker Compose, Git y scripts parametrizados (Docker Inc., 2025)	Compose define PostgreSQL, Neo4j y Qdrant; Ollama se ejecuta, si se utiliza, como servicio local o externo independiente.

Qwen3-8B se conserva como opción de generación local para explicar casos y sintetizar contexto documental. Según la ficha oficial, es un modelo causal de 8.2B parámetros con soporte multilingüe, contexto nativo de 32.768 tokens y licencia Apache-2.0 (Alibaba Cloud, 2024). ProcureWatch puede ejecutarlo mediante Ollama cuando el servicio está disponible, pero esta ruta no es obligatoria: Agent4 mantiene un fallback determinista que permite reproducir la demo sin APIs externas ni infraestructura adicional.

6.5. Justificación del uso de IA y LLM

El uso de IA en ProcureWatch no se justifica por sustituir el juicio experto, sino por resolver dos problemas técnicos que no se abordan adecuadamente con consultas relacionales simples. En primer lugar, la contratación pública combina variables estructuradas con texto administrativo heterogéneo: anuncios, pliegos, resoluciones, memorias justificativas y criterios de adjudicación. En segundo lugar, la detección temprana de riesgo requiere relacionar señales tabulares, patrones de red y evidencia documental de forma explicable.

Por ello, el prototipo adopta un enfoque **explicable y controlado**. Las reglas RF-01 a RF-06 y las métricas de red calculan variables verificables. Agent4 recupera fragmentos y construye fichas citadas mediante una ruta determinista; cuando se habilita Qwen3-8B, el modelo se limita a apoyar la redacción sobre evidencias ya recuperadas. Esta separación impide que el LLM decida el nivel de riesgo.

La opción de un LLM local vía Ollama aporta privacidad operativa y evita enviar documentos a APIs externas. No obstante, la reproducibilidad de la versión evaluada se apoya en el fallback determinista y en la vinculación de cada ficha con indicadores y fragmentos recuperados. Esta arquitectura es coherente con el principio *human-in-the-loop*: el sistema prioriza y explica, pero la evaluación final corresponde siempre a una persona.

6.6. Pipeline de datos

El *pipeline* de datos convierte fuentes heterogéneas en un modelo analítico común. La fuente principal es el *dataset* longitudinal BOE 2014–2024 en formato OCDS (Muñoz Plà, 2026); PLACE y OpenTender actúan como fuentes complementarias para contrastar variables, recuperar documentos y comparar indicadores europeos.

6.6.1. Fases del pipeline

1. **Ingesta de datos brutos.** Descarga o incorporación de ficheros BOE/OCDS, datos PLACE y extractos OpenTender. Cada fichero conserva ruta, fecha de descarga, URL, hash y versión.
2. **Validación inicial.** Comprobación de esquema, codificación, duplicados, tipos de datos y presencia de campos críticos: identificador de contrato, organismo, adjudicatario, importe, fecha, procedimiento y CPV.
3. **Normalización OCDS.** Transformación de campos a entidades analíticas comunes: contrato, licitación, adjudicación, organización, proveedor, lote, documento y procedimiento.
4. **Filtrado del dominio CPV 71.** Selección de servicios de arquitectura, ingeniería e inspección para reducir heterogeneidad sectorial y mejorar comparabilidad de importes, plazos y criterios técnicos.
5. **Enriquecimiento.** Derivación de variables como duración del procedimiento, desviación de importe, recurrencia proveedor-organismo, importe acumulado, año de adjudicación, subfamilia CPV y presencia documental.
6. **Perfilado de calidad.** Generación automática de métricas de completitud, coherencia temporal, nulos, duplicados, distribución de importes y cobertura por organismo.
7. **Carga analítica.** Persistencia opcional en PostgreSQL para consultas tabulares, artefactos NetworkX/Parquet para el grafo y soporte opcional de Neo4j y Qdrant.

6.6.2. Modelo de datos analítico

El modelo relacional se centra en entidades estables y trazables. La Tabla 6.3 resume el esquema lógico inicial, alineado con el esquema OCDS adaptado definido para el TFM.

Tabla 6.3: Modelo de datos analítico propuesto

Entidad	Clave principal	Campos relevantes
Contrato	<code>contract_id</code>	objeto, CPV, procedimiento, importe estimado, importe adjudicado, fechas, fuente, estado.
Organismo	<code>buyer_id</code>	nombre normalizado, tipo de administración, territorio, número de contratos, importe agregado.
Adjudicatario	<code>suppliers_id</code>	nombre normalizado, NIF cuando exista, forma jurídica, contratos ganados, importe acumulado.
Adjudicación	<code>award_id</code>	contrato, adjudicatario, importe, fecha, número de ofertas, lotes y relación con licitación.
Documento	<code>document_id</code>	URL, tipo documental, texto extraído, hash, contrato asociado y estado de scraping/parsing.
Red flag	<code>flag_id</code>	contrato, indicador, valor observado, umbral, severidad, justificación y versión de regla.
Score	<code>score_id</code>	score total, nivel de riesgo, pesos, fecha de cálculo, modelo/reglas y explicación.

6.7. Motor de red flags y scoring

El motor de red flags transforma variables observables en indicadores de riesgo. Cada indicador se define mediante: nombre, motivación, campos de entrada, regla de cálculo, umbral, severidad, limitaciones y referencia bibliográfica. Esta estructura permite auditar por qué un contrato recibe una puntuación concreta.

La primera versión prioriza indicadores implementables con los campos BOE/OCDS, variables derivadas y evidencia documental:

1. **RF-01 Baja concurrencia o identificación insuficiente:** contratos con una sola oferta, ausencia de adjudicatario o información equivalente cuando el dato esté disponible.

2. **RF-02 Procedimiento de riesgo:** uso de procedimientos sensibles, excepcionales o menos competitivos según la normalización disponible.
3. **RF-03 Recurrencia comprador-proveedor:** adjudicaciones repetidas entre el mismo organismo comprador y adjudicatario.
4. **RF-04 Concentración de importe comprador-proveedor:** peso elevado del importe adjudicado dentro de una misma pareja comprador-proveedor.
5. **RF-05 Desviación de importe:** adjudicaciones por encima del importe estimado o relaciones estimado/adjudicado atípicas.
6. **RF-06 Patrón temporal anómalo:** intervalo anómalo entre publicación, adjudicación y formalización cuando existen fechas suficientes.

RF-07 a RF-15, incluyendo fragmentación avanzada y riesgo de red como indicador directo, forman parte del catálogo de ampliación. La versión evaluada calcula la puntuación mediante RF-01 a RF-06, reglas deterministas y ponderadas que producen un score explicable por contrato. Las features de Agent3 pueden contextualizar RF-03 y RF-04, pero la evidencia documental y un LLM no alteran el score. Isolation Forest, Positive-Unlabeled Learning y una combinación estadística híbrida quedan como comparaciones futuras cuando existan datos y etiquetas suficientes.

6.8. Grafo de contratación

El grafo representa tres tipos de nodos: organismos contratantes, adjudicatarios y contratos. Las aristas conectan organismos con contratos publicados y contratos con adjudicatarios. En escenarios donde existan UTEs, adjudicatarios múltiples o datos societarios abiertos, el grafo incorpora relaciones adicionales entre empresas que participan conjuntamente o comparten vínculos relevantes.

Las métricas previstas son:

- grado y grado ponderado por importe;
- centralidad de intermediación para detectar actores puente;
- centralidad de autovector o PageRank para estimar influencia estructural;
- detección de comunidades con Louvain; Leiden queda como extensión futura;

- concentración por organismo, proveedor y subfamilia CPV.

Estas variables no se interpretan como evidencia de irregularidad por sí solas. Su valor analítico aparece cuando coinciden con red flags tabulares o documentales: por ejemplo, una comunidad densa de adjudicatarios recurrentes en un conjunto reducido de organismos, combinada con baja concurrencia y plazos atípicos.

6.9. Módulo documental, embeddings y RAG

El módulo documental incorpora contexto cualitativo desde un corpus de contenido HTML o textual asociado a contratos. La versión evaluada dispone de parsing con BeautifulSoup y fallback HTML, carga de corpus, fragmentación, recuperación y citas. No implementa scraping masivo, descarga automática de pliegos ni un pipeline principal cerrado de extracción con spaCy; estas capacidades quedan como ampliación.

El flujo RAG se limita a explicar casos, no a decidir el nivel de riesgo:

1. carga y fragmentación del corpus por contrato;
2. recuperación de fragmentos relevantes para cada caso priorizado;
3. composición de contexto con identificadores de documento y fragmento;
4. generación de una ficha explicativa mediante fallback determinista;
5. validación de que las afirmaciones de la ficha están respaldadas por citas recuperadas.

BGE-M3 puede configurarse como modelo de embeddings, Qdrant como almacén vectorial y Qwen3-8B mediante Ollama como apoyo de generación local. Los tres componentes son opcionales y no fueron necesarios para la demo integrada evaluada.

La ficha explicativa tendrá formato estructurado: resumen del contrato, indicadores activados, evidencia tabular, evidencia de red, evidencia documental, limitaciones y recomendación de revisión humana.

7. Implementación por agentes y componentes

Una vez definido el diseño general, la implementación se organiza en agentes y componentes con responsabilidades separadas. Esta separación evita que la ingesta de datos, el cálculo de riesgo, el análisis relacional, la recuperación documental y la visualización queden acoplados en una sola pieza de software. El resultado es un prototipo que puede ejecutarse por fases, validarse con tests y defenderse mediante artefactos intermedios.

7.1. Implementación multiagente

La arquitectura multiagente se plantea como un flujo de responsabilidades explícito y auditable. La decisión clave es mantener **cuatro agentes especializados**, alineados con los objetivos definidos inicialmente, y tratar la orquestación como infraestructura de ejecución, no como un quinto agente funcional.

Tabla 7.1: Agentes previstos en ProcureWatch Analytics

Agente	Responsabilidad	Salida
Agente 1: Pipeline de Datos y Normalización OCDS	Descarga, limpia, valida y normaliza fuentes BOE/OCDS, PLACE y OpenTender; detecta inconsistencias de importes, fechas, duplicados y NIF; prepara tablas analíticas y artefactos de carga.	Dataset normalizado, perfil de calidad, hashes de fuentes, ficheros Parquet y tablas PostgreSQL opcionales.
Agente 2: Red Flags y Scoring	Calcula RF-01 a RF-06 por contrato mediante reglas deterministas; incorpora recurrencia, concentración, desviaciones y variables relacionales para priorizar la revisión humana.	Red flags activadas, score 0–100 por contrato, nivel de riesgo, justificación por indicador y reporte de ejecución.

Agente	Responsabilidad	Salida
Agente 3: Análisis de Redes y Comunidades	Construye el grafo empresa-organismo-contrato con NetworkX; calcula métricas, comunidades Louvain y features relacionales. Leiden y el enriquecimiento societario quedan como extensiones.	Nodos, aristas, métricas de red, comunidades Louvain, resumen y features para otros agentes.
Agente 4: Recuperación Documental	Carga contenido HTML o textual, fragmenta el corpus, recupera contexto y genera fichas citadas mediante fallback determinista. Qdrant, Ollama y BGE-M3 son opcionales; spaCy queda como ampliación.	Manifest de corpus, chunks, fragmentos recuperados, citas, contexto y ficha explicativa de caso.

El diseño de orquestación es compatible con un estado compartido de tipo LangGraph que contenga, como mínimo, identificador de ejecución, versión del dataset, contratos seleccionados, variables normalizadas, red flags, métricas de red, fragmentos documentales, prompts versionados, salida JSON de cada agente y errores detectados. En la versión evaluada, esa coordinación se materializa mediante CLI, batch y artefactos intermedios. Las transiciones se definen de forma secuencial y auditable: el Agente 1 prepara datos; los Agentes 2 y 3 calculan riesgo tabular y relacional; el Agente 4 recupera evidencias y genera explicaciones; finalmente, el sistema consolida la ficha de caso para el dashboard.

Trazabilidad de una ficha de riesgo				
Fuente	Transformación	Indicador	Evidencia	Salida
BOE/PLACE	OCDS + calidad	RF/score/grafó	corpus + documentos	ficha revisable
<code>url/hash</code>	<code>run_id</code>	<code>rule.version</code>	<code>chunk_id</code>	<code>case_id</code>
↓				
Cada afirmación de la ficha debe poder enlazarse con una regla, un valor observado o un fragmento documental recuperado.				

Figura 7.1: Flujo de trazabilidad desde la fuente hasta la ficha explicativa.

7.2. Interfaz analítica

El dashboard se diseña para una tarea concreta: localizar contratos de mayor riesgo, entender por qué han sido priorizados y revisar la evidencia asociada. La versión de cierre organiza la información en torno a:

- KPIs generales: número de contratos, importe total, organismos, adjudicatarios y distribución de riesgo;
- filtros por año, CPV, organismo, adjudicatario, procedimiento y nivel de riesgo;
- ranking de contratos por score y filtros por adjudicatario;
- vista de detalle de contrato con red flags activadas y valores observados;
- grafo interactivo de relaciones organismo-adjudicatario-contrato;
- ficha explicativa generada con evidencias tabulares, de red y documentales.

La interfaz no debe mostrar el score como una verdad absoluta. Debe presentarlo como una prioridad de revisión, acompañada de factores explicativos y limitaciones de datos.

7.3. Protocolo de revisión humana asistida

El dashboard no se concibe como un sistema de decisión automática, sino como una herramienta para ordenar el trabajo de revisión. Por ello, el flujo de uso previsto se estructura en cinco pasos. Primero, el usuario observa KPIs agregados y distribución de niveles de riesgo. Segundo, filtra por dominio, organismo, adjudicatario, procedimiento o nivel. Tercero, selecciona un contrato o proveedor priorizado. Cuarto, revisa las señales activadas, las relaciones de grafo y las evidencias documentales. Quinto, decide si procede abrir una revisión manual, solicitar más información o descartar el caso por justificación suficiente.

Esta secuencia es relevante desde el punto de vista normativo y metodológico. Si el sistema mostrara solo un score, el usuario podría interpretar la puntuación como una conclusión. Al mostrar también flags, evidencias, campos no evaluables y advertencias, el dashboard obliga a leer el riesgo como una hipótesis de trabajo. La decisión final permanece fuera del sistema.

La Tabla 7.2 resume el protocolo de revisión.

Tabla 7.2: Protocolo de revisión humana asistida

Paso	Acción del usuario	Soporte del sistema
1. Panorama	Revisa volumen, importes y distribución de riesgo.	KPIs agregados y filtros iniciales.
2. Priorización	Ordena contratos por riesgo y filtra por adjudicatario.	Ranking contractual, score y nivel bajo/medio/alto/crítico.
3. Diagnóstico	Comprueba por qué se prioriza el caso.	Red flags activadas, pesos, campos usados y no evaluables.
4. Contexto	Analiza relaciones y evidencias documentales.	Grafo, comunidades, fragmentos recuperados y citas.
5. Decisión humana	Decide si abre revisión, pide información o descarta.	Ficha explicativa con limitaciones y trazabilidad.

El sistema debe hacer visibles también los motivos de cautela. Un contrato con riesgo alto puede estar justificado por urgencia, especialización técnica o ausencia de competencia real en el mercado. Un contrato con riesgo bajo puede contener problemas que no se detectan por falta de datos o porque la irregularidad no se expresa en los campos analizados. Por ello, el dashboard no responde a la pregunta “¿hay fraude?”, sino a una pregunta más adecuada: “¿qué casos merecen revisión prioritaria y con qué evidencias disponibles?”.

La ficha de caso actúa como puente entre el análisis automático y la revisión experta. Debe contener identificadores del contrato, fuente, organismo, adjudicatario, CPV, importe, procedimiento, score, red flags, relaciones relevantes, fragmentos documentales y limitaciones. La presencia de limitaciones no degrada la utilidad de la ficha; al contrario, permite al revisor saber qué no puede concluirse con los datos disponibles.

7.4. Diseño de visualización y explicabilidad

La visualización del prototipo sigue un principio de progresividad. El usuario no debe comenzar por una tabla extensa de contratos, sino por una síntesis que indique dónde concentrar la atención. Desde esa visión general se desciende al ranking, después al detalle de contrato y finalmente a evidencias concretas. Esta progresividad reduce carga cognitiva y evita que la interfaz se convierta en una copia gráfica de los Parquet.

La explicabilidad se organiza en tres niveles. El primer nivel es **tabular**: reglas activadas, valores observados, pesos y score. El segundo nivel es **relacional**: comprador, adjudicatario, contratos conectados, comunidad y métricas de red. El tercer nivel es **documental**: fragmentos, citas y ficha generada. Esta separación permite que un revisor identifique si el riesgo procede de datos estructurados, de posición en la red o de evidencias textuales.

La interfaz también debe evitar diseños que induzcan conclusiones indebidas. Por ejemplo, no se emplean colores o textos que etiqueten un contrato como fraudulento; se utilizan niveles de riesgo y prioridad. Tampoco se ocultan los contratos no evaluables: si una regla no pudo calcularse, esa información aparece como parte del diagnóstico. En un sistema de apoyo a auditoría, la ausencia de dato es una señal de calidad, no un detalle secundario.

En la versión de cierre, Streamlit cubre esta necesidad de forma suficiente para una defensa académica y una demo local. Tecnologías como Sigma.js o una API FastAPI pueden mejorar una versión productiva, pero no son necesarias para demostrar el flujo de revisión humana asistida. Esta decisión mantiene el prototipo ejecutable y reduce dependencias, sin cerrar la puerta a una evolución posterior.

7.5. Trazabilidad, calidad y reproducibilidad

La trazabilidad se considera un requisito de diseño. Cada resultado debe poder reconstruirse desde los datos de origen. Para ello, el prototipo registrará: versión del dataset, hash de ficheros, fecha de ejecución, versión de reglas, pesos de scoring, modelo LLM utilizado, prompt versionado y fragmentos documentales recuperados.

Los criterios mínimos de calidad son:

- completitud de campos OCDS críticos superior al 90 %;
- coherencia temporal superior al 98 %;
- documentación de nulos y campos no disponibles;
- explicación individual de cada red flag activada;
- fichas generadas solo con evidencias recuperadas y citables;
- separación entre hechos observados, inferencias analíticas y limitaciones.

7.6. Orquestación batch, planificación reproducible y evidencia operativa

El prototipo incorpora un batch con modos semanal y mensual. La implementación comprueba la existencia de entradas y compara metadatos, tamaños, fechas y hashes; cuando no detecta cambios relevantes, registra la ejecución como **skipped**. Cuando procede, ejecuta Agent 1 y genera un manifest con las fuentes revisadas, el estado, los avisos y las rutas de los artefactos.

El batch no ejecuta automáticamente Agent 2, Agent 3 ni Agent 4. En su lugar, incorpora **derived_rebuild_plan**, que registra el estado previsto de las salidas downstream; Agent 2 se lanza mediante su comando específico y Agent 4 mantiene una ruta manual o de demo. Tampoco existe un **freeze_manifest.json** implementado en la versión evaluada. El snapshot o congelado final se conserva como criterio metodológico para una entrega reproducible, no como artefacto ya generado por el código.

La persistencia de estado en **run_batch_state.json** y el manifest por batch en **data/manifest/batches/<run_mode>/<batch_id>/manifest.json** permiten auditar cada ejecución y decidir qué componentes deben regenerarse sin atribuir al batch una orquestación downstream que no realiza.

7.7. Cierre integrado de Agent3 y Agent4

El bloque de grafos y documental quedó cerrado como una demo integrada reproducible. Agent 3 generó el grafo de relaciones, las métricas de red y las *features* por **contract_key_canon**; Agent 4 combinó el contrato canónico, el *score* de Agent 2, las métricas de Agent 3 y las evidencias documentales recuperadas para construir una ficha de caso trazable.

La integración se apoyó en una muestra sintética mínima, suficiente para demostrar el flujo extremo a extremo sin introducir ruido adicional en la defensa técnica. El resultado documentado mostró 11 nodos, 13 aristas, 3 métricas contractuales y 3 filas de *features* en Agent 3, junto con una ficha integrada en Agent 4 para PW-2024-0001 con dos evidencias documentales y dos citas.

La validación técnica del cierre se realizó con pruebas unitarias y reglas de estilo sobre los módulos de Agent 3 y Agent 4, y dejó alineado el bloque de relaciones y el bloque

documental con el contrato canónico y con el resto del sistema.

7.8. Implementación final del prototipo

La implementación final del prototipo consolida el trabajo desarrollado en los distintos bloques del sistema y constituye la base técnica de esta memoria. El resultado no se limita a cuatro scripts aislados, sino que se organiza como un paquete Python con CLI, parsers de fuentes, agentes, servicios opcionales, tests, documentación operativa y una interfaz local de demostración.

El proyecto se estructura bajo el paquete `scr/procurewatch`. La CLI central se implementa en `scr/procurewatch/cli.py`; los conectores de fuentes se ubican en `scr/procurewatch/data_sou`; los agentes se separan en `agent1`, `agent2`, `agent3` y `agent4`; la ejecución batch reside en `batch.py`; y la configuración de servicios se centraliza en `settings.py`. En la raíz del proyecto se incluyen `pyproject.toml`, `docker-compose.yml`, `README.md`, `ARCHITECTURE.md`, `SETUP.md`, `AGENTS.md` y una carpeta `docs/` con hojas de ruta, seguimiento de agentes, planes técnicos y documentos de cierre. La interfaz de demostración se encuentra en `frontend/agent3_demo.py`.

La Tabla 7.3 resume los componentes implementados y su función en el sistema. Esta tabla distingue entre tecnologías obligatorias para el flujo reproducible y servicios opcionales utilizados cuando el entorno de ejecución dispone de ellos.

Tabla 7.3: Componentes implementados en ProcureWatch Analytics

Componente	Ruta principal	Función verificada
CLI	<code>scr/procurewatch/cli.py</code>	Expone comandos de diagnóstico, normalización, ejecución de agentes, batch, RAG, evaluación y carga Neo4j.
Fuentes BOE	<code>data_sources/boe.py</code>	Normaliza el CSV BOE 2014–2024, extrae contratos y líneas de adjudicación CPV 71 y genera reportes de calidad.
Fuentes PLACE	<code>data_sources/place.py</code> , <code>place_normalize.py</code>	Gestiona descarga y lectura de fuentes PLACE, parsing de ZIP/Atom/XML, filtrado CPV y deduplicación.

Componente	Ruta principal	Función verificada
Fuentes OpenTender	<code>data.sources/opentender.py</code>	Descubre y procesa ficheros JSONL/GZ/ZIP de OpenTender, normalizando registros CPV 71 para el canónico.
Agent1	<code>agent1/pipeline.py</code>	Construye el canónico de contratos, cobertura entre fuentes, datasets analíticos y reportes de calidad.
Esquema analítico	<code>agent1/analytical_schema.py</code>	Define los campos mínimos de CONTRATO y ADJUDICATARIO definidos en el alcance del TFM.
Persistencia relacional	<code>db.py</code>	Inserta tablas analíticas de Agent1 en PostgreSQL cuando el servicio está disponible.
Agent2 estable	<code>agent2/pipeline.py</code>	Calcula red flags y scores sobre el canónico de Agent1 con reglas versionadas.
Agent2 versión evaluada	<code>agent2/mvp_pipeline.py</code>	Ejecuta RF-01 a RF-06 y genera flags por contrato, scores por contrato y reporte JSON.
Agent3	<code>agent3/</code>	Construye grafo, nodos, aristas, métricas, comunidades Louvain, resumen de red y features para otros agentes.
Neo4j opcional	<code>agent3/neo4j_store.py</code>	Define constraints, carga nodos/aristas y ejecuta consultas de control cuando Neo4j está activo.
Agent4	<code>agent4/</code>	Carga corpus, fragmenta documentos, recupera evidencias, genera contexto de caso y evalúa la calidad local del RAG.
Qdrant/Ollama opcionales	<code>agent4/qdrant_store.py</code> , <code>embeddings.py</code> , <code>generation.py</code>	Indexación vectorial y generación local si los servicios existen; fallback determinista en tests.
Batch	<code>batch.py</code>	Gestiona la ejecución de Agent1 por lote, genera manifests y hashes, registra skipped y planifica el estado downstream.

Componente	Ruta principal	Función verificada
Dashboard	<code>frontend/agent3_demo.py</code>	Interfaz Streamlit con pestañas de resumen, contratos priorizados, caso, relaciones, evidencias y trazabilidad.
Tests	<code>tests/</code>	Pruebas automatizadas de agentes, fuentes, batch, CLI, PostgreSQL, Neo4j, RAG y configuración.

7.9. Flujo extremo a extremo del prototipo

El flujo extremo a extremo del prototipo se diseñó con una restricción central: cada agente debe producir un artefacto legible por el siguiente agente sin depender de estado implícito. Esta decisión permite aislar errores, repetir una fase concreta y defender el sistema ante un tercero sin necesidad de ejecutar toda la cadena completa en cada revisión. La Tabla 7.4 resume las entradas, procesos y salidas de cada etapa.

Tabla 7.4: Flujo extremo a extremo del prototipo

Etapas	Entrada	Proceso	Salida
Fuentes	BOE, PLACE, Open-Tender	Parsing, normalización de tipos, filtrado CPV 71, deduplicación.	Tablas por fuente.
Agent1	Tablas normalizadas	Construcción de <code>contract_key_canon</code> , esquema analítico, calidad y cobertura.	Canónico y reports.
Agent2	Canónico Agent1	Aplicación de RF-01 a RF-06, score 0–100 y nivel de riesgo.	Flags, scores y reporte.
Agent3	Canónico o muestra demo	Grafo comprador-contrato-adjudicatario-CPV-fuente, métricas y comunidades.	Nodos, aristas, métricas y features.

Etapa	Entrada	Proceso	Salida
Agent4	Contrato, score, grafo y corpus	Recuperación de fragmentos, composición de contexto y ficha explicativa.	Evidencias, citas y ficha.
Dashboard	Outputs Agent2– Agent4	Visualización, filtros, caso seleccionado y trazabilidad.	Exploración humana.

Esta organización evita dos problemas frecuentes en prototipos analíticos. El primero es la dependencia de notebooks manuales: si una fase solo existe como celdas ejecutadas de forma interactiva, la trazabilidad desaparece. El segundo es la mezcla de cálculo y visualización: si el dashboard calcula resultados en tiempo real a partir de datos brutos, resulta difícil saber qué versión de reglas se aplicó. En ProcureWatch, el dashboard consume artefactos ya calculados; su función es explicar y explorar, no redefinir el resultado analítico.

7.10. Contratos de datos entre agentes

Para que el prototipo sea reproducible, cada agente se diseña con un contrato de datos explícito: entradas esperadas, columnas mínimas, salidas generadas y condiciones bajo las cuales una fila puede considerarse evaluable. Esta decisión es más importante que la tecnología concreta de persistencia. Si el contrato de datos está bien definido, el sistema puede ejecutarse con ficheros Parquet, con PostgreSQL o con servicios auxiliares sin cambiar la semántica de los resultados.

El contrato principal es el que conecta Agent1 y Agent2. Agent1 debe entregar una tabla canónica con identificadores, fuente, organismo, adjudicatario, CPV, procedimiento, importes, fechas y campos de trazabilidad. Agent2 no debe leer directamente ficheros brutos de BOE, PLACE u OpenTender, porque eso mezclaría normalización y scoring. Esta separación permite sustituir o corregir un parser sin modificar las reglas de riesgo, siempre que el canónico mantenga el mismo contrato de columnas.

La Tabla 7.5 resume los contratos operativos utilizados por el prototipo.

Tabla 7.5: Contratos de datos entre agentes del prototipo

Interfaz	Entrada mínima	Salida esperada	Criterio de control
Fuentes– Agent1	Ficheros BOE, PLA- CE u OpenTender con metadatos de fuente.	Tablas por fuente norma- lizadas y canónico.	Filas procesadas, errores de parseo, hashes y rutas de entrada.
Agent1–Agent2	<code>agent2_contracts_canonical_perquet</code>	Recall por queries contrato.	Campos obligatorios, contratos evaluables y reglas activadas.
Agent2–Agent3	Canónico y, cuando existe, score por con- trato.	Nodos, aristas, comuni- dades, métricas y featu- res relacionales.	Integridad de claves, au- sencia de nodos huérfa- nos y consistencia de aristas.
Agent2/3– Agent4	Caso priorizado, sco- re, relaciones y corpus documental.	Contexto recuperado, evidencias, citas y ficha de caso.	Evidencia enlazada a fragmentos y adverten- cias si falta contexto.
Agentes– Dashboard	Artefactos Par- quet/JSON generados por el pipeline.	Vistas de resumen, deta- lle, relaciones, evidencias y trazabilidad.	Lectura sin recalcular re- glas ni alterar outputs analíticos.

El concepto de *contrato evaluable* evita una fuente habitual de error en sistemas de riesgo: tratar la ausencia de datos como si fuera evidencia negativa o positiva. En ProcureWatch, una regla puede quedar en tres estados. El primer estado es **activada**, cuando los datos disponibles cumplen la condición definida por la regla. El segundo es **no activada**, cuando los datos permiten evaluarla y no se observa el patrón de riesgo. El tercero es **no evaluable**, cuando faltan campos o existe una inconsistencia que impide calcularla con seguridad. Esta tercera situación no debe ocultarse, porque puede ser relevante para la calidad de datos y para la revisión humana, pero tampoco debe convertirse automáticamente en una acusación.

La decisión de conservar estados no evaluables afecta especialmente a tres grupos de variables. En primer lugar, las fechas: si no existen fecha de publicación y fecha de adjudicación comparables, no se calcula una duración fiable. En segundo lugar, los identificadores fiscales: si no hay NIF, no puede afirmarse que un adjudicatario sea el mismo en distintas fuentes solo por similitud textual. En tercer lugar, los importes: si falta importe estimado o adjudicado, no se debe calcular una desviación aparente. El sistema prefiere devolver

una limitación trazable antes que completar artificialmente el dato.

Esta política también facilita la integración con servicios opcionales. PostgreSQL puede persistir las tablas analíticas, Neo4j puede explorar relaciones, Qdrant puede indexar fragmentos documentales y Ollama puede generar explicaciones, pero ninguno de esos servicios debe cambiar el estado evaluable de una regla. Los servicios auxiliares enriquecen consulta, exploración o explicación; la definición de los datos y de las reglas permanece en los artefactos versionados del pipeline.

7.11. Implementación detallada de los agentes

La implementación final del prototipo se apoya en cuatro agentes con contratos de entrada y salida explícitos. Esta decisión es metodológicamente relevante porque permite evaluar cada fase por separado: Agent1 se valida por calidad y cobertura de datos; Agent2 por coherencia de reglas, scores y niveles de riesgo; Agent3 por consistencia de nodos, aristas y métricas; y Agent4 por trazabilidad de evidencias y generación de contexto de caso. El resultado no es una cadena opaca, sino una composición de artefactos auditables.

7.11.1. Agent1: ingesta, normalización y canónico

Agent1 concentra la mayor parte del riesgo de datos del sistema. Sus entradas proceden de BOE, PLACE y OpenTender, pero cada fuente presenta una estructura distinta: CSV u OCDS tabular en el caso BOE, paquetes XML/Atom y ZIP en PLACE, y registros JSONL o comprimidos en OpenTender. Por ello, la implementación no intenta tratar todas las fuentes con un parser genérico, sino con conectores específicos en `scr/procurewatch/data.sources/`. Cada conector genera salidas normalizadas con una semántica común: fuente, identificador de registro, organismo, proveedor, CPV, importes, fechas, procedimiento y campos auxiliares de trazabilidad.

El artefacto central de Agent1 es `agent2_contracts_canonical.parquet`. Este fichero cumple dos funciones. En primer lugar, actúa como frontera estable entre ingesta y scoring: Agent2 no necesita conocer cómo se descargaron o parsearon las fuentes, sino que consume una tabla ya normalizada. En segundo lugar, conserva la columna `contract_key_canon`, que permite deduplicar por fuente, medir cobertura y conectar los agentes posteriores. La política de esta clave es deliberadamente conservadora: si no existe evidencia suficiente para considerar dos filas como el mismo contrato, se evita forzar el enlace.

Agent1 también produce salidas de diagnóstico. El reporte de ejecución registra fuentes procesadas, filas generadas, rutas y errores; el resumen de calidad mide completitud, validez de identificadores y coherencia temporal; y la cobertura por clave canónica muestra intersecciones entre BOE, PLACE y OpenTender. Esta capa es esencial para la defensa del TFM porque permite explicar no solo cuántos contratos se procesan, sino qué parte del análisis queda limitada por datos ausentes, diferencias de granularidad o falta de intersección entre fuentes.

7.11.2. Agent2: red flags, score y priorización

Agent2 transforma el canónico de Agent1 en señales de riesgo. La versión evaluada implementa RF-01 a RF-06 como reglas trazables y evita presentar como implementadas las red flags no verificadas en la versión de cierre. Las reglas activadas generan dos tipos de artefactos: una tabla de flags por contrato y una tabla de scores. La primera permite saber qué indicador se activó, con qué versión de regla y sobre qué contrato; la segunda resume el *risk score*, el nivel de riesgo, el número de señales y los principales factores explicativos.

El diseño de scoring se mantiene explicable. Las reglas no se sustituyen por un modelo opaco porque el proyecto no dispone de etiquetas supervisadas de fraude confirmado. En su lugar, el score agrega pesos asociados a señales comprensibles para revisión humana: baja concurrencia, procedimientos de mayor riesgo, recurrencia comprador-proveedor, concentración, desviación entre importe estimado y adjudicado y patrones temporales. Cuando una regla no puede evaluarse por falta de campos, el contrato no debe recibir una penalización artificial; el estado no evaluable forma parte de la salida analítica.

Agent1 genera una tabla analítica de adjudicatarios y Agent2 utiliza variables de proveedor para evaluar recurrencia y concentración en cada contrato. Estas salidas permiten filtrar o agrupar la revisión por adjudicatario, pero la versión evaluada no publica un artefacto final versionado de score agregado por proveedor. Por ello, la priorización demostrada y evaluada se presenta a nivel de contrato.

7.11.3. Agent3: grafo derivado y variables relacionales

Agent3 construye un grafo a partir del canónico o de una muestra integrada de demostración. Sus nodos representan contratos, compradores, adjudicatarios, CPV y fuentes; sus aristas expresan relaciones observadas en los datos normalizados. Esta decisión evita que el grafo cree una fuente paralela de verdad: si un contrato no aparece correctamente

normalizado en Agent1, Agent3 no debe inventar relaciones para completarlo.

La salida de Agent3 se divide en varios artefactos: nodos, aristas, métricas contractuales, features para scoring y resumen de red. Las métricas incluyen grado, relaciones de comprador y proveedor, comunidad y variables de contexto. En entornos donde Neo4j está disponible, el sistema puede cargar nodos y aristas para exploración mediante consultas Cypher; cuando no lo está, el flujo sigue siendo reproducible con Parquet y NetworkX. Por tanto, Neo4j mejora la exploración, pero no es condición obligatoria para ejecutar el prototipo.

Las variables relacionales generadas por Agent3 son especialmente útiles para contextualizar RF-03 y RF-04: recurrencia, concentración y patrones comprador-proveedor. Sin embargo, la memoria evita interpretar la centralidad o la pertenencia a una comunidad como prueba de irregularidad. Un actor central puede ser simplemente un proveedor relevante en un nicho técnico; el valor analítico aparece cuando esa posición se combina con red flags tabulares y evidencia documental.

7.11.4. Agent4: evidencia documental y ficha de caso

Agent4 aporta la capa documental del sistema. Su entrada combina un contrato priorizado, el score de Agent2, las features de Agent3 y un corpus de documentos o fragmentos asociados. El flujo incluye carga de corpus, fragmentación, recuperación de evidencias, composición de contexto y generación de una ficha de caso. Qdrant y Ollama se soportan como servicios opcionales para búsqueda vectorial y generación local; en pruebas y demo se mantiene un fallback determinista para no hacer depender la validación de servicios externos.

La ficha de caso no es una conclusión jurídica. Su estructura separa datos del contrato, score, red flags, relaciones de grafo, evidencias recuperadas, citas y advertencias. Esta separación reduce el riesgo de que el LLM introduzca afirmaciones no respaldadas: el texto generado debe apoyarse en fragmentos recuperados o en variables calculadas por los agentes anteriores. Si no hay evidencia suficiente, la respuesta correcta es reflejar esa limitación.

En conjunto, Agent4 convierte el ranking en un objeto revisable. Un auditor no recibe solo una puntuación, sino un resumen estructurado que permite comprobar por qué el caso fue priorizado, qué datos faltan, qué evidencias se recuperaron y qué aspectos requieren revisión humana.

7.11.5. Separación entre capa analítica y capa de demostración

La implementación distingue entre dos niveles de uso. El primer nivel es el flujo analítico general, que opera sobre los artefactos disponibles en `data/processed`. Este flujo es el que produce resultados cuantitativos sobre miles de registros y permite medir calidad, cobertura y distribución de riesgo. El segundo nivel es la demo integrada Agent3–Agent4, que usa una muestra pequeña y controlada para enseñar el comportamiento de grafo, evidencias y dashboard sin obligar a procesar datasets grandes durante la defensa.

Esta separación no genera dos sistemas distintos. El código reutiliza los mismos contratos de datos, los mismos nombres de columnas y la misma clave `contract_key_canon`. Lo que cambia es el tamaño y finalidad de los artefactos: el flujo general sirve para resultados; la demo sirve para explicar el recorrido de un caso concreto. Esta distinción es relevante porque evita confundir una muestra didáctica con una ejecución completa del pipeline.

7.12. Política de clave canónica y trazabilidad del cruce

La clave `contract_key_canon` es el mecanismo utilizado para identificar contratos de forma estable dentro del sistema. Su objetivo no es sustituir los identificadores oficiales de cada fuente, sino proporcionar una referencia común para deduplicación, cobertura y conexión entre agentes. En contratación pública española esta tarea no es trivial porque las fuentes pueden publicar un mismo hecho contractual con granularidades distintas: anuncio, expediente, lote, adjudicación, formalización o línea de adjudicación.

La política aplicada es conservadora. Cuando existe un identificador de expediente o registro fiable, se prioriza; cuando no existe, se combina con organismo, fecha, fuente e identificador de entrada. Antes de construir la clave se normalizan textos: mayúsculas, eliminación de espacios superfluos, reducción de caracteres no estables y fechas en formato comparable. Si faltan campos críticos, el sistema conserva la fila con su identificador de fuente, pero no fuerza un cruce interfuente.

La Tabla 7.6 resume la regla funcional.

Tabla 7.6: Política funcional de `contract_key_canon`

Fuente	Campos priorizados	Criterio de seguridad
BOE	Identificador de aviso, expediente, línea de adjudicación, organismo y fecha.	Preferencia por unidad de adjudicación cuando existe.
PLACE	Expediente, DIR3 u organismo, fecha de publicación e identificador de entrada.	Conserva granularidad de perfil/expediente sin inferir equivalencias dudosas.
OpenTender	Identificador de registro, comprador, fecha y fuente.	Mantiene trazabilidad europea y evita emparejar solo por nombres similares.
Canónico	Fuente, identificador normalizado, organismo y fecha cuando procede.	No hay match si la evidencia no permite equivalencia razonable.

La consecuencia práctica de esta decisión es que el sistema prefiere subestimar coincidencias antes que introducir falsos emparejamientos. En un contexto de auditoría, esta decisión es defendible: si se unieran por error dos contratos diferentes, se contaminarían las red flags, las métricas de red y las evidencias documentales. En cambio, si dos registros potencialmente relacionados quedan sin unir, el resultado es una limitación de cobertura visible y documentada.

7.12.1. Trazabilidad cuando no hay intersecciones

El hecho de que una ejecución no encuentre intersecciones útiles entre BOE, PLACE y OpenTender no debe quedar como un resultado opaco. Por ello, Agent1 produce un artefacto específico de cobertura por `contract_key_canon`. Este artefacto permite saber cuántas claves proceden de cada fuente, cuáles son las intersecciones y cuál es el universo total. En la ejecución disponible del prototipo, esa trazabilidad muestra un universo de 17.748 claves, 3.883 claves BOE, 12.605 PLACE, 25.057 OpenTender y matching conservador entre fuentes.

Este resultado afecta de forma distinta a cada agente. Agent1 queda afectado porque no puede afirmar integración contractual fuerte entre fuentes. Agent2 puede seguir calculando señales intrafuente sobre contratos reales, pero no debe afirmar contraste entre fuentes.

Agent3 puede construir grafos con entidades del canónico, aunque las relaciones interfuerza deben interpretarse con cautela. Agent4 puede recuperar y citar evidencias asociadas a la clave disponible, pero no debe mezclar documentos de fuentes distintas como si fueran necesariamente el mismo contrato si la clave no lo acredita.

7.13. Gobernanza de evidencias y revisión humana

ProcureWatch se diseña bajo el principio de supervisión humana. El score no cierra un caso, no declara fraude y no recomienda una sanción. Su función es ordenar prioridades de revisión y reunir evidencia trazable. Para que esta función sea defendible, la memoria distingue tres tipos de afirmaciones:

1. **Hechos observados:** valores procedentes de fuentes o artefactos, como importe, procedimiento, fecha, adjudicatario, número de ofertas o fuente de origen.
2. **Señales analíticas:** red flags, scores, niveles de riesgo, métricas de red o estado no evaluable, calculados por reglas o algoritmos documentados.
3. **Inferencias explicativas:** texto generado o redactado para ayudar a entender el caso, siempre apoyado en hechos o señales y nunca presentado como prueba jurídica.

Esta separación responde a una exigencia normativa y metodológica. Un contrato con riesgo alto puede ser perfectamente legal si existe justificación administrativa. Del mismo modo, un contrato con riesgo bajo puede contener problemas que el sistema no observa por falta de datos. Por tanto, la salida correcta de ProcureWatch no es “fraude/no fraude”, sino una ficha revisable que indica: qué se observó, qué regla se activó, con qué datos se calculó y qué limitaciones afectan al caso.

7.14. Evaluación por objetivos específicos

Los objetivos definidos al inicio del TFM pueden evaluarse de forma más precisa al contrastarlos con la versión de cierre del prototipo. La Tabla 7.7 sintetiza ese contraste.

Tabla 7.7: Evaluación de objetivos específicos frente a la implementación

Obj.	Resultado implementado	Valoración
OE1	Análisis de fuentes BOE, PLACE y OpenTender; foco CPV 71; perfilado de calidad.	Cumplido como análisis de disponibilidad y calidad, con limitaciones documentadas.
OE2	Esquema OCDS adaptado, canónico, tablas analíticas de contrato/adjudicatario y grafo derivado.	Cumplimiento parcial: Parquet está evaluado; PostgreSQL y Neo4j son persistencias opcionales y no existe enriquecimiento societario cerrado.
OE3	Pipeline reproducible con parsers, canónico, reportes y persistencia opcional.	Cumplido parcialmente en calidad: el pipeline existe, pero algunas métricas no alcanzan umbral.
OE4	RF-01 a RF-06 y scoring 0–100 por contrato.	Cumplido en la versión evaluada; el catálogo completo de 15 RF y el scoring agregado por adjudicatario quedan como extensiones.
OE5	Grafo con nodos, aristas, métricas, comunidades y features.	Cumplido como grafo derivado; Neo4j es opcional.
OE6	Agent4 con corpus, chunks, retrieval, contexto de caso y citas.	Cumplido como componente documental del prototipo; evaluación masiva RAG queda limitada.
OE7	Dashboard Streamlit integrado con resumen, ranking, caso, relaciones, evidencias y trazabilidad; validación headless con 10 comprobaciones y 0 excepciones.	Cumplimiento funcional para demo local; sin SUS, Likert ni medición temporal con usuarios.
OE8	Validación con suites mínima/completa, benchmark en dos modos, modularidad y diez fichas cualitativas.	Cumplimiento parcial: valida funcionamiento, priorización y explicación, sin etiquetas de fraude, revisión experta, RAGAS amplio ni estudio formal de usabilidad.

Esta evaluación permite defender el trabajo sin sobredimensionar el resultado. La implementación cumple el núcleo de los objetivos definidos inicialmente para un prototipo

reproducible, pero no convierte todas las líneas de investigación planteadas inicialmente en producto final productivo. La diferencia entre objetivo ambicioso y cierre implementado queda documentada como adaptación de alcance.

7.15. Decisiones de diseño por agente

La arquitectura multiagente no se adoptó solo como división narrativa del trabajo, sino como una forma de reducir acoplamiento entre problemas diferentes. La ingesta de datos, el scoring, el grafo y la recuperación documental tienen ritmos de cambio distintos. Si se hubieran implementado en un único script monolítico, cualquier ajuste de una regla de riesgo podría afectar al parsing de fuentes o al dashboard. La separación por agentes permite modificar una capa sin invalidar todo el flujo.

7.15.1. Agent1: priorizar trazabilidad frente a completitud aparente

Agent1 se diseñó con una regla conservadora: no completar datos mediante inferencias no trazables. Si una fuente no proporciona NIF, fecha de adjudicación o número de ofertas, el campo se mantiene como nulo o no evaluable. Esta decisión reduce algunas métricas de calidad, pero preserva la integridad del análisis. En un sistema de revisión de contratación pública, una imputación no justificada podría producir red flags artificiales.

También se decidió conservar artefactos intermedios por fuente. Aunque el resultado principal es el canónico, las tablas BOE, PLACE y OpenTender permiten auditar la transformación. Si un contrato aparece con un importe inesperado o sin adjudicatario, puede revisarse la fuente normalizada antes de atribuir el problema al scoring. Esta trazabilidad es especialmente importante porque los errores de datos se propagan: un fallo en importe afecta a RF-05; un fallo en organismo afecta a concentración; y un fallo en clave canónica afecta a grafo y RAG.

7.15.2. Agent2: scoring explicable antes que modelo opaco

Agent2 prioriza reglas deterministas y ponderadas porque el proyecto no dispone de etiquetas supervisadas de fraude. Un modelo puramente no supervisado podría ordenar anomalías, pero sería más difícil explicar por qué un contrato aparece arriba en el ranking. La decisión de implementar RF-01 a RF-06 responde a esa necesidad de explicabilidad: cada señal tiene una motivación comprensible para un auditor, un campo de entrada y

una consecuencia en el score.

El nivel de riesgo bajo, medio, alto o crítico se interpreta como ayuda de priorización. La clasificación no sustituye una revisión jurídica porque no incorpora todos los elementos del expediente: informes técnicos completos, comunicaciones internas, justificación administrativa, posibles recursos o información societaria no publicada en las fuentes analizadas. Por tanto, el valor de Agent2 está en reducir el volumen de revisión y ordenar indicios, no en cerrar una conclusión sancionadora.

7.15.3. Agent3: grafo derivado y no fuente primaria

Agent3 no descarga datos nuevos ni redefine contratos. Su entrada es el canónico o una demo canónica. Esta decisión evita que el grafo cree una versión paralela de la verdad. Las relaciones que aparecen en nodos y aristas son derivadas de los contratos ya normalizados. Así, si Agent1 marca una clave como no cruzada, Agent3 no debe resolver el cruce por su cuenta salvo que se implemente una fase explícita de reconciliación.

Las métricas de red se utilizan como contexto. Un adjudicatario con alta centralidad puede ser un proveedor legítimamente relevante en un sector especializado. Del mismo modo, una comunidad densa puede reflejar estructura territorial o especialización técnica. Por ello, las variables de grafo se interpretan junto con red flags tabulares y evidencias documentales, no como indicadores aislados.

7.15.4. Agent4: evidencia citada y control de alucinaciones

Agent4 se limita a producir contexto explicativo basado en documentos o fragmentos recuperados. La arquitectura evita que el LLM genere conclusiones sin soporte: la ficha de caso debe incluir citas, identificadores de fragmento y advertencias cuando la evidencia no es suficiente. En la demo integrada, esta función se demuestra con corpus pequeño y fallback offline, lo que permite validar el flujo sin depender de un servicio externo.

El papel del LLM es, por tanto, lingüístico y documental. Resume, organiza y redacta; no decide el score. Esta separación es coherente con el AI Act y con el principio de revisión humana. Si en el futuro se ampliara el corpus documental, sería necesario medir fidelidad, cobertura de contexto y tasa de citas válidas antes de utilizar las fichas en un entorno operativo.

7.16. Controles de calidad y riesgos técnicos

El sistema incorpora controles técnicos para evitar que las limitaciones de datos se conviertan en resultados engañosos. La Tabla 7.8 resume los principales riesgos y las medidas de mitigación aplicadas en el prototipo.

Tabla 7.8: Riesgos técnicos y controles aplicados

Riesgo	Impacto	Control aplicado
Campos ausentes	Red flags no evaluables o scores incompletos.	Marcado explícito como <code>no evaluable</code> ; no imputación artificial.
Duplicados por fuente	Sobrerrepresentación de contratos o adjudicatarios.	Deduplicación por fuente y clave canónica.
Matching incorrecto	Mezcla de contratos distintos y evidencias erróneas.	Política conservadora de <code>contract_key_canon</code> .
Reglas opacas	Dificultad de defensa ante auditor o tribunal.	RF versionadas, tabla de reglas y reporte JSON.
Servicios no disponibles	Fallo de demo si Neo4j/Qdrant/Ollama no están activos.	Servicios opcionales y fallback determinista en tests.
Resultados pesados en Git	Repositorio inmanejable o datos duplicados.	<code>.gitignore</code> para datasets y outputs pesados.
Alucinación LLM	Ficha con afirmaciones no respaldadas.	Uso de evidencias recuperadas, citas y separación entre hechos e inferencias.
Confusión fraude/riesgo	Interpretación jurídica indebida.	Redacción como priorización de revisión humana.

Estos controles no eliminan todas las limitaciones, pero hacen que sean visibles. La calidad de un prototipo de auditoría no depende solo de producir rankings, sino de saber cuándo no se puede evaluar un contrato y por qué. En ese sentido, el estado `not evaluable` es una salida válida: informa de que el dato disponible no permite aplicar una regla sin riesgo de error.

7.17. Lectura operativa de los resultados

La evaluación disponible de Agent2 comprende los 3.437 contratos del canónico de muestra versionado y la demo integrada de 3 contratos. La primera ejecución permite medir evaluabilidad, frecuencia de flags, distribución del score y sensibilidad de umbrales; la segunda conserva un caso explicativo completo con grafo y evidencia documental. Estos resultados son una evaluación proxy reproducible y no una estimación de fraude ni una ejecución sobre el canónico completo de 51.720 registros. El objetivo práctico demostrado es generar un ranking contractual que permita seleccionar casos para una revisión más profunda.

La interpretación operativa puede organizarse en tres niveles:

Nivel 1: cribado automático. Se ejecuta el pipeline sobre el canónico y se obtienen flags, scores y niveles. Esta fase es automática, reproducible y no concluye fraude.

Nivel 2: revisión analítica. Un analista revisa contratos alto/crítico, comprueba campos no evaluables, interpreta recurrencias y observa el entorno de grafo.

Nivel 3: revisión documental. Para casos priorizados, se recuperan documentos y se contrasta si las señales tabulares tienen justificación administrativa o indicios adicionales.

Esta lectura evita usar el sistema como caja negra. Un contrato crítico no es automáticamente irregular; es un contrato que merece atención antes que otros por acumulación de señales. Del mismo modo, un contrato sin score alto no queda exento de revisión si existe denuncia, alerta externa o información documental no incluida en el dataset.

8. Orquestación, reproducibilidad y artefactos

La reproducibilidad del prototipo no se limita a conservar el código. En ProcureWatch Analytics se apoya en comandos repetibles, artefactos Parquet y JSON, manifests, hashes de entrada y salida, reportes de calidad y una estrategia batch que evita reprocesar fuentes cuando no existen cambios. Este capítulo describe esa capa operativa y explica cómo se reconstruye una ejecución del prototipo.

8.1. Reproducibilidad y gestión de artefactos

La reproducibilidad se aborda mediante tres capas. La primera es la reproducibilidad de código: el proyecto fija dependencias en `pyproject.toml`, define un entrypoint de CLI y contiene tests automatizados. La segunda es la reproducibilidad de datos: los ficheros de entrada y salida se acompañan de rutas, hashes, manifests y reportes. La tercera es la reproducibilidad de ejecución: los comandos documentados permiten repetir fases concretas sin reescribir notebooks.

La decisión de no versionar datasets grandes en Git forma parte de esa estrategia. Un proyecto con Parquet pesados, ZIPs de fuentes y temporales de descarga sería difícil de revisar y sincronizar. En su lugar, se versionan código, esquemas, documentación y, cuando procede, manifests ligeros. Para una entrega final consolidada, el elemento adecuado no sería subir todos los datos al control de versiones, sino acompañar el TFM de un snapshot controlado o de instrucciones de regeneración con hashes verificables.

8.2. Estrategia batch de la versión de cierre

El sistema no se plantea como streaming. La contratación pública no requiere procesar eventos segundo a segundo para el objetivo de este TFM; requiere reproducibilidad, trazabilidad y capacidad de actualizar datos sin repetir trabajo innecesario. Por ello, la implementación utiliza un batch incremental para Agent 1 y genera un plan explícito de reconstrucción para los componentes downstream.

El código admite dos modos: **semanal**, que ejecuta Agent 1 si detecta cambios o se

fuerza la ejecución, y **mensual**, que permite además las descargas PLACE configuradas y ejecuta Agent 1 siempre que las entradas obligatorias estén disponibles. Ambos modos generan un manifest y un **derived_rebuild_plan**; ninguno ejecuta automáticamente Agent 2, Agent 3 o Agent 4.

La Tabla 8.1 resume la finalidad de cada modo.

Tabla 8.1: Modos de ejecución batch del prototipo

Modo	Objetivo	Salida principal	Plan downstream
Semanal	Detectar cambios sin reprocesar Agent 1 innecesariamente.	Manifest y estado de ejecución.	No planifica agentes posteriores; registra el bloqueo si faltan entradas.
Mensual	Revisar fuentes y ejecutar Agent 1 con descarga PLACE habilitada por defecto.	Manifest, reporte de Agent 1 y snapshots de entrada.	Planifica la reconstrucción si existe el canónico; no ejecuta otros agentes.

El estado **skipped** es importante porque también es un resultado. Si una fuente no cambia, la ejecución semanal no debe regenerar outputs por rutina, ya que podría introducir diferencias no explicadas por el dato. Registrar una ejecución omitida permite demostrar que el sistema revisó las fuentes y decidió, con criterios trazables, no recalcular. Esta propiedad es especialmente útil para un TFM con datasets grandes, porque evita descargas repetidas y reduce uso de disco.

8.3. Manifiestos, hashes y limpieza de temporales

Cada ejecución batch deja evidencia suficiente para responder a cuatro preguntas: qué fuentes se revisaron, cuáles cambiaron, qué artefactos se generaron y con qué versión del sistema. Para ello, el manifest de batch incluye identificador de ejecución, modo, rutas o URLs de entrada, tamaños, hashes, salidas, estado y versiones relevantes. No es necesario guardar todos los datos brutos en el repositorio para tener trazabilidad; basta con conservar el manifest y el snapshot correspondiente cuando se cierre una ejecución final.

La política de hashes cumple dos funciones. Primero, permite detectar cambios sin

depender solo del nombre del fichero, que puede permanecer constante aunque el contenido cambie. Segundo, permite verificar que un Parquet usado en resultados corresponde al mismo contenido que se documentó en el manifest. Esta verificación es especialmente importante cuando el equipo trabaja con datasets de distinto tamaño o cuando una parte de las fuentes se descarga bajo demanda.

Los temporales se tratan de forma separada. `data/tmp` se reserva para descargas, extracciones intermedias y ficheros de trabajo que no forman parte de la evidencia final. Al terminar una ejecución correcta, esos temporales deben eliminarse o quedar fuera del control de versiones. Esta política evita que el repositorio crezca con ZIPs, XMLs o Parquet intermedios, y permite repetir la ejecución sin mezclar resultados antiguos con resultados nuevos.

El snapshot final se interpreta como un criterio metodológico de cierre, no como un modo o artefacto implementado por `batch.py`. Una futura entrega congelada debería incluir el canónico consumido por Agent2, las salidas de red flags y scores por contrato, los reports, los manifests, los hashes, la versión de dependencias y los esquemas. Si se incorpora una ejecución más completa, ese snapshot debería regenerarse y las cifras de resultados actualizarse.

8.4. Comandos de ejecución y reproducción

El prototipo se ejecuta mediante una CLI Python. Esta decisión reduce la ambigüedad en la defensa técnica: cada parte relevante del sistema puede reproducirse con un comando versionado. La Tabla 8.2 recoge los comandos identificados en la implementación. Algunos comandos procesan datos reales y otros generan muestras o validan componentes de forma aislada. El código fuente de la implementación técnica de ProcureWatch Analytics se encuentra disponible en el repositorio del proyecto: <https://github.com/sebastianalfonsocorrea-byte/tfm-procurewatch.git>.

Tabla 8.2: Comandos disponibles en la CLI de ProcureWatch

Comando	Uso principal
<code>doctor</code>	Comprueba configuración del entorno y disponibilidad de servicios opcionales.
<code>make-agent1-sample</code>	Genera una muestra pequeña para pruebas de Agent1.

Comando	Uso principal
<code>normalize-boe</code>	Normaliza la fuente BOE.
<code>place-sources</code>	Inspecciona o descarga fuentes PLACE configuradas.
<code>normalize-place</code>	Normaliza ficheros PLACE.
<code>normalize-opentender</code>	Normaliza registros OpenTender.
<code>run-agent1</code>	Ejecuta la ingesta integrada y construye el canónico de Agent1.
<code>report-agent1-coverage</code>	Genera informe de cobertura y calidad de Agent1.
<code>report-agent1-source-diagnosis</code>	Genera diagnóstico de intersecciones y candidatos de matching entre fuentes.
<code>run-mvp</code>	Ejecuta un flujo compacto de Agent1 con persistencia opcional.
<code>run-agent2</code>	Calcula red flags y scores con el pipeline estable de Agent2.
<code>run-agent2-mvp</code>	Ejecuta la versión evaluada de Agent2 con RF-01 a RF-06.
<code>run-agent3</code>	Genera grafo, métricas, comunidades y features relacionales.
<code>agent3-load-neo4j</code>	Carga outputs de Agent3 en Neo4j si el servicio está disponible.
<code>agent4-smoke</code>	Comprueba disponibilidad básica de Qdrant/Ollama y fallback.
<code>agent4-build-manifest</code>	Construye manifiesto de corpus documental.
<code>agent4-index-corpus</code>	Indexa corpus documental para recuperación.
<code>agent4-run-flow</code>	Ejecuta el flujo RAG de Agent4.
<code>agent4-case-context</code>	Genera contexto integrado de un caso combinando contrato, score, grafo y evidencias.
<code>agent4-evaluate</code>	Evalúa localmente casos RAG con métricas proxy.
<code>run-benchmark</code>	Ejecuta el benchmark global de Agent1, Agent2, Agent3, Agent4 e integración.
<code>run-batch</code>	Ejecuta lotes incrementales con manifiestos, hashes y estado de ejecución.

Un ejemplo representativo de reproducción de la demo integrada es:

```
PYTHONPATH=scr python -m procurewatch.cli doctor
PYTHONPATH=scr python -m procurewatch.cli run-agent2-mvp \
  --input data/processed/agent2_contracts_canonical.parquet \
  --output-dir data/processed
PYTHONPATH=scr python -m procurewatch.cli run-agent3 \
```

```
--input data/processed/agent2_contracts_canonical.parquet \
--output-dir data/processed/agent3
PYTHONPATH=scr python -m procurewatch.cli run-integrated-demo
PYTHONPATH=scr python -m procurewatch.cli validate-dashboard-demo
```

El valor de este diseño no está solo en que los comandos existan, sino en que fuerzan una separación clara entre entradas, salidas y parámetros. En una herramienta de auditoría analítica, esta separación es esencial: permite repetir una ejecución, comparar dos versiones de reglas y saber qué artefacto se utilizó para calcular cada resultado.

8.5. Artefactos generados por el sistema

La salida del sistema se materializa en ficheros Parquet y JSON. El uso de Parquet permite manejar tablas analíticas con tipos de datos estables y bajo coste de lectura; los JSON se reservan para manifiestos, reportes de calidad, informes de ejecución y fichas explicativas. La Tabla 8.3 agrupa los artefactos principales.

Tabla 8.3: Artefactos principales producidos por ProcureWatch

Artefacto	Productor	Función
contracts.boe.parquet	BOE parser	Contratos normalizados desde BOE.
contracts.boe_cpv71.parquet	BOE parser	Subconjunto CPV 71.
contracts.boe_award_lines_cpv71.parquet	BOE parser	Líneas de adjudicación CPV 71, unidad preferida para análisis adjudicado.
contracts.place_cpv71.parquet	PLACE parser	Contratos PLACE filtrados por CPV 71.
contracts.opentender_2024_cpv71.parquet	OpenTender parser	Registros OpenTender españoles CPV 71.
agent2_contracts_canonical.parquet	Agent2	Dataset canónico consumido por Agent2.
contracts_analytical.parquet	Agent1	Tabla analítica de contratos alineada con el esquema objetivo del TFM.

Artefacto	Productor	Función
suppliers_analytical.parquet	Agent1	Tabla analítica de adjudicatarios.
agent1_run_report.json	Agent1	Informe de fuentes, versiones de parsers, filas procesadas y errores.
agent1_data_quality_summary.json	Agent1	Resumen de calidad, cobertura, completitud y coherencia.
agent1_contract_key_coverage.parquet	Agent1	Cobertura y cruce por contract_key_canon .
agent1_source_coverage_analysis.parquet	Agent1	Interpretación de cobertura aditiva, intersecciones y readiness institucional.
agent1_matching_diagnostics.parquet	Agent1	Intersecciones exactas y candidatos aproximados de matching entre fuentes.
agent2_risk_flags.parquet	Agent2	Red flags activadas por contrato.
agent2_risk_scores.parquet	Agent2	Score y nivel de riesgo por contrato.
agent2_run_report.json	Agent2	Informe de reglas, filas evaluables, distribución de riesgo y versión de scoring.
agent2_evaluation_report.json	Agent2	Evaluabilidad por regla y comparación de umbrales -10 %, base y +10 %.
agent3_nodes.parquet	Agent3	Nodos del grafo.
agent3_edges.parquet	Agent3	Aristas del grafo.
agent3_contract_metrics.parquet	Agent3	Métricas relacionales por contrato.
agent3_agent2_features.parquet	Agent3	Variables relacionales consumibles por scoring o explicación.
case_studies_report.json	Evaluación	Informe agregado y diez fichas individuales con selección 5/3/2, evidencia y advertencias.

Artefacto	Productor	Función
<code>agent4_case_context_integrated_demo.json</code>	Agent4	Ficha/contexto integrado de caso con evidencias y citas.
<code>agent4_evaluation_report.md</code>	Agent4	Evaluación de fidelidad, cobertura, trazabilidad y consistencia de citas.
<code>benchmark_report.md</code>	Benchmark	Evaluación global por agentes, matriz de alcance y conclusiones académicas.
<code>manifest.json</code>	Batch	Fuentes revisadas, hashes, entradas, salidas, estado y versión de ejecución.

Esta organización también permite separar resultados pesados de evidencia ligera. Los datasets completos no deben versionarse en Git; en cambio, los manifiestos, esquemas y reportes pequeños sí son adecuados para documentar reproducibilidad.

8.6. Fragmentos de implementación relevantes

El objetivo de esta sección no es reproducir listados completos, sino mostrar decisiones de implementación que explican el comportamiento del prototipo. Los fragmentos se muestran de forma resumida y proceden de componentes concretos del sistema; su función es ilustrar decisiones de diseño, no sustituir la documentación completa del código.

8.6.1. Esquema analítico ejecutable

Ruta: `scr/procurewatch/agent1/analytical_schema.py`. El primer fragmento corresponde al esquema analítico mínimo. La implementación no deja el modelo de datos como una tabla informal de la memoria, sino como una estructura ejecutable que los tests pueden comprobar.

```
CONTRACT_REQUIRED_FIELDS = (  
    "id_contrato",  
    "id_licitacion",  
    "organismo_contratante",  
    "codigo_organismo",
```

```
"nivel_administracion",  
"tipo_contrato",  
"procedimiento",  
"cpv_codigo",  
"importe_estimado",  
"importe_adjudicado",  
"fecha_publicacion",  
"fecha_adjudicacion",  
"numero_ofertas_recibidas",  
"nif_adjudicatario",  
"nombre_adjudicatario",  
"score_red_flags_total",  
"red_flags_activados",  
"nivel_riesgo",  
"score_centralidad_red",  
"comunidad_red",  
"fragmentos_documentales_recuperados",  
"fuentes_cruzadas",  
"estado_revision",  
)
```

Este fragmento es importante porque vincula directamente el alcance inicial con el código: los campos de `CONTRATO` no son una aspiración documental, sino una lista de requisitos que el pipeline puede validar. Además, cada campo queda asociado a un agente responsable: Agent1 cubre identificadores, fuentes, organismos, fechas e importes; Agent2 completa red flags y riesgo; Agent3 aporta centralidad y comunidad; Agent4 aporta fragmentos documentales; y la interfaz gestiona el estado de revisión.

8.6.2. Construcción conservadora de `contract_key_canon`

Ruta: `scr/procurewatch/agent1/pipeline.py`. La clave canónica se construye con criterios distintos por fuente y con fallback trazable. El fragmento siguiente resume la parte central de la función `_build_contract_keys`; se omiten detalles auxiliares de normalización para mantener legibilidad.

```
if source == "boe":
    file_id = _norm_text(id_frame["file_number"]).fillna("")
    buyer_name = _norm_text(id_frame["buyer_name"]).fillna("")
    date = _norm_date(_norm_text(id_frame["publication_date"])).fillna("")
    primary = file_id + "|" + buyer_name + "|" + date
    fallback = id_frame["contract_id"].astype("string")
    final = _norm_key_candidates(primary, fallback)
elif source == "place":
    contract_folder_id = _norm_text(id_frame["contract_folder_id"])
    buyer_dir3 = _norm_text(id_frame["buyer_dir3"]).fillna("")
    published_date = _norm_date(_norm_text(id_frame["published_date"])).fillna("")
    key_parts = contract_folder_id + "|" + buyer_dir3 + "|" + published_date
    fallback = _norm_text(id_frame["source_entry_id"]).fillna("")
    final = _norm_key_candidates(key_parts, fallback)
```

La decisión técnica consiste en priorizar identificadores fuertes cuando existen y combinar campos auxiliares solo como apoyo. Esta regla mejora la trazabilidad porque cada clave puede reconstruirse desde campos de origen. Su limitación es deliberada: no resuelve emparejamientos dudosos por similitud aproximada, por lo que puede dejar contratos sin cruce interfente.

8.6.3. Red flags y score de Agent2

Ruta: `scr/procurewatch/agent2/mvp_scoring.py`. Agent2 calcula el score con pesos explícitos y evidencias asociadas a cada regla. El siguiente fragmento muestra la lógica principal de las reglas evaluables a nivel de contrato individual: RF-01, RF-02, RF-05 y RF-06. RF-03 y RF-04 se aplican posteriormente en `agent2/mvp_pipeline.py`, donde existe contexto agregado suficiente para medir recurrencia y concentración en la pareja comprador-proveedor.

```
FLAG_WEIGHTS = {
    "RF-01": 15.0,
    "RF-02": 20.0,
    "RF-05": 25.0,
    "RF-06": 15.0,
```



```
}

if missing_supplier(contract):
    red_flags.append("RF-01")
    evidence["supplier_name"] = contract.supplier_name
elif contract.supplier_count_in_tender == 1:
    red_flags.append("RF-01")
    evidence["supplier_count_in_tender"] = contract.supplier_count_in_tender
if risky_procedure(contract):
    red_flags.append("RF-02")
    evidence["procedure"] = contract.procedure
if awarded_above_estimate(contract, deviation_threshold=deviation_threshold):
    red_flags.append("RF-05")
if temporal_anomaly(contract):
    red_flags.append("RF-06")

risk_score = min(sum(FLAG_WEIGHTS.get(flag, 0.0) for flag in red_flags), 100.0)
```

Este fragmento muestra por qué el score es explicable: cada incremento está ligado a una regla con código, peso y evidencia. La limitación es que no todas las red flags previstas inicialmente están implementadas en la versión evaluada; por tanto, RF-07 a RF-15 no deben presentarse como resultado evaluado.

Ruta: `scr/procurewatch/agent2/mvp_pipeline.py`. El nivel de riesgo se asigna mediante umbrales transparentes:

```
def _risk_level_from_score(score: float) -> str:
    if score >= 75:
        return "critico"
    if score >= 50:
        return "alto"
    if score >= 25:
        return "medio"
    return "bajo"
```

La ventaja de esta decisión es que un evaluador puede reproducir manualmente la

categoría de riesgo a partir del score. La desventaja es que los umbrales no proceden de entrenamiento supervisado contra fraude confirmado; son criterios de priorización para revisión humana.

8.6.4. Batch incremental y estado skipped

Rutas: `scr/procurewatch/batch.py` y `tests/test_batch.py`. La lógica de batch registra hashes y evita reejecutar Agent1 cuando las fuentes no han cambiado. El test de regresión reproduce el escenario semanal sin cambios:

```
with mock.patch("procurewatch.batch.run_agent1") as run_agent1_mock:
    result = run_batch(
        run_mode="weekly",
        force=False,
        boe_input=boe_input,
        open_tender_input=open_tender_input,
        batch_state_path=batch_state_path,
        batch_manifest_dir=workspace / "data/manifest/batches",
    )

self.assertFalse(run_agent1_mock.called)
self.assertEqual(result["status"], "skipped")
self.assertEqual(result["agent1_executed"], False)
```

Este fragmento es relevante porque valida una propiedad operativa del prototipo: el sistema no descarga ni procesa todo en cada ejecución si los hashes, tamaños y rutas de entrada no cambian. La limitación es que esta estrategia depende de que las fuentes mantengan metadatos comparables y de que el estado anterior de batch no se pierda.

8.6.5. Consulta de vecindario en Neo4j

El último fragmento muestra una consulta Cypher representativa del análisis de grafo. En la versión de cierre, Neo4j es opcional, pero el modelo de nodos y relaciones sí está preparado para cargar contratos y recuperar su entorno relacional.

```
MATCH (contract:Contract {contract_key_canon: $contract_key_canon})-[rel]-(neighbor)
RETURN contract, rel, neighbor
```

Esta consulta permite explicar un contrato desde su vecindario: comprador, adjudicatario, CPV, fuente y entidades relacionadas. Para la defensa del TFM, esta capacidad es más relevante que la mera visualización del grafo, porque conecta las métricas de red con la ficha explicativa de caso.

9. Resultados y evaluación

Los resultados cuantitativos presentados corresponden a los artefactos generados por el pipeline en la ejecución disponible del prototipo. No se presentan como cifras definitivas absolutas de todo el proyecto si existe una ejecución consolidada con datasets adicionales del equipo. La prioridad metodológica ha sido no inventar resultados y distinguir entre salidas generadas por el sistema, validaciones técnicas realizadas y cifras históricas procedentes de documentos anteriores.

Para reforzar esa delimitación, la versión final incorpora un benchmark reproducible del prototipo. El informe estándar del directorio `benchmark`, ejecutado sin validar el dashboard, evalúa 34 métricas distribuidas entre Agent1, Agent2, Agent3, Agent4 e integración. El resultado global fue `warning`: 31 métricas en `pass`, 1 en `warning`, 0 en `fail` y 2 como `not_applicable`. El reporte complementario se conserva en `benchmark_dashboard`. Con `--include-dashboard` mantiene 34 métricas y obtiene 32 en `pass`, 1 en `warning`, ninguna en `fail` y 1 no aplicable. La métrica `integrated.dashboard` cambia de `not_applicable` a `pass`; no se añade una métrica nueva. La advertencia de ambos modos corresponde a la ausencia de intersecciones exactas entre fuentes. La Tabla 9.1 resume qué partes quedan implementadas, qué partes quedan evaluadas y qué elementos se mantienen como extensión futura.

Tabla 9.1: Matriz de alcance evaluado en el benchmark final del TFM

Componente	Estado evaluado	Evidencia	Límite o extensión
Agent1	Implementado y evaluado.	Calidad de campos, duplicados y cobertura interfente.	Sin matching exacto transversal validado.
Agent2	Implementado y evaluado sobre muestra reproducible.	3.437 scores, evaluabilidad por regla y sensibilidad del 10 %.	Falta repetir sobre el canónico completo y contrastar con referencia externa.

Componente	Estado evaluado	Evidencia	Límite o extensión
Agent3	Implementado y evaluado.	Nodos, aristas, comunidades, modularidad y cobertura de features.	Validado en muestra reproducible y demo local; falta escala institucional.
Agent4	Implementado y evaluado.	Retrieval, citas, trazabilidad y prudencia del lenguaje.	Corpus local/sintético; RAGAS queda futuro.
Integración y dashboard	Demo integrada evaluada.	Flujo Agent2–Agent3–Agent4 y validaciones internas.	No equivale a despliegue productivo institucional.

9.1. Resultados de ingesta y normalización

La ejecución documentada del pipeline de Agent1 integra BOE, PLACE y OpenTender para el dominio CPV 71 y genera el dataset canónico consumido por los agentes posteriores. La Tabla 9.2 resume las magnitudes principales recogidas en los reportes de ejecución y artefactos generados por el sistema.

Tabla 9.2: Filas observadas en artefactos generados por Agent1

Artefacto generado	Filas	Interpretación
<code>agent1_run_report.json</code>	97.155	Líneas BOE leídas por el parser.
<code>contracts_boe.parquet</code>	96.801	Registros BOE parseados y normalizados desde el CSV longitudinal.
<code>contracts_boe_cpv71.parquet</code>	8.097	Subconjunto BOE con CPV 71.
<code>contracts_place_cpv71.parquet</code>	18.831	Registros PLACE CPV 71 tras parsing y deduplicación.
<code>contracts_opentender_2024_cpv71.parquet</code>	25.057	Registros OpenTender españoles CPV 71 disponibles en la ejecución.

Artefacto generado	Filas	Interpretación
agent2_contracts_ canonical.parquet	51.720	Universo canónico reportado para análisis posterior.

El informe `agent1_run_report.json` documenta que el parser BOE leyó 97.155 líneas de datos, parseó 96.801 registros y registró 354 errores de parseo, con una tasa de éxito del 99,64 %. Para BOE CPV 71 se obtuvieron 8.097 contratos. El parser PLACE produjo 18.831 registros CPV 71 y OpenTender aportó 25.057 registros CPV 71. En conjunto, la ejecución documentada genera 51.720 registros canónicos para análisis posterior.

La diferencia entre estas cifras y las 3.443 adjudicaciones CPV 71 citadas en la parte contextual de esta memoria se explica por la diferencia de unidad de análisis y por el estado de reconciliación. La cifra de 3.443 procede de la referencia contextual del dataset BOE y se usó para justificar la selección del dominio. La ejecución documentada de Agent1 trabaja con registros, expedientes y fuentes complementarias; por tanto, no debe forzarse una equivalencia directa entre ambas magnitudes.

9.2. Calidad del canónico y cobertura entre fuentes

La calidad de Agent1 se evalúa con dos tipos de evidencia: resultados cuantitativos del parser y reportes de cobertura del canónico. La tasa de parseo BOE del 99,64 % indica que la extracción estructurada es estable para la fuente principal. En cambio, la cobertura entre BOE, PLACE y OpenTender mediante `contract_key_canon` se mantiene como una limitación relevante: el criterio conservador de emparejamiento no fuerza equivalencias cuando los identificadores o campos auxiliares no permiten asegurar que dos registros representan el mismo contrato.

Este resultado es metodológicamente relevante. No significa que las fuentes sean inútiles, sino que el criterio conservador de emparejamiento no encontró coincidencias suficientemente seguras. La decisión de no forzar el matching evita falsos positivos de integración. En una plataforma de apoyo a auditoría, un cruce incorrecto entre fuentes puede ser más perjudicial que dejar constancia de una limitación.

El diagnóstico específico de cobertura entre fuentes generado para el benchmark final trabaja con un universo de 3.437 claves canónicas: 976 procedentes de BOE, 1.587 de PLACE y 874 de OpenTender. Las intersecciones exactas BOE–PLACE, BOE–OpenTender y

PLACE–OpenTender son 0. Por tanto, la cobertura observada debe interpretarse como **aditiva por fuente**, no como cobertura transversal. Este matiz es central para la robustez del sistema: Agent1 es reproducible y trazable por fuente, pero no permite todavía afirmar que una misma licitación haya sido contrastada entre plataformas distintas.

Esta ausencia de intersecciones útiles se incorpora como resultado metodológico del TFM. La integración de datos abiertos no queda resuelta por llevar distintas fuentes a un esquema común: requiere una política auditable de identificadores, normalización de organismos, comparación de expedientes, ventanas temporales, importes y validación manual de pares candidato/no-match. En una eventual versión institucional, las métricas transversales deberían mantenerse como exploratorias hasta disponer de esa capa de matching validada.

La Tabla 9.3 recoge las métricas de calidad principales utilizadas para interpretar la ingesta y la normalización.

Tabla 9.3: Métricas de calidad de Agent1 en la ejecución disponible

Métrica	Valor observado	Referencia	Lectura
Tasa de parseo BOE	99,64 %	Reporte Agent1	La fuente principal se transforma con baja pérdida de registros.
Errores de parseo BOE	354	Reporte Agent1	Los errores quedan registrados para trazabilidad y revisión.
Registros canónicos	51.720	Canónico Agent2	El sistema genera una frontera estable para scoring y agentes posteriores.
Cobertura de NIF	Limitada por fuente	Reportes de calidad	La ausencia de identificadores fiscales reduce evaluabilidad y agregación por adjudicatario.
Coherencia temporal	Parcialmente evaluable	Reportes de calidad	No todas las fuentes contienen fechas suficientes para aplicar reglas temporales.

Métrica	Valor observado	Referencia	Lectura
Intersecciones BOE/PLACE/OpenTender	0 exactas	Diagnóstico Agent1	La cobertura es aditiva por fuente; no se acredita triangulación transversal.

Estos resultados obligan a una lectura prudente. El pipeline está implementado, el esquema existe y los artefactos se generan, pero parte de la evaluabilidad depende de la disponibilidad de NIF, fechas y claves homogéneas en las fuentes públicas. Esta diferencia forma parte de la evaluación del prototipo y delimita el alcance de los resultados.

9.3. Resultados de red flags y scoring

Agent2 se ejecutó sobre los 3.437 contratos del canónico de muestra versionado y generó una puntuación para cada fila mediante RF-01 a RF-06. En el escenario base, 2.066 contratos activaron al menos una señal y se generaron 2.420 flags. Los 3.437 contratos fueron parcialmente evaluables; ninguno fue completamente evaluable porque RF-06 requería simultáneamente fecha de publicación y fecha de adjudicación válidas, combinación ausente en esta muestra. Ningún contrato quedó sin score: la evaluabilidad se registra por regla y los campos ausentes no se imputan.

La demo integrada permite observar el flujo completo sobre el caso PW-2024-0001. En ese caso, Agent2 asigna `risk_score=0.5`, nivel `medium` y activa dos señales: `risky_procedure` y `awarded_above_estimate`. La implementación de scoring de Agent2 trabaja conceptualmente con escala 0–100 y umbrales bajo/medio/alto/crítico; la demo integrada expresa el resultado en escala normalizada 0–1 para facilitar la visualización conjunta con grafo y ficha documental.

La distribución del escenario base fue:

Tabla 9.4: Distribución de riesgo de Agent2 sobre 3.437 contratos

Nivel de riesgo	Contratos	Interpretación
Sin flags	1.371	Score 0; ninguna regla activa en los datos disponibles.
Bajo positivo	1.500	Score mayor que 0 y menor que 25.

Nivel de riesgo	Contratos	Interpretación
Medio	553	Score entre 25 y 49.
Alto	13	Score entre 50 y 74.
Crítico	0	No observado en la muestra evaluada.

El score medio fue 13,26 sobre 100, con mediana 15, primer cuartil 0, tercer cuartil 20 y máximo 65. La Tabla 9.5 muestra que la principal limitación no es la ejecución del motor, sino la disponibilidad desigual de los campos requeridos por cada regla.

Tabla 9.5: Evaluabilidad por regla de Agent2 en el escenario base

Regla	Evaluables	No evaluables	Cobertura	Limitación principal
RF-01	3.437	0	100,00 %	La ausencia de adjudicatario forma parte de la propia señal.
RF-02	3.425	12	99,65 %	Procedimiento ausente.
RF-03	2.333	1.104	67,88 %	Adjudicatario no identificado.
RF-04	1.445	1.992	42,04 %	Pareja o importe insuficiente para calcular concentración.
RF-05	1.796	1.641	52,25 %	Falta importe adjudicado o estimado positivo.
RF-06	0	3.437	0,00 %	No existen ambas fechas válidas en una misma fila.

La Tabla 9.6 resume las reglas RF-01 a RF-06 documentadas e implementadas en la versión evaluada de Agent2. Las reglas no deben interpretarse como prueba directa de irregularidad, sino como indicadores de priorización.

Tabla 9.6: Red flags implementadas en Agent2

Código	Indicador	Lógica analítica	Tipo
RF-01	Baja concurrencia o identificación insuficiente	Activa riesgo cuando existe una sola oferta, falta el adjudicatario o la información disponible apunta a competencia reducida.	Competencia

Código	Indicador	Lógica analítica	Tipo
RF-02	Procedimiento de riesgo	Penaliza procedimientos sensibles, excepcionales o menos competitivos según la normalización disponible.	Procedimiento
RF-03	Recurrencia comprador-proveedor	Detecta adjudicaciones repetidas entre el mismo comprador y adjudicatario.	Recurrencia
RF-04	Concentración de importe comprador-proveedor	Evalúa si una pareja comprador-proveedor concentra una proporción relevante del importe adjudicado.	Concentración
RF-05	Desviación de importe	Analiza adjudicaciones por encima del importe estimado o relaciones estimado/adjudicado atípicas.	Importe
RF-06	Patrón temporal anómalo	Identifica intervalos anómalos entre publicación, adjudicación y formalización cuando existen fechas suficientes.	Temporal

La frecuencia observada en el escenario base fue: RF-01, 1.104 activaciones; RF-02, 12; RF-03, 730; RF-04, 34; RF-05, 540; y RF-06, 0. La ausencia de activaciones de RF-06 no significa que la muestra carezca de anomalías temporales, sino que la regla no pudo calcularse sin las dos fechas requeridas. Los campos ausentes más relevantes fueron 1.104 adjudicatarios, 12 procedimientos, 1.511 importes adjudicados, 2.461 fechas de publicación no válidas y 3.067 fechas de adjudicación no válidas.

Para comprobar la estabilidad del motor se repitió la ejecución multiplicando los umbrales numéricos de recurrencia, concentración, desviación y ventana temporal por 0,9 y 1,1. La Tabla 9.7 compara ambos escenarios con el escenario base. RF-01 y RF-02 se mantienen constantes porque no dependen de esos umbrales. En RF-03 y RF-06 debe considerarse que los conteos y los días son discretos, por lo que una variación decimal puede trasladarse al siguiente entero efectivo.

Tabla 9.7: Sensibilidad de Agent2 ante variación de umbrales

Métrica	-10 %	Base	+10 %
Flags totales	2.427	2.420	2.224
Contratos sin flags	1.368	1.371	1.516
Score medio	13,31	13,26	12,12
Scores sin cambios respecto a base	99,80 %	100,00 %	95,11 %
Nivel de riesgo sin cambios	99,80 %	100,00 %	99,04 %
Conjunto de flags sin cambios	99,80 %	100,00 %	95,11 %

Estos resultados documentan una estabilidad alta ante la perturbación definida, sin necesidad de incorporar Isolation Forest. La comparación mide robustez interna de reglas deterministas, no precisión frente a fraude confirmado. La demo integrada, por su parte, conserva las señales `risky_procedure` y `awarded_above_estimate` para mostrar el flujo completo del caso PW-2024-0001.

9.4. Evaluación cualitativa de diez fichas

Para comprobar si el sistema prioriza y explica casos de forma coherente se generaron diez fichas individuales a partir del escenario base de Agent2. La selección aplica una política reproducible: cinco contratos con el score máximo observado, resolviendo empates mediante diversidad comprador-proveedor; tres contratos de riesgo medio, priorizando diversidad entre PLACE, BOE y OpenTender; y dos controles con score cero, sin flags y con la mayor evaluabilidad disponible. Los controles no representan contratos “sin riesgo”, sino referencias comparativas en las que no se activaron señales con los datos evaluables.

Agent3 se ejecutó sobre los 3.437 contratos antes de construir las fichas. El grafo resultante contiene 7.333 nodos y 16.303 aristas, publica 30 comunidades y 3.437 filas de features relacionales. La partición Louvain obtiene una modularidad $Q = 0,6051129926$, superior al objetivo metodológico $Q > 0,30$. Las diez fichas combinan estas relaciones con los scores y evidencias congelados del escenario base; Agent3 no recalcula ni modifica el riesgo. La Tabla 9.8 resume la selección observada.

Tabla 9.8: Selección de diez fichas cualitativas

Caso	Grupo	Fuente	Score	Reglas activadas
CS-01	Score máximo	PLACE	65	RF-03, RF-04 y RF-05
CS-02	Score máximo	PLACE	65	RF-03, RF-04 y RF-05
CS-03	Score máximo	PLACE	65	RF-03, RF-04 y RF-05
CS-04	Score máximo	PLACE	65	RF-03, RF-04 y RF-05
CS-05	Score máximo	PLACE	65	RF-03, RF-04 y RF-05
CS-06	Riesgo medio	PLACE	45	RF-03 y RF-05
CS-07	Riesgo medio	BOE	45	RF-02 y RF-05
CS-08	Riesgo medio	OpenTender	40	RF-03 y RF-04
CS-09	Control	PLACE	0	Ninguna
CS-10	Control	BOE	0	Ninguna

Cada ficha conserva identificador canónico, fuente y registro de origen, comprador, adjudicatario, procedimiento, importes, reglas activadas, evidencia textual de cada flag, reglas no evaluables, relaciones de Agent3, disponibilidad documental y advertencias. La Tabla 9.9 presenta los datos cuantitativos utilizados en la lectura. Los códigos numéricos de procedimiento se mantienen tal como aparecen en PLACE u OpenTender; su interpretación requiere consultar el catálogo de la fuente.

Tabla 9.9: Importes, procedimiento y contexto relacional de las diez fichas

Caso	Estimado (EUR)	Adjudicado (EUR)	Procedimiento	Recurr. Agent3
CS-01	86.011.743,00	104.074.209,00	Cód. PLACE 1	1
CS-02	1.335.452,00	1.615.897,00	Cód. PLACE 7	2
CS-03	60.480,00	731.808,00	Cód. PLACE 7	3
CS-04	44.544,00	449.152,00	Cód. PLACE 9	2
CS-05	1.697.623,00	2.054.124,00	Cód. PLACE 7	2
CS-06	5.208,00	557.256,00	Cód. PLACE 7	4
CS-07	53.000,00	63.767,00	Negociado sin pu- blicitad	1
CS-08	644,22	644,22	Cód. OpenTender 1	2
CS-09	826.446,00	10.000,00	Cód. PLACE 9	1

Caso	Estimado (EUR)	Adjudicado (EUR)	Procedimiento	Recurr. Agent3
CS-10	495.867,77	432.817,00	Abierto	1

En conjunto se observaron 21 activaciones de reglas y las 21 disponen de un registro de evidencia trazable. Las relaciones están disponibles en las diez fichas, la composición 5/3/2 se cumple sin duplicados y no se genera ninguna afirmación de fraude. El corpus documental versionado no contiene documentos asociados a estos contratos reales, por lo que las diez fichas registran cero evidencias documentales y una advertencia explícita; no se descargaron documentos ni se crearon evidencias sintéticas para ocultar esa limitación.

Las fichas también advierten que los importes extremos deben comprobarse en la fuente original, por posibles diferencias de lote, unidad o agregación. En las señales relacionales se mantiene otra frontera metodológica: Agent2 calcula RF-03 y RF-04 con nombres normalizados y cuota de importe, mientras Agent3 usa identificadores del grafo y cuota de contratos. Por ello sus valores son contexto complementario y no se fuerza que coincidan. El informe `case_studies_report.md` y las diez fichas JSON/Markdown conservan el detalle completo, incluidas 56 advertencias. La validación práctica es positiva para priorización y explicación; no equivale a precisión frente a fraude confirmado ni a revisión experta.

9.5. Resultados de grafo, RAG y dashboard

La demo integrada del sistema muestra el flujo completo entre scoring, grafo, recuperación documental y dashboard. Para el caso PW-2024-0001, el sistema asigna `risk_score=0.5`, activa dos señales de riesgo —`risky_procedure` y `awarded_above_estimate`—, genera un grafo de 11 nodos y 13 relaciones, recupera 2 evidencias documentales con 2 citas trazables y permite visualizar la ficha de caso en el dashboard Streamlit.

Agent3 transforma tres contratos demostrativos en un grafo compuesto por compradores, proveedores, contratos, CPV y fuentes. En esa ejecución se generan 11 nodos, 13 aristas, 2 comunidades, 1 componente conexa y 3 filas de features reutilizables por Agent2 y Agent4. La mayor componente tiene 11 nodos. La descomposición por tipos es: 2 compradores, 3 contratos, 3 nodos CPV, 1 fuente y 2 proveedores. Las relaciones generadas son 3 `AWARDED_TO`, 3 `FROM_SOURCE`, 4 `HAS_CPV` y 3 `PUBLISHED`. No se registran contratos sin proveedor ni sin CPV en la demo integrada. La modularidad de su partición Louvain es $Q = 0,3165680473$, también por encima del umbral $Q > 0,30$. Este resultado cuantifica

separación comunitaria en el grafo simple no ponderado; no demuestra fraude ni validez jurídica de las comunidades.

Agent4 recupera evidencia documental trazable asociada al contrato. Para PW-2024-0001, recupera dos fragmentos documentales y genera dos citas con `document_id`, `chunk_id` y `contract_key_canon`. La generación se realiza en modo `deterministic_fallback`; en esta demo Qdrant y Ollama no son necesarios. Las evidencias incluyen un fragmento HTML del pliego con score 0,6 y un fragmento TXT de adjudicación con score 0,4. La ficha conserva la frontera de decisión: el sistema no declara fraude, sino que apoya la revisión humana.

La evaluación local de Agent4 sobre corpus sintético recoge 3 casos evaluados, 2 con evidencia, `expectation_accuracy=1.0`, `precision@k` media de 1,0, `expected_document_recall=1.0`, trazabilidad media de citas de 1,0, consistencia media de contrato de 1,0 y validación práctica de 1,0. RAGAS queda registrado como `not_run`. Estos resultados validan la trazabilidad y recuperación en la versión evaluada del sistema, no el rendimiento jurídico sobre un corpus real amplio.

Tabla 9.10: Evaluación práctica de Agent4 para fichas trazables

Dimensión evaluada	Métrica usada	Resultado	Lectura para el TFM
Fidelidad documental	Precisión@k media	100 %	La evidencia recuperada coincide con documentos esperados.
Cobertura documental	Recall documental	100 %	Los documentos esperados aparecen en el contexto recuperado.
Trazabilidad de citas	Cita trazable	100 %	Las citas conservan <code>document_id</code> , <code>chunk_id</code> y contrato.
Consistencia del contrato	Contrato consistente	100 %	La evidencia pertenece al contrato consultado.
Prudencia explicativa	Sin fraude no soportado	100 %	La ficha no declara fraude sin soporte documental.
Validación práctica	Ficha trazable	100 %	Los casos cumplen las condiciones mínimas de ficha trazable.

La evaluación anterior es rigurosa como validación práctica de una capa documental local: mide fidelidad, cobertura, trazabilidad, consistencia y prudencia del lenguaje. Su límite es igualmente explícito: no valida corrección jurídica, no demuestra generalización sobre documentos reales heterogéneos y no evalúa aún RAGAS con volumen representativo. En consecuencia, el LLM local se presenta como capa explicativa controlada y no como motor decisor.

El dashboard integrado está implementado en `frontend/agent3_demo.py`. La validación técnica recoge estado `ready`, 3 contratos cargados, 11 nodos, 13 aristas, 2 comunidades, 14 métricas visibles, 7 pestañas, 2 evidencias, 2 citas y 0 excepciones en renderizado headless. La interfaz está organizada para revisión humana: el usuario no recibe una decisión automática, sino una ruta de exploración desde el ranking de riesgo hasta la evidencia concreta.

9.6. Evaluación técnica y tests

La versión evaluada del sistema incluye una batería amplia de tests. La validación técnica realizada indica:

- `PYTHONPATH=scr python -m procurewatch.cli doctor`: resultado OK, con PostgreSQL, Neo4j, Qdrant y Ollama como servicios opcionales;
- `PYTHONPATH=scr python -m pytest tests`: 122 pruebas superadas y 1 omitida en el entorno mínimo sin SQLAlchemy; la prueba omitida corresponde a persistencia PostgreSQL opcional;
- la misma suite con SQLAlchemy disponible: 123 pruebas superadas y 0 omitidas;
- `PYTHONPATH=scr python -m ruff check api scr tests frontend`: sin errores;
- `PYTHONPATH=scr python -m procurewatch.cli run-benchmark`: 34 métricas, con 31 en `pass`, 1 en `warning`, 0 en `fail` y 2 como `not_applicable`;
- el mismo benchmark con `--include-dashboard`: 34 métricas, con 32 en `pass`, 1 en `warning`, 0 en `fail` y 1 como `not_applicable`;
- demo integrada y dashboard Streamlit validados en modo headless con 0 excepciones.

La Tabla 9.11 resume la cobertura funcional de los tests existentes.

Tabla 9.11: Resumen de pruebas automatizadas existentes

Bloque de test	Cobertura principal
Agent1	Pipeline, canónico, calidad, esquema analítico, cobertura, PostgreSQL y catálogo de compradores.
Fuentes	Parser BOE, unidades de análisis BOE, OpenTender, PLACE y normalización de entradas.
Agent2	Reglas, scoring, pipeline estable, RF-01 a RF-06 y reportes de salida.
Agent3	Construcción de grafo, nodos, aristas, métricas, comunidades, modularidad, features y carga Neo4j.
Agent4	Carga documental, chunks, retrieval, citas, contexto de caso, fallback determinista e integraciones opcionales.
Fichas cualitativas	Selección 5/3/2, empates, controles, evidencia por flag, relaciones y ausencia documental.
Benchmark	Evaluación agregada por agente, estados <code>pass/warning/fail</code> y matriz de alcance del TFM.
CLI y batch	Comandos principales, diagnóstico, ejecución del prototipo y skip semanal por ausencia de cambios.

La evaluación no equivale a una validación supervisada contra fraude real, porque el proyecto no dispone de etiquetas fiables de fraude confirmado para el conjunto analizado. Por ello, las métricas de Agent2 deben describirse como evaluación exploratoria, de estabilidad, de consistencia de reglas y de priorización proxy. Este enfoque es metodológicamente correcto: resulta preferible declarar el alcance real que presentar una precisión de fraude no verificable. El benchmark final refuerza esta misma idea: es riguroso como evaluación técnica, objetiva y reproducible del prototipo, pero no debe confundirse con una validación estadística de detección de fraude.

9.7. Matriz de evidencias de evaluación

La evaluación del prototipo se apoya en evidencias técnicas heterogéneas. No todas las evidencias tienen el mismo alcance: algunas prueban que el código se ejecuta, otras prueban que una salida existe, otras cuantifican calidad de datos y otras solo documentan una limitación. Separar estos niveles evita presentar como resultado estadístico lo que en

realidad es una prueba funcional, o presentar como validación de fraude lo que es una evaluación proxy.

La Tabla 9.12 resume esta distinción. Su función no es añadir cifras nuevas, sino ordenar qué tipo de afirmación puede defenderse con cada artefacto.

Tabla 9.12: Matriz de evidencias utilizadas en la evaluación del prototipo

Evidencia	Qué permite afirmar	Alcance	Límite
Reports de Agent1	Fuentes procesadas, filas, errores, calidad y cobertura.	Cuantitativo sobre la ejecución disponible.	No resuelve por sí solo la falta de intersecciones entre fuentes.
Parquet canónico	Existencia de una frontera estable Agent1–Agent2.	Operativo y reproducible.	Depende de la calidad de los campos de origen.
Reports de Agent2	Flags, scores y distribución de riesgo en la muestra evaluada.	Cuantitativo/proxy.	No equivale a validación supervisada contra fraude.
Reports de Agent3	Grafo, partición Louvain y modularidad $Q = 0,6051129926$ en la muestra.	Cuantitativo sobre estructura relacional.	Q no valida fraude ni significado jurídico de las comunidades.
Diez fichas cualitativas	Priorización 5/3/2, evidencia completa de 21 flags y contexto relacional.	Cualitativo/proxy sobre la muestra.	Sin etiquetas, revisión experta ni documentos asociados en el corpus actual.
Tests automatizados	Correctitud funcional de parsers, agentes, CLI y batch.	Validación técnica.	No sustituye validación por expertos.
Benchmark ProcureWatch	Evaluación agregada de Agent1, Agent2, Agent3, Agent4 e integración.	Técnico, objetivo y reproducible.	Mantiene warning por matching interfuerente; no valida fraude supervisado.
Dashboard headless	Disponibilidad de la interfaz y lectura de KPIs.	Prueba de integración.	No mide usabilidad con usuarios finales.

Evidencia	Qué permite afirmar	Alcance	Límite
Demo integrada Agent2-Agent3-Agent4	Integración de scoring, grafo, evidencias y dashboard sobre caso demostrativo.	Evidencia funcional de extremo a extremo.	No es una evaluación masiva del RAG.
Manifests y hashes	Trazabilidad de entradas, salidas y ejecuciones batch.	Reproducibilidad.	Requiere conservar el estado de ejecución y rutas asociadas.

Esta matriz permite formular las conclusiones con el nivel de fuerza adecuado. Por ejemplo, los reports de Agent1 permiten afirmar que se generó un canónico de 51.720 filas en la ejecución disponible, pero no permiten afirmar que todas las fuentes públicas se crucen correctamente entre sí. Del mismo modo, Agent2 permite afirmar que existen reglas reproducibles y una distribución de riesgo calculada, pero no que el sistema detecte fraude confirmado. Esta distinción es central para que la memoria sea defendible.

9.8. Lectura de evaluación por agente

La evaluación de Agent1 se interpreta principalmente como evaluación de datos. Sus resultados son positivos en tres aspectos: el parser BOE presenta una tasa de parseo alta, las fuentes se normalizan en artefactos Parquet y el sistema genera reports de calidad. El diagnóstico de benchmark aporta además una lectura específica de cobertura entre fuentes: 3.437 claves canónicas, cobertura por BOE, PLACE y OpenTender, y 0 intersecciones exactas entre pares de fuentes. En términos de TFM, Agent1 queda operativo, pero sus resultados obligan a no sobredimensionar el alcance de los cruces entre fuentes.

La evaluación de Agent2 se interpreta como evaluación de reglas y priorización. La muestra de scoring documentada confirma que el motor genera una puntuación por contrato y registra las señales activadas; la demo integrada muestra además cómo esas señales se combinan con grafo, evidencias y dashboard para construir una ficha revisable. Esta salida permite priorizar la revisión, pero no debe transformarse en un juicio jurídico. La principal evidencia de calidad en Agent2 no es una precisión supervisada, sino la trazabilidad: cada score puede descomponerse en reglas activadas, pesos y datos que permitieron o impidieron la evaluación.

La evaluación de Agent3 se interpreta como evaluación relacional. El grafo no sustituye

al canónico, sino que lo proyecta en una estructura de relaciones. Por ello, sus métricas deben leerse como contexto: grado, comunidad o recurrencia ayudan a explicar patrones, pero no son evidencia autónoma de irregularidad. Esta precaución es importante porque, en contratación pública, la centralidad puede deberse a especialización técnica, tamaño empresarial o presencia territorial legítima. La modularidad observada, $Q = 0,6051129926$ en la muestra y $Q = 0,3165680473$ en la demo, supera el objetivo $Q > 0,30$ y permite afirmar que Louvain encuentra una partición con estructura comunitaria cuantificable en ambos grafos evaluados. No permite atribuir una interpretación irregular a esas comunidades sin evidencia adicional.

La evaluación de Agent4 se interpreta como evaluación documental y de explicabilidad. El resultado esperado no es que el LLM descubra fraude, sino que ayude a transformar una combinación de datos tabulares, red flags y relaciones en una ficha revisable con evidencias y advertencias. La calidad de Agent4 depende de tres condiciones: que el corpus contenga fragmentos relevantes, que la recuperación devuelva contexto suficiente y que la generación no añada afirmaciones sin soporte. La evaluación final mide estas condiciones mediante fidelidad documental, cobertura, trazabilidad de citas, consistencia de contrato, ausencia de declaraciones de fraude no soportadas y validación práctica de la ficha. Cuando estas condiciones no se cumplen, el sistema debe mostrar la limitación.

El dashboard se evalúa como capa de consumo. Su aportación principal es ordenar la revisión: primero muestra una visión agregada, después permite seleccionar contratos priorizados y finalmente conecta cada caso con flags, relaciones, evidencias y trazabilidad. La validación headless prueba que la interfaz carga, pero no sustituye una evaluación de usabilidad con auditores. Por tanto, la memoria presenta el dashboard como una interfaz funcional de demostración, no como un producto final validado con usuarios.

9.9. Interpretación de la evaluación proxy

La ausencia de *ground truth* obliga a usar una evaluación proxy. Esta decisión no reduce el valor académico del trabajo si se declara correctamente. En un problema como la contratación pública, disponer de etiquetas fiables de fraude confirmado exige resoluciones firmes, auditorías cerradas o investigaciones validadas. Tratar todos los contratos no investigados como negativos introduciría un sesgo grave: muchos contratos no tienen etiqueta simplemente porque nunca fueron auditados con suficiente detalle.

Por ese motivo, el prototipo evalúa propiedades observables del sistema: completitud de datos, capacidad de generar artefactos, consistencia de reglas, estabilidad de ejecución, trazabilidad de manifests, existencia de salidas por agente y capacidad de explicar casos. Estas propiedades no demuestran fraude, pero sí demuestran que el sistema puede utilizarse como herramienta de priorización. La afirmación defendible es, por tanto, más limitada y más rigurosa: ProcureWatch reduce el espacio de revisión manual mediante señales trazables, pero la validación sustantiva de cada caso corresponde a una persona experta.

Esta lectura también explica por qué Isolation Forest, Positive-Unlabeled Learning, SHAP o RAGAS se tratan con cautela en la versión de cierre. Son técnicas coherentes con la evolución del sistema, pero no deben presentarse como resultados concluyentes si no existe un artefacto evaluado que respalde esa afirmación. En la memoria se conservan como marco metodológico, extensión o soporte de comparación, no como validación cerrada del sistema.

10. Limitaciones, Adaptación del Alcance y Trabajo Futuro

El desarrollo de ProcureWatch Analytics confirma la viabilidad de un prototipo multiagente orientado a priorización de riesgo, pero también muestra limitaciones propias de trabajar con datos públicos heterogéneos, ausencia de etiquetas de fraude confirmadas y servicios locales opcionales. Este capítulo documenta esas limitaciones como parte del resultado del TFM. No se tratan como fallos ocultos, sino como restricciones verificables que delimitan el uso correcto del sistema.

10.1. Adaptación del alcance y grado de cumplimiento de los objetivos

El planteamiento inicial del TFM definía una plataforma ambiciosa con integración de fuentes, red flags, machine learning, grafos, NLP, RAG, dashboard y servicios de datos. La implementación final conserva esa arquitectura conceptual, pero la concreta como un prototipo académico reproducible. La diferencia principal es que el objetivo del cierre no ha sido construir una plataforma productiva completa, sino demostrar un flujo defendible de extremo a extremo: ingesta, normalización, scoring, grafo, evidencia documental, dashboard y validación técnica.

La Tabla 10.1 resume la relación entre alcance previsto e implementación final.

Tabla 10.1: Correspondencia entre alcance previsto e implementación final

Elemento previsto	Implementación final	Lectura metodológica
Ingesta BOE, PLACE y OpenTender	Parsers y normalizadores implementados; canónico generado.	Cumplido como flujo reproducible, con limitación de matching entre fuentes.
Esquema OCDS adaptado	Campos mínimos implementados en <code>analytical.schema.py</code> .	Cumplido y validable por tests.

Elemento previsto	Implementación final	Lectura metodológica
PostgreSQL	Persistencia analítica de Agent1 implementada.	Parcial para el conjunto multi-agente completo.
Red flags y scoring	RF-01 a RF-06 implementadas en la versión evaluada; scoring por contrato.	Cumplido para el prototipo; RF adicionales y scoring agregado por adjudicatario quedan como extensiones.
Isolation Forest y PU Learning	Enfoque contemplado como comparativa deseable, sin resultado final verificable en los artefactos disponibles de la versión de cierre.	No se presenta como validación cerrada si no hay artefacto verificable.
Grafo y comunidades	Agent3 implementa grafo, métricas, Louvain, modularidad y features; Neo4j opcional.	Cumplido como análisis relacional reproducible; $Q > 0,30$ en muestra y demo.
Neo4j	Carga opcional implementada.	Servicio no obligatorio para la demo local.
RAG documental	Agent4 implementa corpus, chunks, retrieval, contexto de caso y fallback.	Cumplido como componente documental del prototipo, no como evaluación masiva.
Qdrant y Ollama	Soportados como servicios opcionales.	Diseño flexible: la demo no queda bloqueada por servicios externos.
Dashboard	Streamlit integrado y documentado.	Cumplido para exploración local.
Validación de fraude	No hay etiquetas reales de fraude confirmado.	La evaluación es proxy/exploratoria y orientada a revisión humana.

Esta adaptación es coherente con la finalidad académica del TFM. El trabajo no pretende certificar irregularidades administrativas, sino demostrar que es posible construir una cadena analítica trazable que reduzca el espacio de revisión manual y explique por qué determinados contratos deben priorizarse.

10.2. Impacto de las limitaciones por componente

Las limitaciones no afectan por igual a todos los agentes. Algunas reducen cobertura de datos, otras afectan a la interpretación del score y otras condicionan la profundidad de la demostración documental. Identificar el impacto por componente permite defender el sistema con precisión: no todo queda invalidado por una limitación, pero tampoco debe ocultarse qué conclusiones quedan restringidas.

La Tabla 10.2 resume este impacto.

Tabla 10.2: Impacto de las limitaciones sobre los componentes del prototipo

Componente	Limitación principal	Impacto	Mitigación aplicada
Agent1	Heterogeneidad BOE/PLACE/OpenTender y falta de intersecciones por clave canónica.	Limita análisis cruzado entre fuentes.	Matching conservador, reports de cobertura y no forzar equivalencias.
Agent1	NIF y fechas incompletas.	Reduce evaluabilidad de algunas métricas de calidad.	Marcar no evaluable y separar validez cuando el dato está presente.
Agent2	Ausencia de etiquetas de fraude confirmado.	Impide medir precisión supervisada real.	Evaluación proxy, reglas explicables y revisión humana.
Agent2	Catálogo RF reducido en la versión evaluada.	No cubre todos los patrones planteados inicialmente.	Priorizar RF-01 a RF-06 verificables y documentar extensiones.
Agent3	Muestra relacional menor que el canónico completo.	No permite afirmar escalado institucional ni significado sustantivo de las comunidades.	Features reproducibles, modularidad cuantificada y Neo4j opcional para exploración.
Agent4	Corpus documental no masivo en la evidencia disponible.	Limita evaluación RAG cuantitativa.	Métricas de fidelidad, cobertura, citas, consistencia y prudencia explicativa.

Componente	Limitación principal	Impacto	Mitigación aplicada
Dashboard	Validación headless, no estudio formal de usuarios.	No mide usabilidad institucional completa.	Diseño orientado a revisión humana y trazabilidad visible.
Batch	Dependencia de manifests y estado anterior.	Si se pierde estado, se reduce capacidad incremental.	Hashes, modo force y criterio metodológico de snapshot final.

Esta matriz muestra que la limitación más fuerte se concentra en la validación sustantiva del fraude, no en la ejecución del pipeline. El sistema puede ingerir, normalizar, puntuar, construir artefactos y presentar casos; lo que no puede afirmar, sin una capa externa de validación, es que un contrato sea fraudulento. Esta diferencia es esencial para interpretar correctamente el alcance del TFM.

También permite distinguir limitaciones de **datos**, **modelo** y **operación**. Las limitaciones de datos afectan a completitud, NIF, fechas y matching. Las limitaciones de modelo afectan a la ausencia de etiquetas y a la imposibilidad de calibrar una precisión supervisada. Las limitaciones operativas afectan a servicios opcionales, demos y disponibilidad de snapshots. En todos los casos, la respuesta metodológica elegida fue hacer visible la restricción, no esconderla en el cálculo.

10.3. Limitaciones de datos

La primera limitación procede de la heterogeneidad de fuentes. BOE, PLACE y OpenTender no siempre publican los mismos identificadores, ni con la misma granularidad. Un contrato puede aparecer como expediente, lote, anuncio, adjudicación o línea de adjudicación. Por ello, la clave `contract_key_canon` se construye con una política conservadora basada en identificadores disponibles, organismo, fechas y fuente. Esta política evita emparejamientos dudosos, pero reduce la cobertura de intersecciones.

En la ejecución disponible del prototipo, las intersecciones BOE/PLACE/OpenTender fueron cero. El sistema puede seguir funcionando con datos reales intrafuente, porque Agent2, Agent3 y Agent4 no dependen obligatoriamente de que una misma licitación aparezca en varias fuentes. Sin embargo, los análisis que requieren contraste cruzado fuerte entre fuentes quedan limitados. En la memoria, esta situación se interpreta como limitación

de integración y no como prueba de ausencia de coincidencias en la realidad.

La segunda limitación afecta a la completitud de campos críticos. El informe de calidad generado muestra que la completitud OCDS crítica no alcanza el objetivo inicial de 0,90 en el canónico completo. La cobertura de NIF del adjudicatario también es insuficiente cuando se calcula sobre todo el dataset, aunque la validez del identificador es muy alta cuando el dato está presente. Esto confirma que el problema principal no es la normalización del NIF disponible, sino la ausencia del campo en parte de las fuentes.

La tercera limitación afecta a la coherencia temporal. Muchos registros carecen de todas las fechas necesarias para calcular con seguridad los días entre publicación y adjudicación. Por tanto, los indicadores temporales solo son aplicables a la fracción evaluable. Cuando el dato no existe, el sistema marca el caso como no evaluable en lugar de imputar artificialmente una fecha.

10.4. Limitaciones de modelado y evaluación

La principal limitación de modelado es la ausencia de etiquetas de fraude confirmado. La literatura utiliza en ocasiones datasets con casos judiciales, sanciones o auditorías cerradas, pero el proyecto no dispone de una fuente equivalente para el conjunto BOE/PLACE/OpenTender analizado. Por este motivo, no es metodológicamente correcto hablar de precisión supervisada en detección de fraude.

El score de Agent2 debe interpretarse como una priorización de riesgo basada en señales observables. Las red flags capturan patrones compatibles con riesgo: baja concurrencia, recurrencia, procedimiento menos competitivo, concentración, desviaciones de importe o falta de evaluabilidad. Ninguna de estas señales prueba por sí sola una irregularidad. Su valor aparece al combinarse con contexto, trazabilidad y revisión humana.

La evaluación disponible se apoya en cuatro tipos de evidencia:

1. métricas de calidad de datos y completitud;
2. consistencia interna de reglas y salidas;
3. pruebas automatizadas sobre parsers, agentes y CLI;
4. demo integrada documentada con caso explicable.

Esta evaluación es suficiente para defender un prototipo académico, pero no sustituye una validación externa por expertos ni una auditoría sobre casos reales confirmados.

10.5. Limitaciones operativas

El fichero `docker-compose.yml` define servicios para PostgreSQL, Neo4j y Qdrant. Ollama no forma parte de Compose: puede ejecutarse como servicio local o externo opcional. La demo final no exige que estos servicios estén activos. Esta decisión reduce el riesgo de dependencia operativa en la defensa y permite ejecutar tests con fallback local. La contrapartida es que algunas capacidades se demuestran como soporte opcional o PoC, no como despliegue productivo permanente.

También existe una limitación de versión de artefactos. La demo integrada y el benchmark final están disponibles como evidencias reproducibles, pero proceden de una ejecución local concreta. Por ello, la memoria diferencia entre resultados históricos de ejecuciones amplias y artefactos generados para la validación final. Si se incorpora un snapshot posterior consolidado, las cifras cuantitativas deberán actualizarse con ese snapshot, en lugar de mezclar ejecuciones distintas.

10.6. Trabajo futuro

El trabajo futuro se organiza en cinco líneas. La primera es mejorar el matching entre fuentes. Para ello sería necesario ampliar `contract_key_canon` con reglas jerárquicas, similitud de texto, normalización avanzada de organismos, importes aproximados, ventanas temporales y validación manual de una muestra. El objetivo no debe ser maximizar coincidencias, sino aumentar recall sin degradar precisión.

La segunda línea consiste en ampliar el catálogo de red flags. El alcance inicial planteaba un conjunto mayor de indicadores; el prototipo implementa las señales principales y deja margen para RF adicionales sobre fraccionamiento, cambios contractuales, patrones de lotes, recurrencia interanual, adjudicaciones cercanas a umbrales, relaciones societarias y documentación técnica dirigida.

La tercera línea es incorporar validación externa. La forma más sólida sería construir un pequeño conjunto etiquetado por expertos o cruzar el sistema con resoluciones administrativas, informes de fiscalización, expedientes sancionadores o casos judiciales cerrados. Sin esa capa, el sistema debe seguir describiéndose como priorizador de riesgo y no como detector supervisado de fraude.

La cuarta línea es robustecer la capa documental. Agent4 ya demuestra el flujo de cor-

pus, fragmentación, recuperación y ficha explicativa, pero sería necesario ampliar el corpus real, medir RAGAS con suficiente volumen, incorporar controles de citas más estrictos y separar mejor hechos recuperados, inferencias y advertencias.

La quinta línea es evolucionar de prototipo local a plataforma desplegable. Esto implica API productiva, autenticación, control de roles, auditoría de acciones de usuario, persistencia completa de outputs multiagente, monitorización, CI/CD y políticas de retención de datos. Estos elementos quedan fuera del prototipo académico, pero son necesarios para un uso institucional.

11. Conclusiones

ProcureWatch Analytics quedó definido como una plataforma multiagente reproducible para integrar, normalizar y analizar datos de contratación pública española y europea con un enfoque explicable y trazable. El desarrollo materializó el alcance principal del TFM en una cadena coherente de datos, reglas, persistencia y evidencia, sin presentar el score como una afirmación de fraude ni como una decisión automática.

La primera conclusión es que el modelo de datos canónico funcionó como eje de integración. Agent 1 consolidó BOE, PLACE y OpenTender en un esquema OCDS adaptado, con una política documentada para `contract_key_canon`, perfiles de calidad y trazabilidad por hash. La cobertura entre fuentes se registró de forma explícita, y las limitaciones de cruce se mantuvieron visibles en lugar de forzar coincidencias artificiales.

La segunda conclusión es que Agent 2 quedó operativo como motor determinista de red flags y scoring explicable. El sistema calculó señales y scores por contrato y almacenó la evidencia necesaria para justificar cada activación. Agent1 aporta una tabla analítica de adjudicatarios, pero no se presenta un score final agregado por proveedor. La evaluación se presentó como contraste proxy cuando no existieron etiquetas confirmadas de fraude, lo que preservó la coherencia metodológica del TFM y evitó afirmar una validación supervisada que no estaba disponible en los artefactos de la versión de cierre.

La tercera conclusión es que el batch reforzó la reproducibilidad de Agent 1. Los modos semanal y mensual registran snapshots de entrada, hashes, manifests, estado y un plan de reconstrucción downstream. El batch evita reprocesar Agent 1 cuando no hay cambios, pero no ejecuta automáticamente los agentes posteriores. El freeze final queda como criterio metodológico pendiente de materializar en un artefacto específico.

La cuarta conclusión es que el sistema desarrollado permite sostener la defensa del proyecto con soporte documental y técnico. La arquitectura, las limitaciones, las métricas y los manifests de lote quedaron alineados con los artefactos generados, de forma que cada afirmación relevante puede rastrearse a una salida del sistema o a una decisión de diseño documentada.

El benchmark final refuerza esta conclusión con una lectura agregada. Sin solicitar el dashboard, 34 métricas producen 31 `pass`, 1 `warning`, 0 `fail` y 2 `not_applicable`; con `--include-dashboard`, las mismas 34 métricas producen 32 `pass`, 1 `warning`, 0 `fail` y 1 `not_applicable`. El único `warning` corresponde al matching interfuente, no a un fallo de

ejecución. Esta separación permite afirmar que el prototipo es técnicamente funcional y medible, al mismo tiempo que mantiene visible la principal limitación metodológica para una versión institucional.

La evaluación relacional convierte la existencia de comunidades en un resultado cuantitativo: Agent 3 obtiene $Q = 0,6051129926$ en el grafo de la muestra y $Q = 0,3165680473$ en la demo integrada. Ambos valores superan el objetivo $Q > 0,30$. Esta evidencia respalda la calidad estructural de la partición Louvain en los grafos evaluados, sin convertir las comunidades en prueba de fraude ni extrapolar el resultado al canónico completo.

La evaluación cualitativa de diez fichas amplía esa evidencia: cinco casos de score máximo, tres de riesgo medio y dos controles conservaron trazabilidad de fuente y relaciones; las 21 activaciones de reglas quedaron justificadas por sus registros de evidencia. La ausencia de documentos asociados en el corpus actual se mantuvo visible en las diez fichas. Este resultado respalda la capacidad de priorizar y explicar, pero no sustituye una validación con etiquetas de fraude, documentos reales ampliados o revisión experta.

La quinta conclusión es que el bloque de extensión relacional y documental no quedó como una idea pendiente, sino como una integración demostrable. Agent 3 y Agent 4 dejaron una demo conjunta cerrada, compatible con el contrato canónico, la evidencia textual y el *score* procedente de Agent 2. Eso permitió explicar en la memoria un flujo completo de priorización, análisis relacional y justificación documental sin recurrir a formulaciones de futuro.

Entre las limitaciones cerradas, la más relevante fue la ausencia de intersecciones útiles entre fuentes para afirmar un cruce contractual fuerte por `contract_key_canon`. También quedó documentado que parte de las métricas de Agent 1 no alcanzaron el objetivo planteado y que la evaluación de Agent 2 se apoyó en una referencia proxy por la ausencia de etiquetas reales de fraude. Lejos de ser un defecto narrativo, estas limitaciones delimitan con precisión el alcance real del prototipo y evitan sobredimensionar sus resultados.

El valor principal del trabajo reside en haber convertido un problema amplio y heterogéneo en una arquitectura ejecutable, trazable y explicable. La combinación de canónico OCDS adaptado, reglas de riesgo, grafo, evidencia documental, dashboard y batch reproducible proporciona una base suficiente para demostrar cómo podría organizarse una revisión analítica de contratación pública sin depender de decisiones opacas. Esta aportación es especialmente relevante porque mantiene separadas tres capas que a menudo se confunden: dato observado, señal de riesgo e interpretación experta.

El uso correcto del sistema exige conservar esa separación. ProcureWatch Analytics no debe utilizarse para automatizar sanciones, excluir proveedores o afirmar irregularidades sin revisión. Su función es reducir el espacio de búsqueda, ordenar casos, mostrar por qué se han priorizado y dejar constancia de los datos que faltan. Desde esta perspectiva, el prototipo no es un producto cerrado de auditoría, sino una plataforma académica reproducible sobre la que pueden incorporarse nuevas reglas, validación experta, corpus documental ampliado y despliegue institucional.

En conjunto, el TFM cerró un prototipo coherente de extremo a extremo: ingesta, normalización, scoring, grafo, recuperación documental, dashboard y reproducibilidad. La ejecución documentada de Agent 1 genera 51.720 registros canónicos; Agent 2 produce puntuaciones explicables mediante RF-01 a RF-06; la demo integrada muestra un caso completo con `risk_score=0.5`, dos señales de riesgo, 11 nodos, 13 relaciones, 2 comunidades, modularidad $Q = 0,3165680473$, 2 evidencias documentales y 2 citas; y la evaluación cualitativa publica diez fichas con composición 5/3/2 y evidencia completa para 21 activaciones de reglas; la validación técnica recoge 122 tests superados y 1 omitido en el entorno mínimo, y 123 tests superados sin omisiones con SQLAlchemy disponible; el benchmark global no presenta fallos y el dashboard Streamlit sin excepciones en modo headless. El valor central del trabajo se sostiene, por tanto, sobre un flujo operativo implementado, trazable y documentado, sin afirmar detección automática de fraude.

Referencias

- Agencia Tributaria de España. (2023). *Desarticulada una trama de fraude en la contratación de obra pública en cantabria*. Descargado de https://sede.agenciatributaria.gob.es/Sede/Desarticulada_una_trama_de_fraude_en_la_contratacion_de_obra_publica_en_Cantabria.htm
- Alibaba Cloud. (2024). *Qwen3: Technical report*. Descargado de <https://huggingface.co/Qwen>
- Almeida, J., y cols. (2025). Data-driven transparency: Machine learning and social network analysis for corruption detection in public procurement. *Procedia Computer Science*. Descargado de <https://www.sciencedirect.com/science/article/pii/S1877050925029722>

- BAAI. (2024). *BGE-M3: Multi-functionality, multi-linguality, multi-granularity text embeddings*. Descargado de <https://huggingface.co/BAAI/bge-m3>
- Blondel, V., y cols. (2024). Fast leiden algorithm for community detection in shared memory. En *Proceedings of the 53rd international conference on parallel processing (icpp 2024)*. ACM. Descargado de <https://dl.acm.org/doi/10.1145/3673038.3673146> doi: 10.1145/3673038.3673146
- Coviello, D., y Mariniello, M. (2023). Corruption red flags in public procurement: New evidence from Italian calls for tenders. *EPJ Data Science*, 11(1), 7. Descargado de <https://link.springer.com/article/10.1140/epjds/s13688-022-00325-x> doi: 10.1140/epjds/s13688-022-00325-x
- Docker Inc. (2025). *Docker Compose documentation*. Descargado de <https://docs.docker.com/compose/>
- DuckDB Foundation. (2025). *DuckDB documentation*. Descargado de <https://duckdb.org/docs/>
- European Parliament and Council. (2014). *Directive 2014/24/EU of the European Parliament and of the council of 26 february 2014 on public procurement*. Descargado de <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32014L0024>
- European Parliament and Council. (2016). *Regulation (eu) 2016/679 of the european parliament and of the council of 27 april 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (general data protection regulation)*. Descargado de <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679>
- European Parliament and Council. (2019). *Directive (eu) 2019/1024 of the european parliament and of the council of 20 june 2019 on open data and the re-use of public sector information*. Descargado de <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32019L1024>
- European Parliament and Council. (2022). *Directive (eu) 2022/2557 of the european parliament and of the council of 14 december 2022 on the resilience of critical entities (cer directive)*. Descargado de <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32022L2557>
- European Parliament and Council. (2024). *Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act)*. Descargado de <https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32024R1689>

- Falcón-Cortés, A., Aldana, A., y Larralde, H. (2021). Practices of public procurement and the risk of corrupt behavior. *arXiv preprint*. Descargado de <https://arxiv.org/abs/2108.02653>
- FastAPI. (2025). *FastAPI documentation*. Descargado de <https://fastapi.tiangolo.com/>
- Fazekas, M., y Cingolani, L. (2020). Uncovering high-level corruption: Cross-national objective corruption risk indicators using government contracting data. *British Journal of Political Science*, 50(1), 155–164. Descargado de https://ideas.repec.org/a/cup/bjposi/v50y2020i1p155-164_7.html doi: 10.1017/S0007123419000425
- Fazekas, M., y Cingolani, L. (2025). *Opentender: Public procurement data for transparency and accountability* (Inf. Téc.). Government Transparency Institute. Descargado de <https://www.govtransparency.eu/opentender/>
- Fazekas, M., y cols. (2025). Detection of fraud in public procurement: A systematic mapping study. *EPJ Data Science*. Descargado de <https://link.springer.com/article/10.1140/epjds/s13688-025-00569-3> doi: 10.1140/epjds/s13688-025-00569-3
- Government Transparency Institute. (2021). *DIGIWHIST: Digital whistleblowing as a tool for reporting corruption in the EU* (Inf. Téc.). European Commission, Horizon 2020. Descargado de <https://digiwhist.eu/>
- Government Transparency Institute. (2024). *Opentender.eu — european public procurement data portal*. Descargado de <https://opentender.eu/>
- Hagberg, A. A., Schult, D. A., y Swart, P. J. (2008). *NetworkX: Network analysis in python*. Descargado de <https://networkx.org/>
- Honnibal, M., Montani, I., y Van Landeghem, S. (2020). *spaCy: Industrial-strength natural language processing in python*. Descargado de <https://spacy.io/>
- Ivalua. (2024). *Procurement dashboard: Kpis, benchmarks and visual tips*. Descargado de <https://www.ivalua.com/blog/procurement-dashboard/>
- Jefatura del Estado. (2013). *Ley 19/2013, de 9 de diciembre, de transparencia, acceso a la información pública y buen gobierno*. Descargado de <https://www.boe.es/eli/es/1/2013/12/09/19> (BOE núm. 295, de 10 de diciembre de 2013)
- Jefatura del Estado. (2017). *Ley 9/2017, de 8 de noviembre, de contratos del sector público*. Descargado de <https://www.boe.es/eli/es/1/2017/11/08/9/con> (BOE núm. 272, de 9 de noviembre de 2017)

- Jefatura del Estado. (2018). *Ley orgánica 3/2018, de 5 de diciembre, de protección de datos personales y garantía de los derechos digitales*. Descargado de <https://www.boe.es/eli/es/lo/2018/12/05/3> (BOE núm. 294, de 6 de diciembre de 2018)
- Kalamkar, P., y cols. (2024). Natural language processing for the legal domain: A survey of tasks, datasets, and techniques. *arXiv preprint*. Descargado de <https://arxiv.org/pdf/2410.21306>
- LangChain AI. (2024). *LangGraph: Build stateful, multi-actor applications with llms*. Descargado de <https://github.com/langchain-ai/langgraph>
- LangChain AI. (2025). *LangChain documentation*. Descargado de <https://python.langchain.com/docs/>
- Leitner, E., y cols. (2020). Fine-grained named entity recognition in legal documents. En *Proceedings of the 17th extended semantic web conference (eswc 2020)*. Springer. Descargado de https://link.springer.com/chapter/10.1007/978-3-030-33220-4_20 doi: 10.1007/978-3-030-33220-4_20
- Liu, Y., y cols. (2023). Pattern mining for anomaly detection in graphs: Application to fraud in public procurement. *Lecture Notes in Computer Science*. Descargado de <https://arxiv.org/abs/2306.10857> doi: 10.1007/978-3-031-43427-3_5
- Martins, P., y cols. (2022). Network analysis for fraud detection in Portuguese public procurement. *ResearchGate Preprint*. Descargado de https://www.researchgate.net/publication/346484234_Network_Analysis_for_Fraud_Detection_in_Portuguese_Public_Procurement
- Ministerio de Hacienda y Función Pública. (2024). *Plataforma de contratación del sector público: datos abiertos de licitaciones* (Inf. Téc.). Gobierno de España. Descargado de https://www.hacienda.gob.es/es-ES/GobiernoAbierto/DatosAbiertos/Paginas/licitaciones_plataforma_contratacion.aspx
- Mistral AI. (2025). *Mistral small 3*. Descargado de <https://mistral.ai>
- Muñoz Plà, M. (2026). A decade of public procurement in spain: Extraction, structuring, and analysis of boe data (2014–2024). *arXiv, 2602.16731v1*. Descargado de <https://doi.org/10.5281/zenodo.15120882> (CC BY 4.0) doi: 10.5281/zenodo.15120882
- Nallakaruppan, M. K., y cols. (2025). A perspective on explainable artificial intelligence methods: SHAP and LIME. *Advanced Intelligent Systems*. Descargado de <https://advanced.onlinelibrary.wiley.com/doi/10.1002/aisy.202400304> doi: 10.1002/aisy.202400304

- Neo4j. (2025). *Neo4j documentation*. Descargado de <https://neo4j.com/docs/>
- Ng'ang'a, K. (2025). A hybrid machine learning model for detecting and preventing corruption in Kenya's public procurement contracts. *Machine Learning Research*, 10(2). Descargado de <https://www.sciencepublishinggroup.com/article/10.11648/j.mlr.20251002.14> doi: 10.11648/j.mlr.20251002.14
- Noé, J., y cols. (2020). GraphSIF: Analyzing flow of payments in b2b networks for supplier impersonation detection. *Applied Network Science*, 5, 70. Descargado de <https://appliednetsci.springeropen.com/articles/10.1007/s41109-020-00283-1> doi: 10.1007/s41109-020-00283-1
- Ocampo-Gutiérrez, F., y cols. (2019). Anomaly detection in public procurements using the open contracting data standard. En *Ceur workshop proceedings* (Vol. 2369). Descargado de <https://ceur-ws.org/Vol-2369/short09.pdf>
- OECD. (2016). *Preventing corruption in public procurement*. OECD Publishing. doi: 10.1787/9789264251762-en
- Oficina Independiente de Regulación y Supervisión de la Contratación. (2025). *Informe anual de supervisión de la contratación pública (ias 2024)*. Descargado de <https://www.hacienda.gob.es/es-ES/Oirescon/Paginas/ias.aspx>
- Ollama. (2025). *Ollama: Run large language models locally*. Descargado de <https://ollama.com/>
- Open Contracting Partnership. (2016). *Red flags for integrity: Giving the green light to open data solutions* (Inf. Téc.). Open Contracting Partnership. Descargado de <https://www.open-contracting.org/wp-content/uploads/2016/11/OCP2016-Red-flags-for-integrityshared-1.pdf>
- Open Contracting Partnership. (2023). *How open is public procurement data in the eu? 2023* (Inf. Téc.). Open Contracting Partnership. Descargado de <https://www.open-contracting.org/wp-content/uploads/2023/06/OCP2023-EU-OpenData.pdf>
- Open Contracting Partnership. (2024a). *Open contracting data standard v1.1.5* (Inf. Téc.). Open Contracting Partnership. Descargado de <https://standard.open-contracting.org/>
- Open Contracting Partnership. (2024b). *Red flags in public procurement: A guide to using data to detect corruption and fraud* (Inf. Téc.). Open Contracting Partnership. Descargado de <https://www.open-contracting.org/wp-content/uploads/2024/12/OCP2024-RedFlagProcurement-1.pdf>

- Pedregosa, F., y cols. (2011). *Scikit-learn: Machine learning in python — IsolationForest*. Descargado de <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.IsolationForest.html>
- PkgPulse. (2026). *Cytoscape.js vs vis-network vs sigma.js: Graph visualization comparison 2026*. Descargado de <https://www.pkgpulse.com/blog/cytoscape-vs-vis-network-vs-sigma-graph-visualization-javascript-2026>
- Plotly Technologies Inc. (2025). *Plotly python graphing library*. Descargado de <https://plotly.com/python/>
- Polars. (2025). *Polars documentation*. Descargado de <https://docs.pola.rs/>
- Portal de Contratación Pública. (2026). *Lista de códigos CPV 71x: Servicios de arquitectura, construcción, ingeniería e inspección*. Descargado de <https://www.publictendering.com/cpv-codes/lista-de-los-codigos-cpv/71x/>
- PostgreSQL Global Development Group. (2025). *PostgreSQL documentation*. Descargado de <https://www.postgresql.org/docs/>
- PyVis contributors. (2025). *PyVis: Interactive network visualizations in python*. Descargado de <https://pyvis.readthedocs.io/>
- Qdrant. (2025). *Qdrant documentation*. Descargado de <https://qdrant.tech/documentation/>
- Qwen Team. (2025). Qwen3 technical report. *arXiv preprint*. Descargado de <https://arxiv.org/abs/2505.09388>
- Ragas contributors. (2025). *Ragas: Evaluation framework for retrieval augmented generation*. Descargado de <https://docs.ragas.io/>
- Richardson, L. (2024). *Beautiful soup 4: A python library for pulling data out of html and xml files*. Descargado de <https://www.crummy.com/software/BeautifulSoup/> (Versión 4.x)
- Sigma.js contributors. (2025). *Sigma.js: A javascript library dedicated to graph drawing*. Descargado de <https://www.sigmapjs.org/>
- Snowflake Inc. (2025). *Streamlit documentation*. Descargado de <https://docs.streamlit.io/>
- Su, X., y cols. (2021). Positive-unlabeled learning from imbalanced data. En *Proceedings of the 30th international joint conference on artificial intelligence (ijcai-21)*. Descargado de <https://www.ijcai.org/proceedings/2021/412>
- Traag, V. A., Waltman, L., y van Eck, N. J. (2019). From Louvain to Leiden: guaranteeing

well-connected communities. *Scientific Reports*, 9(1), 5233. Descargado de <https://www.nature.com/articles/s41598-019-41695-z> doi: 10.1038/s41598-019-41695-z

- Wachs, J., Fazekas, M., y Kertész, J. (2025). A machine learning and network science approach to identifying corruption risks in public procurement. *arXiv preprint*. Descargado de <https://arxiv.org/abs/2512.19491>
- World Bank. (2021). *Open contracting data standard: Implementation guide* (Inf. Téc.). World Bank Group. Descargado de <https://documents1.worldbank.org/curated/en/744551614955316901/pdf/Open-Contracting-Data-Standard.pdf>
- Zhang, W., y cols. (2023a). Fraud detection via contrastive positive unlabeled learning. *IEEE Conference Publication*. Descargado de <https://ieeexplore.ieee.org/document/10020693/> doi: 10.1109/ICDM54844.2023.00001
- Zhang, W., y cols. (2023b). Group-based fraud detection network on e-commerce platforms. En *Proceedings of the 29th acm sigkdd conference on knowledge discovery and data mining (kdd 2023)*. ACM. Descargado de https://cgi.cse.unsw.edu.au/~zhangw/files/2023_KDD_paper.pdf

A. Apéndice: Tipos de contrato incluidos en el análisis

El análisis se centra en las siguientes tipologías contractuales del BOE 2014–2024 y PLACE, filtradas por la división CPV 71:

Tabla A.1: Subtipos CPV 71 relevantes para ProcureWatch Analytics

Código CPV	Descripción
71000000	Servicios de arquitectura, construcción, ingeniería e inspección
71311000	Consultoría en ingeniería civil
71312000	Consultoría en ingeniería de estructuras
71313000	Consultoría en ingeniería medioambiental
71314000	Servicios de energía e ingeniería afines
71320000	Servicios de diseño técnico
71330000	Servicios varios de ingeniería
71340000	Servicios integrados de ingeniería
71350000	Servicios científicos y técnicos en ingeniería
71356000	Servicios técnicos
71500000	Servicios relacionados con la construcción
71600000	Servicios de pruebas, inspección y control
71700000	Servicios de supervisión e inspección

B. Apéndice: Estructura técnica del proyecto

La estructura técnica del proyecto de implementación se resume en la Tabla B.1. No se listan datasets pesados ni artefactos temporales, ya que estos se mantienen fuera de Git por reproducibilidad y control de tamaño.

Tabla B.1: Estructura funcional del proyecto de implementación

Ruta	Función
<code>scr/procurewatch/cli.py</code>	CLI central del proyecto.
<code>scr/procurewatch/batch.py</code>	Orquestación de lotes, manifests y estado de ejecución.
<code>scr/procurewatch/settings.py</code>	Lectura de configuración y disponibilidad de servicios.
<code>scr/procurewatch/db.py</code>	Persistencia PostgreSQL de tablas analíticas.
<code>scr/procurewatch/data_sources/</code>	Parsers y conectores BOE, PLACE y OpenTender.
<code>scr/procurewatch/agent1/</code>	Ingesta, normalización, esquema analítico, calidad y cobertura.
<code>scr/procurewatch/agent2/</code>	Red flags, scoring, agregación y reportes.
<code>scr/procurewatch/agent3/</code>	Grafo, métricas de red, comunidades, features y Neo4j opcional.
<code>scr/procurewatch/agent4/</code>	Corpus, chunking, retrieval, Qdrant/Ollama opcional, contexto de caso y evaluación.
<code>frontend/agent3_demo.py</code>	Dashboard Streamlit integrado para exploración del prototipo.
<code>tests/</code>	Pruebas automatizadas de agentes, fuentes, batch, CLI y servicios auxiliares.
<code>docs/</code>	Hojas de ruta, seguimiento, planes técnicos, modelo de datos y decisiones de cierre.
<code>docker-compose.yml</code>	Servicios locales PostgreSQL, Neo4j y Qdrant.
<code>pyproject.toml</code>	Dependencias, extras, entrypoint de CLI y configuración de tests.

C. Apéndice: Esquema analítico resumido

El esquema analítico implementado sigue la estructura mínima definida para el TFM. La Tabla C.1 agrupa los campos por bloque funcional.

Tabla C.1: Bloques funcionales del esquema CONTRATO

Bloque	Campos principales
Identificación	id_contrato, id_licitacion, fuentes_cruzadas, estado_revision.
Organismo	organismo_contratante, codigo_organismo, nivel_administracion.
Contrato	tipo_contrato, procedimiento, cpv_codigo, cpv_descripcion.
Importes	importe_estimado, importe_adjudicado, ratio_desviacion_importe.
Fechas	fecha_publicacion, fecha_adjudicacion, dias_resolucion.
Concurrencia	numero_ofertas_recibidas.
Adjudicatario	id_adjudicatario, nif_adjudicatario, nombre_adjudicatario.
Riesgo	score_red_flags_total, red_flags_activados, nivel_riesgo.
Grafo	score_centralidad_red, comunidad_red.
Documental	fragmentos_documentales_recuperados.

El esquema ADJUDICATARIO recoge NIF, nombre, forma jurídica, sector, total de contratos, importe adjudicado, organismos distintos, ratio de procedimientos menores, tasa de adjudicación, riesgo agregado, centralidad, comunidades y red flags recurrentes. Esta separación permite evaluar riesgo por contrato y riesgo acumulado por proveedor sin mezclar unidades de análisis.

D. Apéndice: Ejemplo de report y manifest

Los reports de ejecución se almacenan como JSON para facilitar auditoría y comparación. Un report de Agent1 contiene, de forma resumida, las fuentes procesadas, versiones de parser, filas leídas, filas normalizadas, errores y rutas de salida. El siguiente ejemplo reproduce la forma del artefacto sin incluir hashes reales completos:

```
{
  "run_at": "2026-06-23T00:01:04Z",
  "parser_versions": {
    "agent1": "1.0.0",
    "boe": "1.0.0",
    "place": "1.0.0",
    "opentender": "1.3.0"
  },
  "boe": {
    "total_data_lines": 97154,
    "parsed_rows": 96809,
    "parse_errors": 345,
    "cpv71_rows": 8097
  },
  "outputs": {
    "canonical": "data/processed/agent2_contracts_canonical.parquet",
    "quality": "data/processed/agent1_data_quality_summary.json"
  }
}
```

El manifest de batch permite saber si una ejecución procesó datos nuevos o se saltó por no haber cambios. Su estructura mínima es:

```
{
  "batch_id": "monthly-2026-06",
```



```
"run_mode": "monthly",
"status": "updated",
"sources_checked": ["boe", "place", "opentender"],
"sources_changed": ["place"],
"input_hashes": {
  "place": "sha256:..."
},
"output_hashes": {
  "canonical": "sha256:...",
  "agent2_scores": "sha256:..."
},
"versions": {
  "code_version": "version-de-cierre",
  "schema_version": "agent1-analytical-v1",
  "scoring_version": "2.0.0"
}
}
```

Este diseño facilita responder a una pregunta clave de auditoría: con qué datos, reglas y versión de código se generó un determinado score.

E. Apéndice: Comandos de validación

Los comandos siguientes resumen la validación técnica documentada para la versión de cierre. Se incluyen como referencia reproducible; la disponibilidad de servicios opcionales depende de la configuración local.

```
PYTHONPATH=scr python -m procurewatch.cli doctor
PYTHONPATH=scr python -m pytest tests
PYTHONPATH=scr python -m ruff check scr tests
PYTHONPATH=scr python -m procurewatch run-benchmark \
  --processed-dir data/processed_sample \
  --demo-dir data/processed/agent3_agent4_demo_2026_06_23 \
  --agent4-evaluation data/processed/agent4_evaluation_report.json \
  --output-dir data/processed/benchmark
```

Para ejecutar los agentes principales:

```
PYTHONPATH=scr python -m procurewatch.cli run-agent1 --year 2024 --cpv-prefix 71
PYTHONPATH=scr python -m procurewatch.cli run-agent1 --year 2024 --cpv-prefix 71 --place
PYTHONPATH=scr python -m procurewatch report-agent1-source-diagnostics \
  --output-dir data/processed_sample --cpv-prefix 71 --year 2024
PYTHONPATH=scr python -m procurewatch.cli run-agent2-mvp \
  --input data/processed/agent2_contracts_canonical.parquet \
  --output-dir data/processed
PYTHONPATH=scr python -m procurewatch.cli run-agent3 \
  --input data/processed/agent2_contracts_canonical.parquet \
  --output-dir data/processed
PYTHONPATH=scr python -m procurewatch.cli agent4-build-manifest
PYTHONPATH=scr python -m procurewatch.cli agent4-run-flow
PYTHONPATH=scr python -m procurewatch.cli agent4-evaluate
```

Para la demo integrada Agent3-Agent4:

```
PYTHONPATH=scr python -m procurewatch.cli run-integrated-demo
PYTHONPATH=scr python -m procurewatch.cli validate-dashboard-demo
PYTHONPATH=scr python -m procurewatch.cli run-agent3 \
  --input data/processed/agent3_agent4_demo_2026_06_23/agent2_contracts_canonical_demo.
  --output-dir data/processed/agent3_agent4_demo_2026_06_23
```

```
PYTHONPATH=scr python -m procurewatch.cli agent4-case-context \
  --contract-key PW-2024-0001 \
  --question "evidencia documental y riesgos explicables" \
  --canonical-path data/processed/agent3_agent4_demo_2026_06_23/agent2_contracts_canoni
  --agent3-features-path data/processed/agent3_agent4_demo_2026_06_23/agent3_agent2_fea
  --output data/processed/agent3_agent4_demo_2026_06_23/agent4_case_context_integrated_
```

Para visualizar la demo:

```
PYTHONPATH=scr \
PROCUREWATCH_AGENT3_DEMO_DIR=data/processed/agent3_agent4_demo_2026_06_23 \
PROCUREWATCH_AGENT4_CASE_CONTEXT=data/processed/agent3_agent4_demo_2026_06_23/agent4_ca
streamlit run frontend/agent3_demo.py
```

Para ejecutar lotes incrementales:

```
PYTHONPATH=scr python -m procurewatch.cli run-batch --run-mode weekly --year 2024 --cpv
PYTHONPATH=scr python -m procurewatch.cli run-batch --run-mode monthly --year 2024 --cpv
```